INTEGRAÇÃO E ENTREGA CONTÍNUA

Relatório de evidências
Etapas 1 a 7

Projeto Cadastro de Pessoas em JavaScript

Preparação, ferramentas, colaboração, proposta de API, TDD do backend e do frontend, CRUD com JSON e integração no navegador.

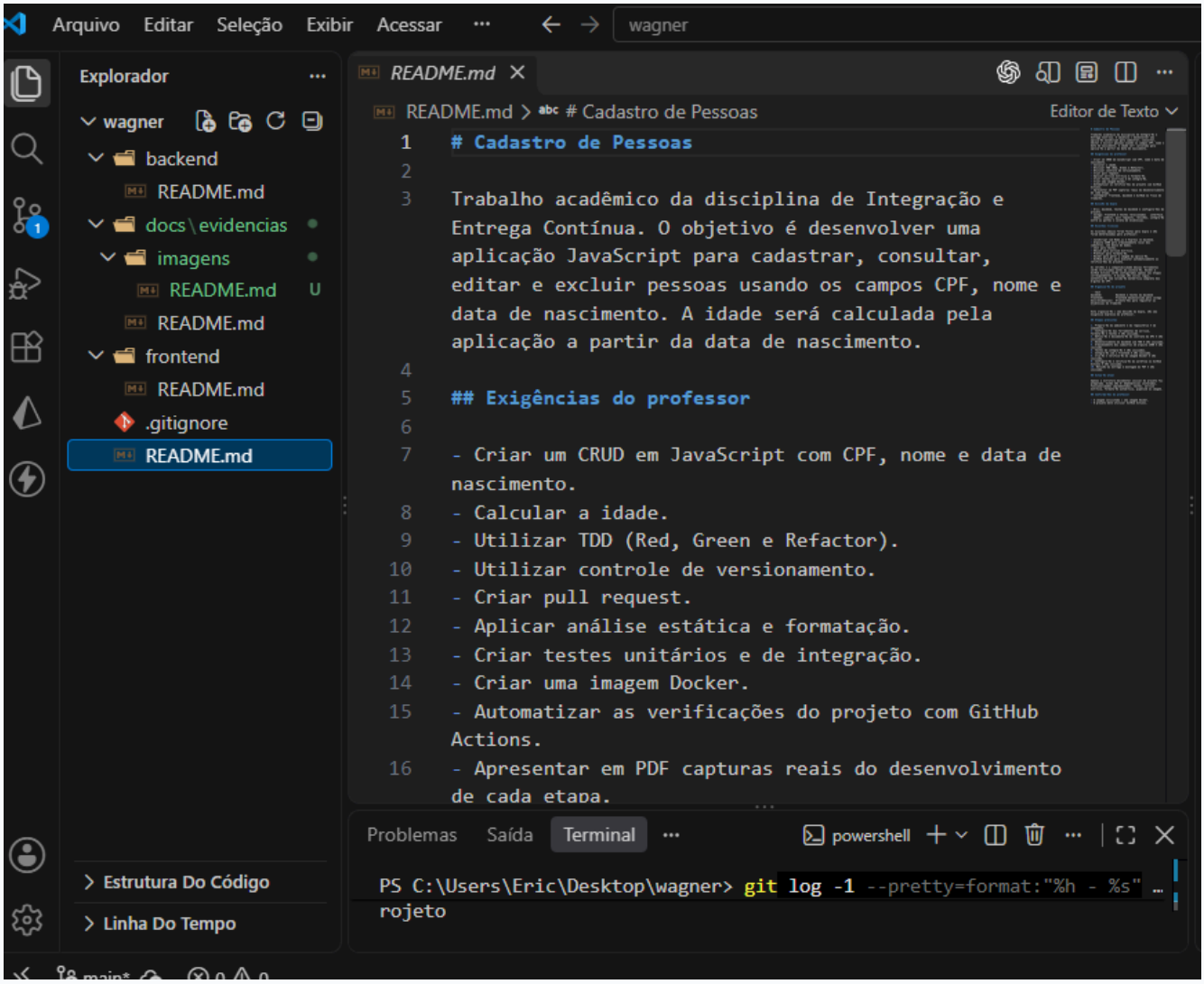
Situação: PR #8 aberto; 46 testes do backend e 24 do frontend passaram. A formatação geral ainda tem pendências. Docker e GitHub Actions serão documentados em etapas futuras.

22 de outubro de 2026

ETAPA 1

Estrutura inicial

Pastas do backend, frontend, documentação e evidências foram organizadas no projeto.

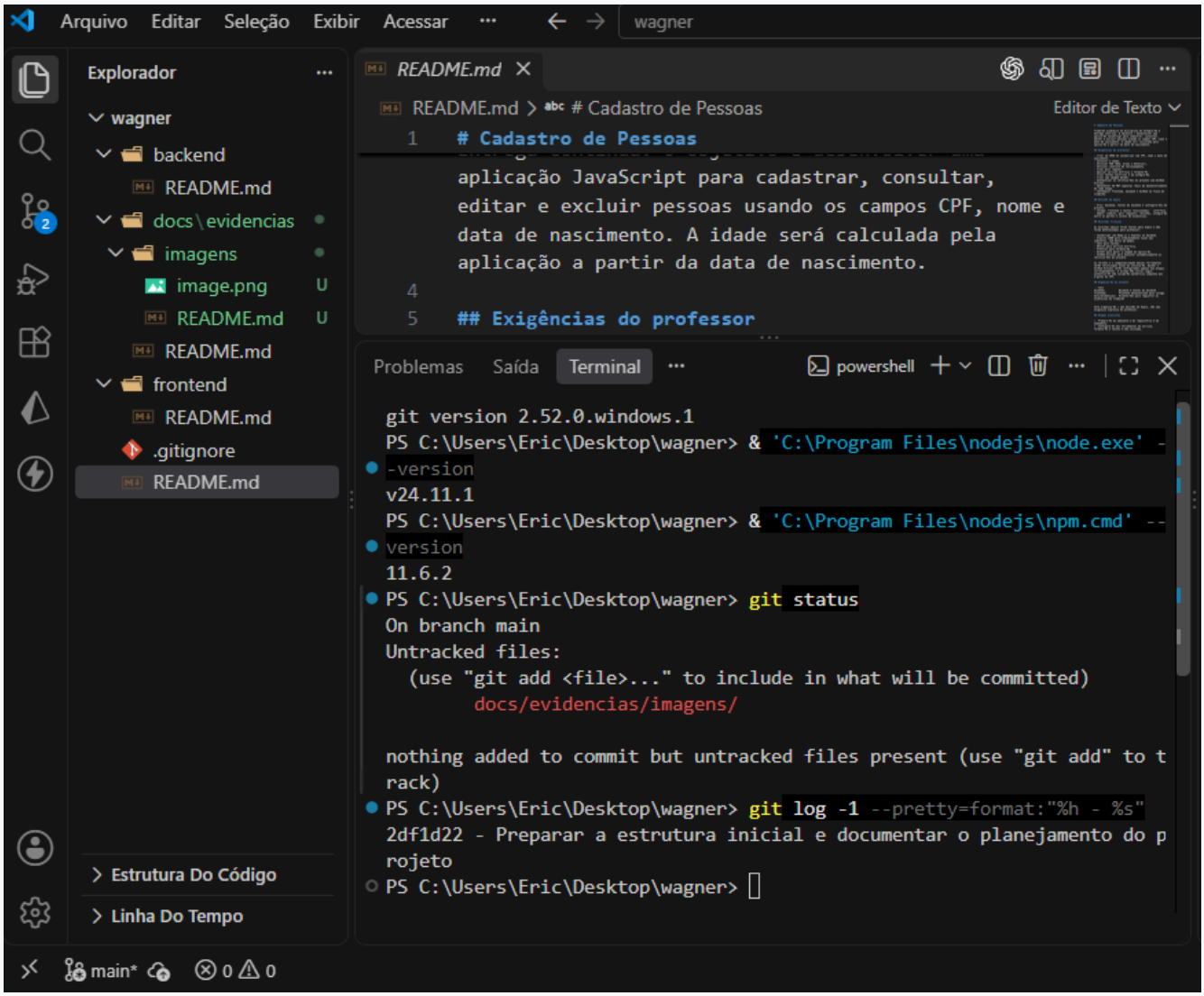


Evidência 01

ETAPA 1

Ambiente e primeiro commit

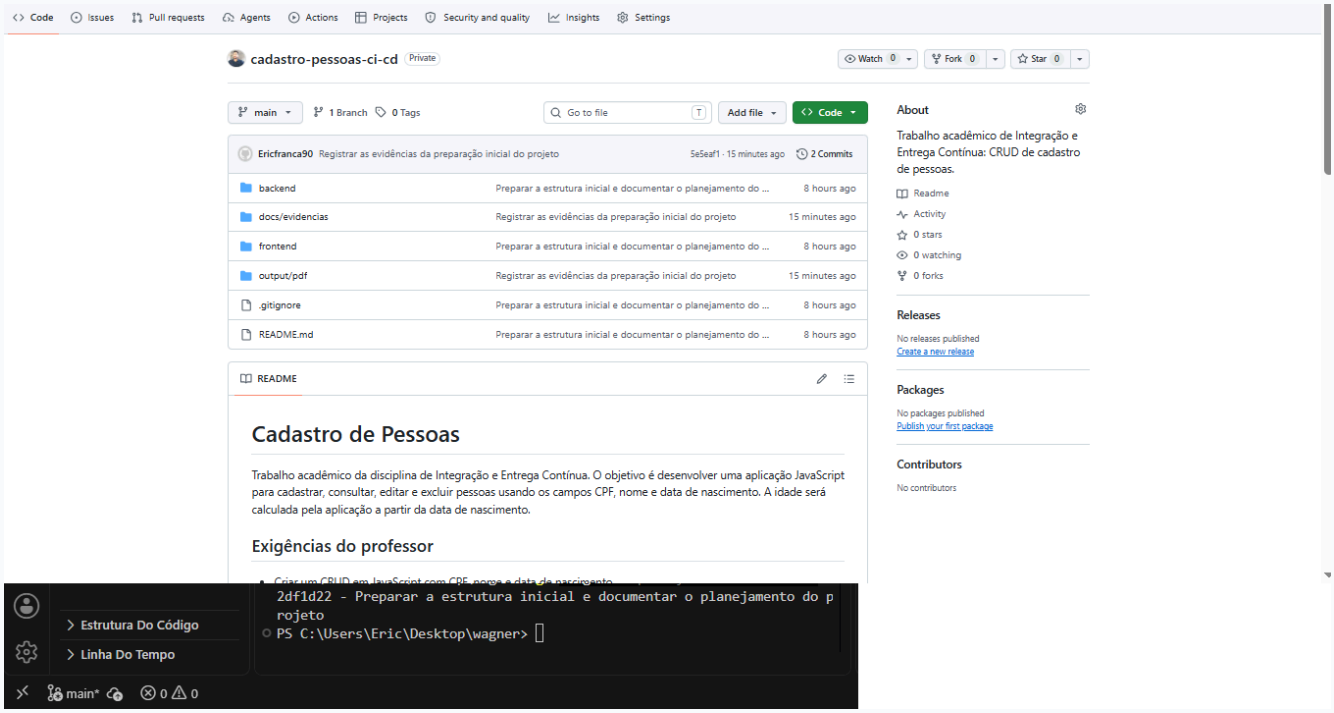
Foram conferidas as ferramentas instaladas e registrado o primeiro commit da estrutura planejada.



ETAPA 1

Repositório publicado

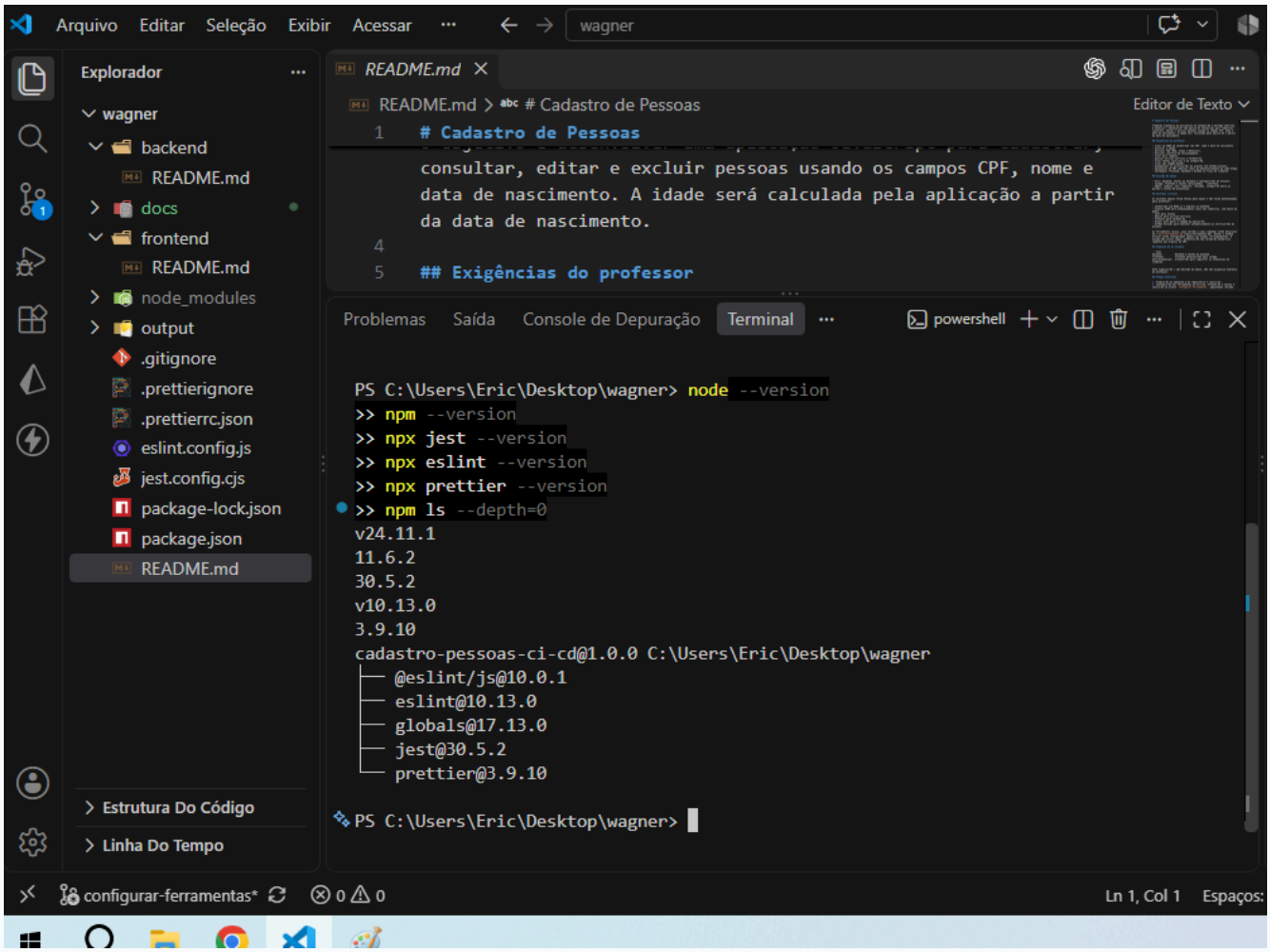
O projeto foi publicado no GitHub para permitir o trabalho em dupla e o controle de versões.



ETAPA 2

Versões e dependências

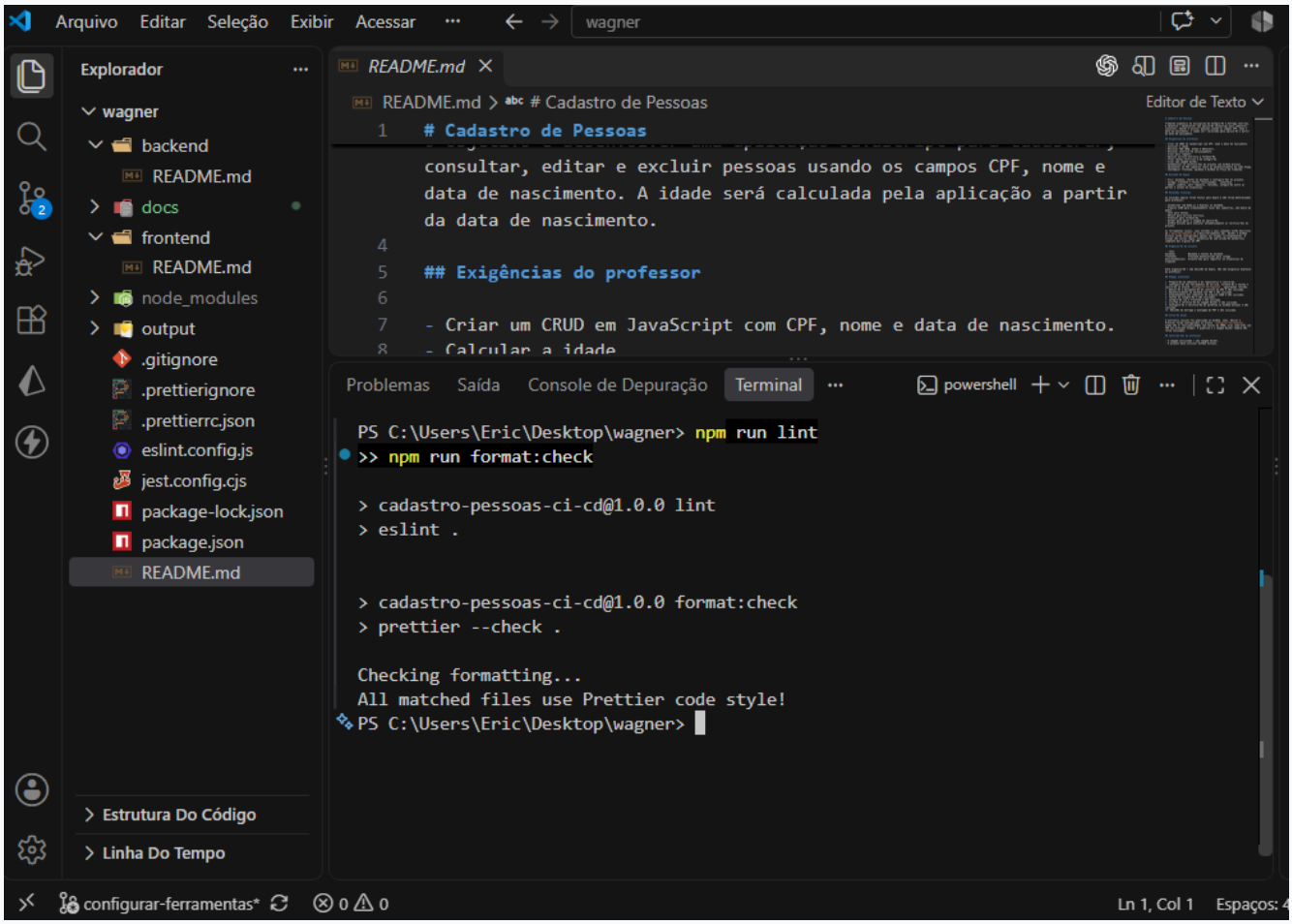
Node.js, npm, Jest, ESLint e Prettier foram conferidos, junto com as dependências instaladas.



ETAPA 2

Análise e formatação

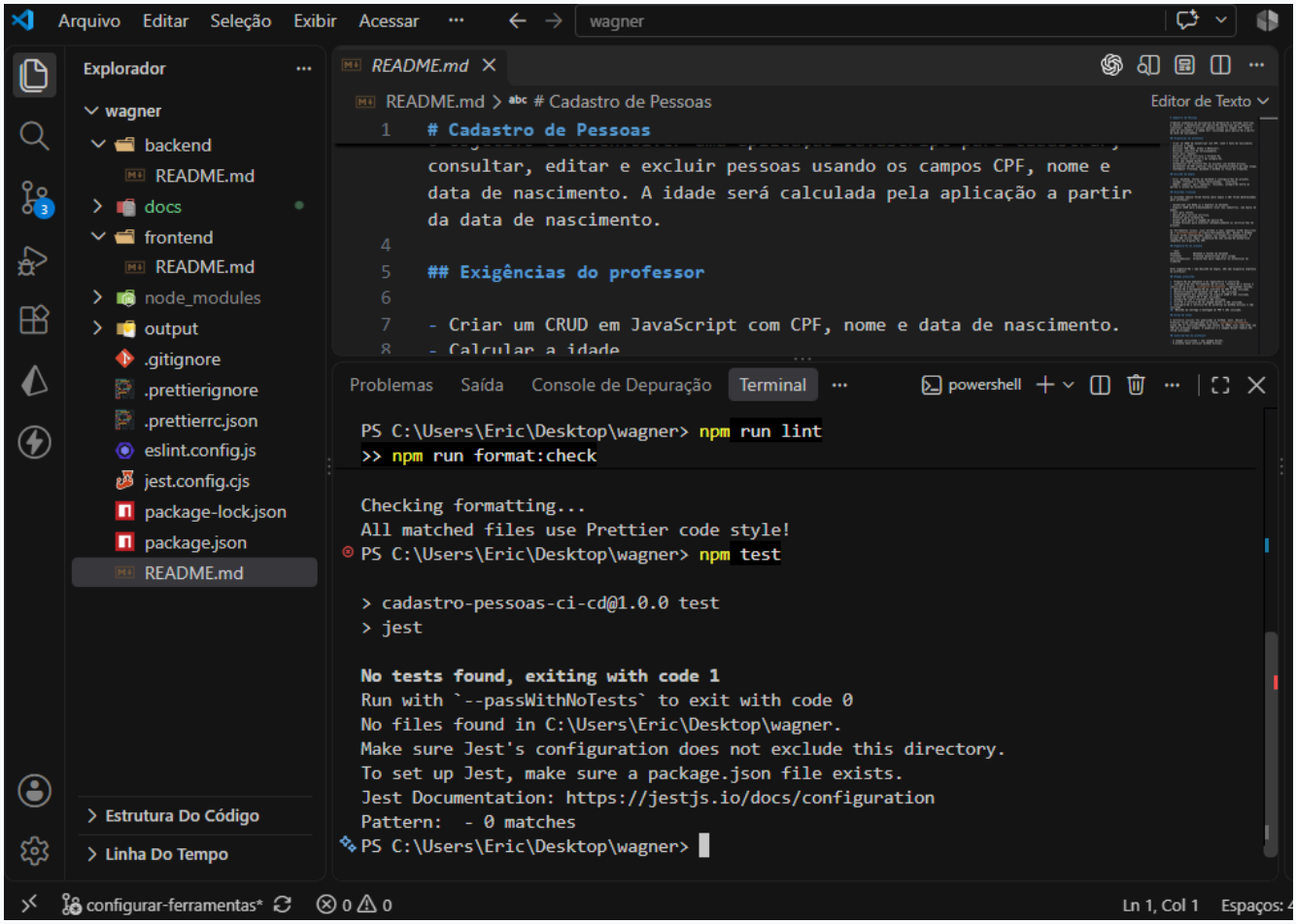
O ESLint terminou sem erros e o Prettier confirmou que os arquivos seguem a formatação definida.



ETAPA 2

Jest antes dos testes

O Jest foi executado e informou que ainda não existem testes, situação esperada antes do início do TDD.

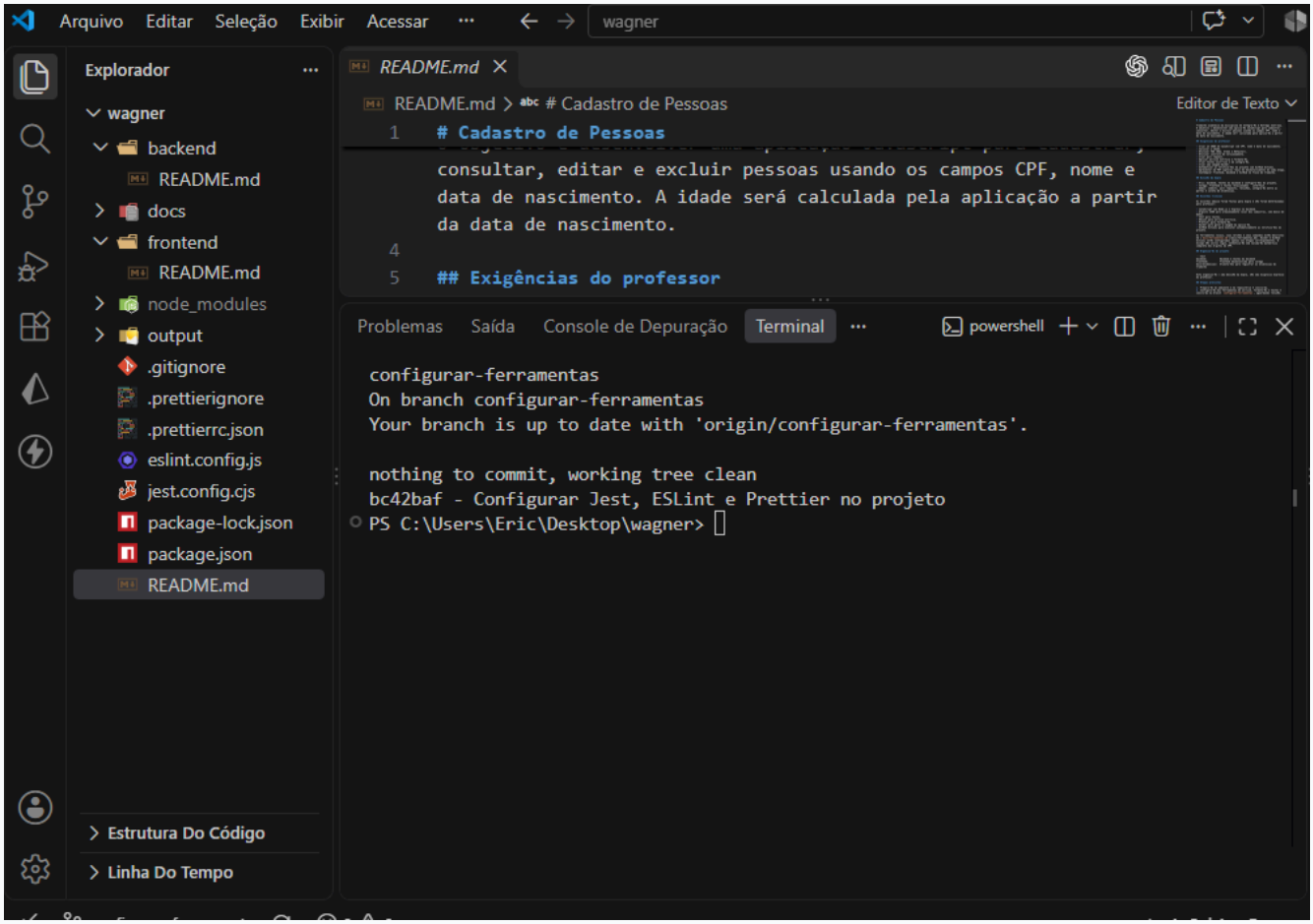


Evidência 06

ETAPA 2

Branch e commit

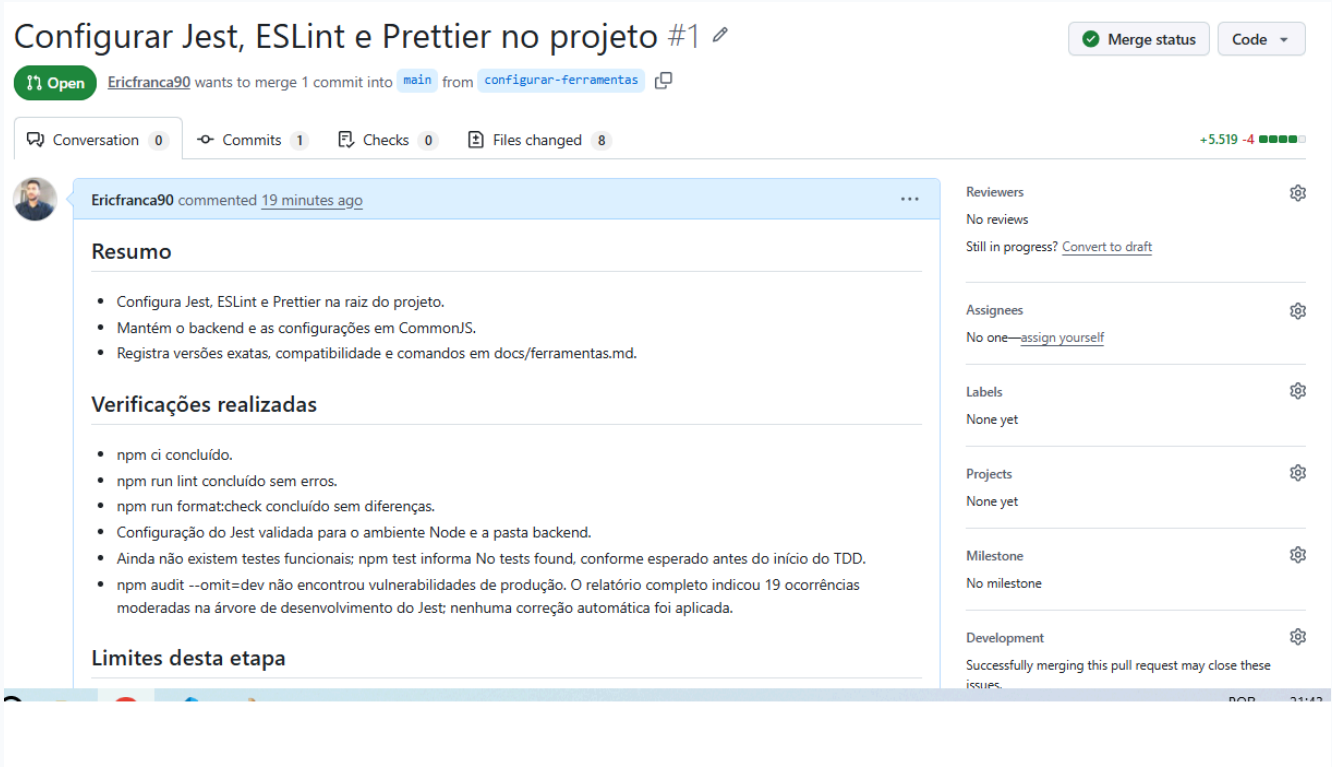
As configurações foram registradas na branch configurar-ferramentas com um commit descritivo.



ETAPA 2

Pull request aberto

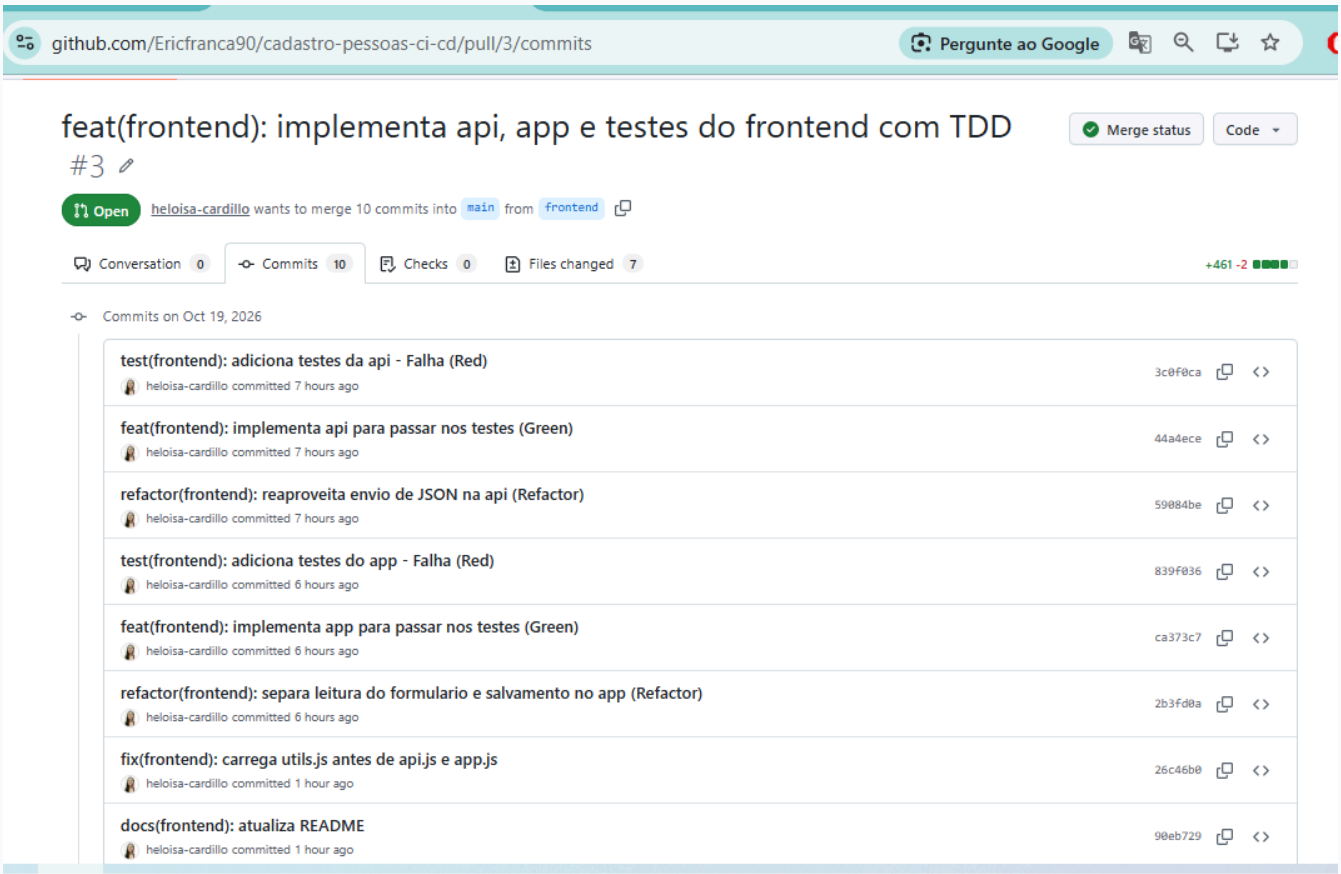
O pull request foi aberto da branch configurar-ferramentas para a main e permaneceu disponível para revisão do colega.



COLABORAÇÃO

Contribuição no frontend

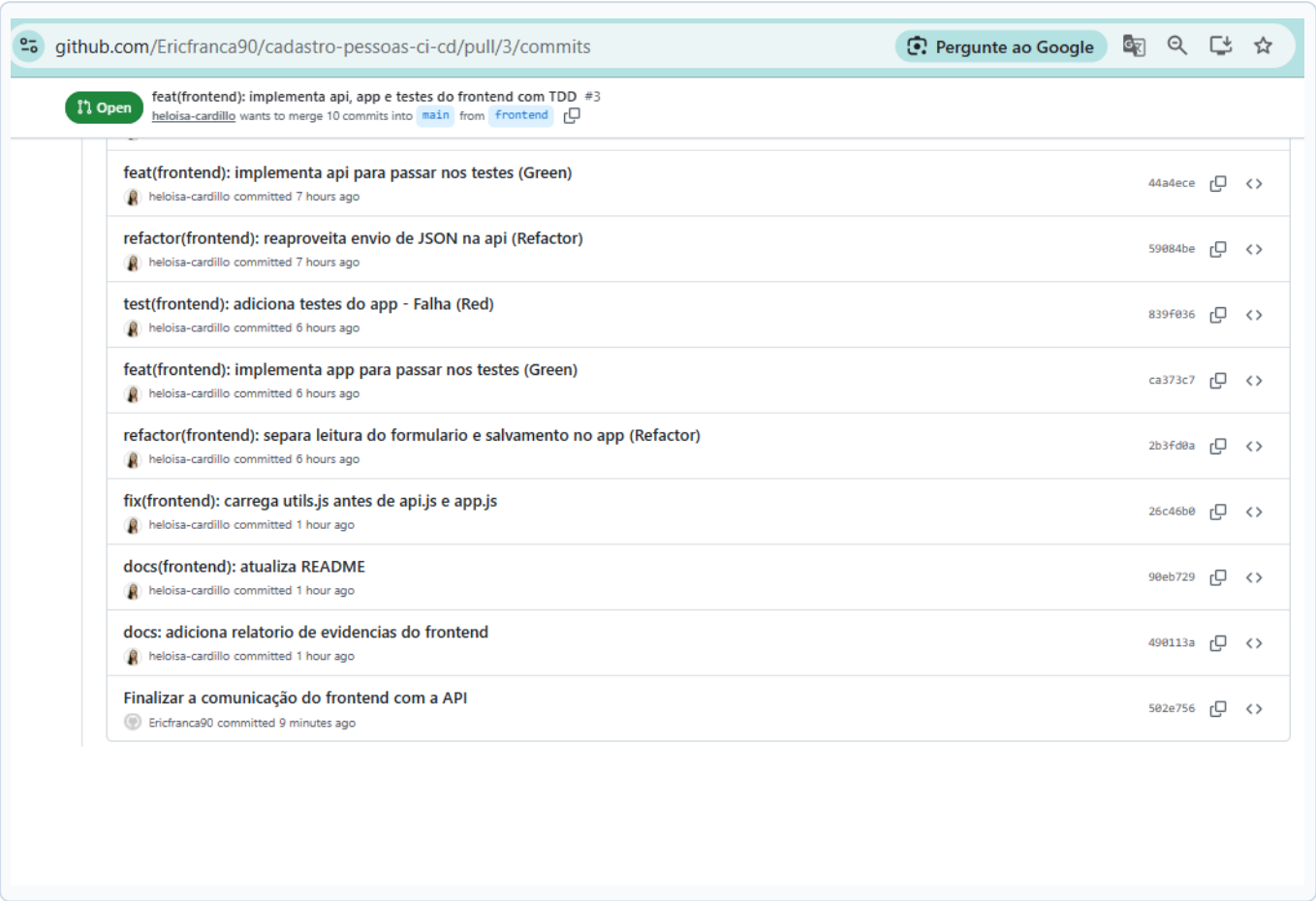
O pull request aberto mostra os commits de testes e desenvolvimento feitos pela Heloisa.



COLABORAÇÃO

Revisão em dupla

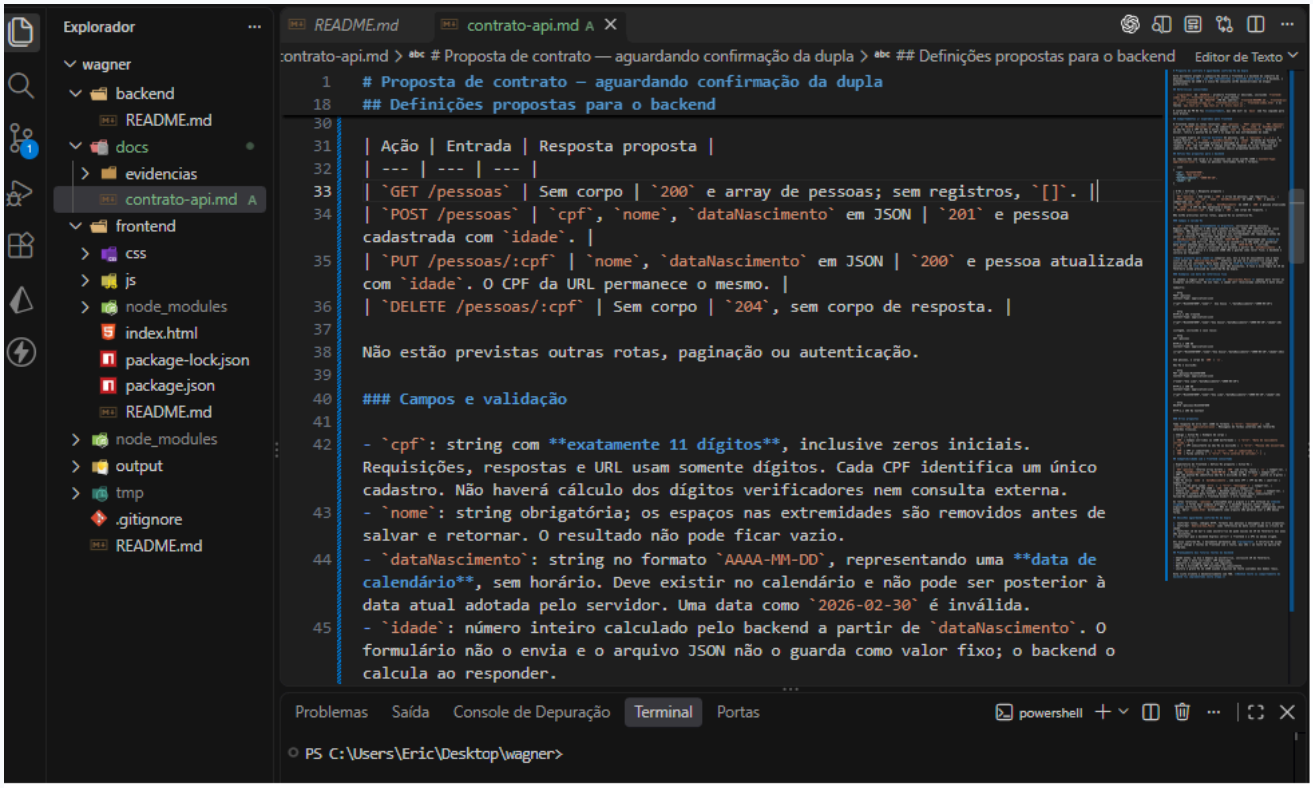
O histórico preserva os commits da Heloisa e registra a finalização feita pelo Eric.



ETAPA 3

Rotas e formatos

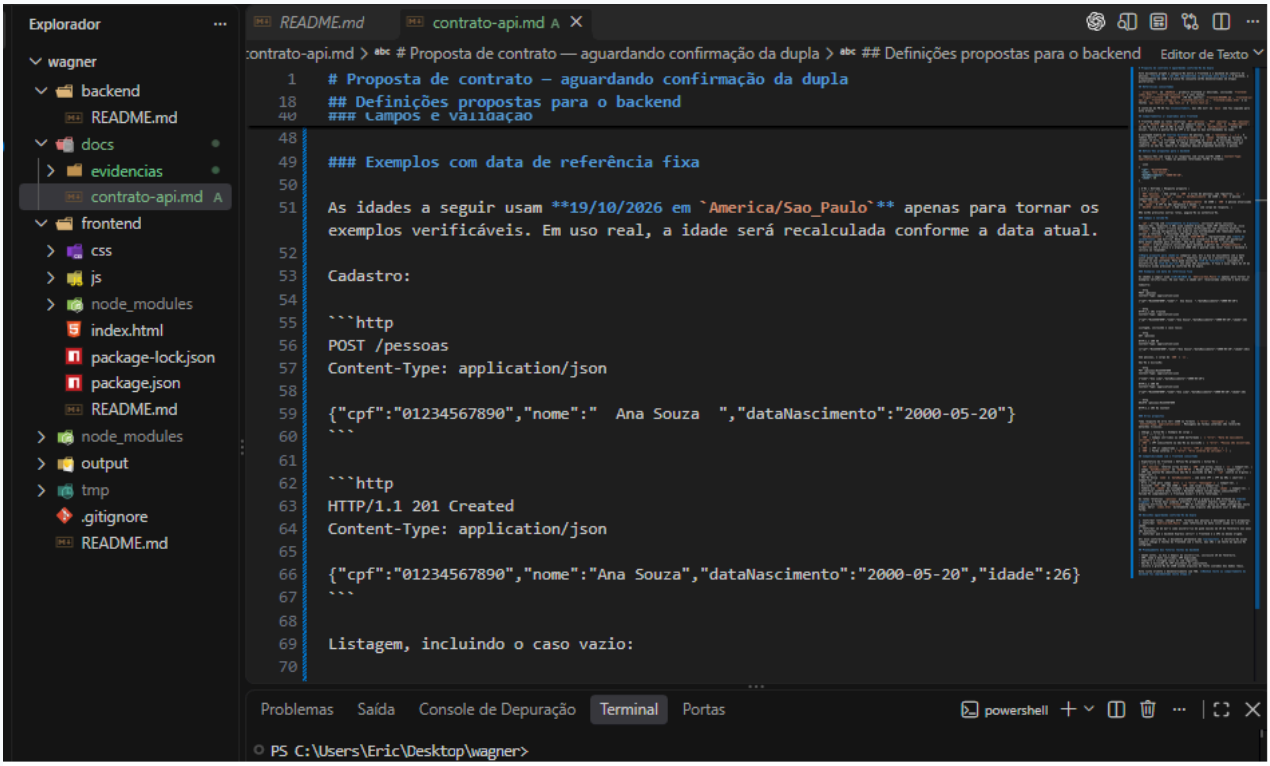
O documento propõe as quatro rotas do cadastro e os formatos dos campos.



ETAPA 3

Exemplos da API

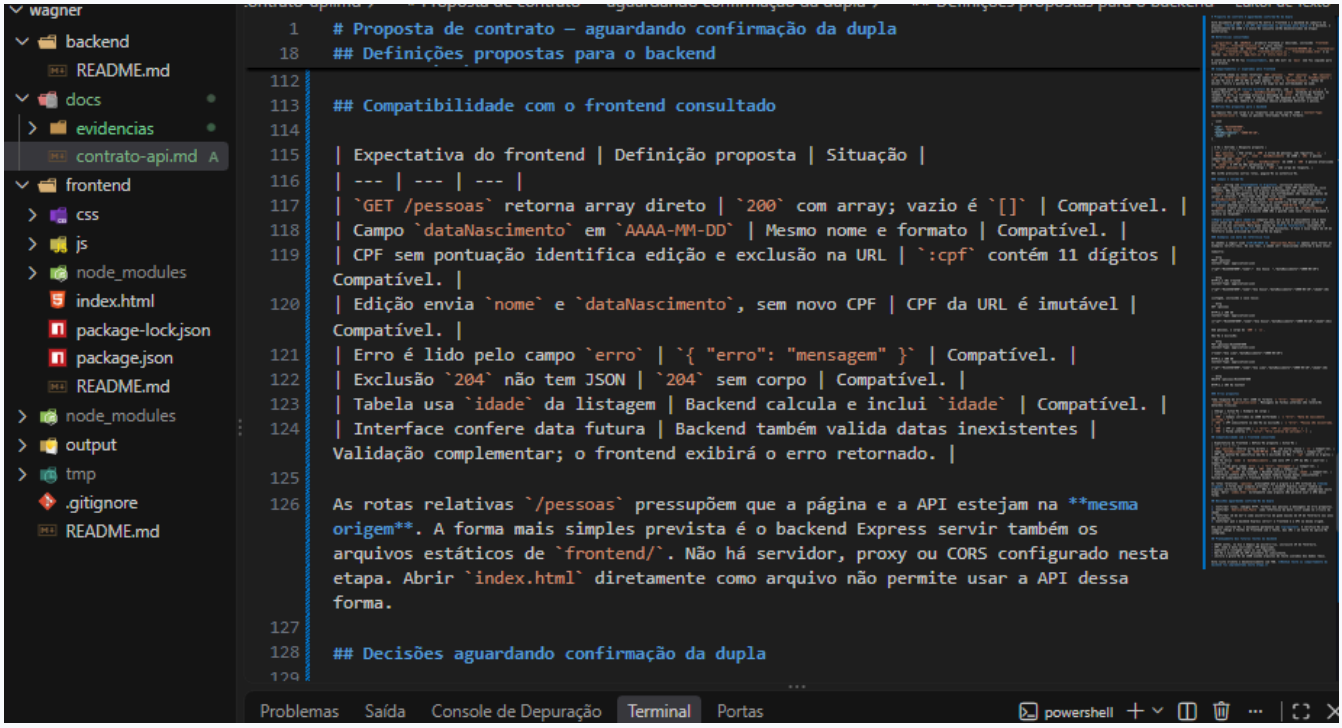
A proposta mostra um cadastro e sua resposta com a idade calculada.



ETAPA 3

Compatibilidade com o frontend

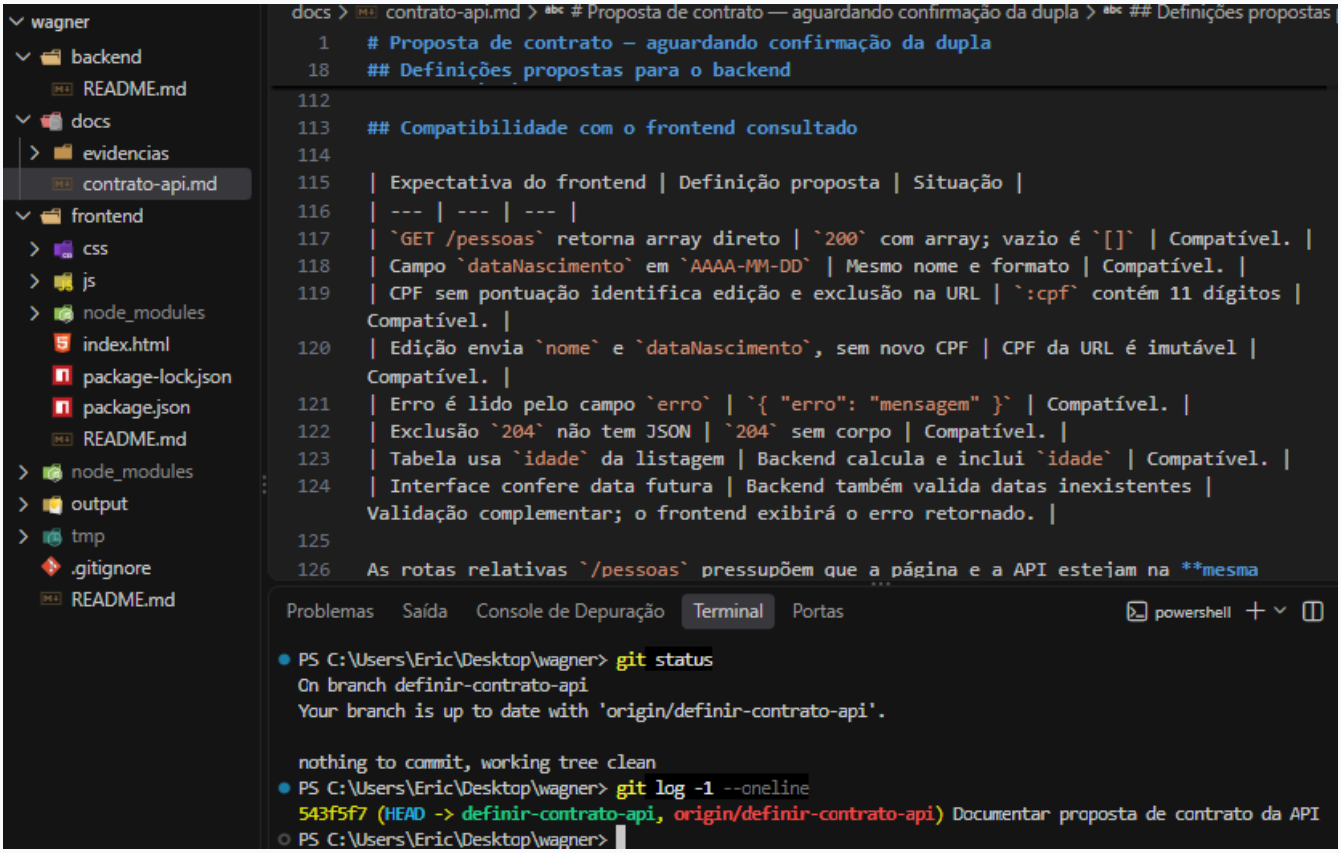
A tabela compara o contrato proposto com o código já escrito para a interface.



ETAPA 3

Branch e commit

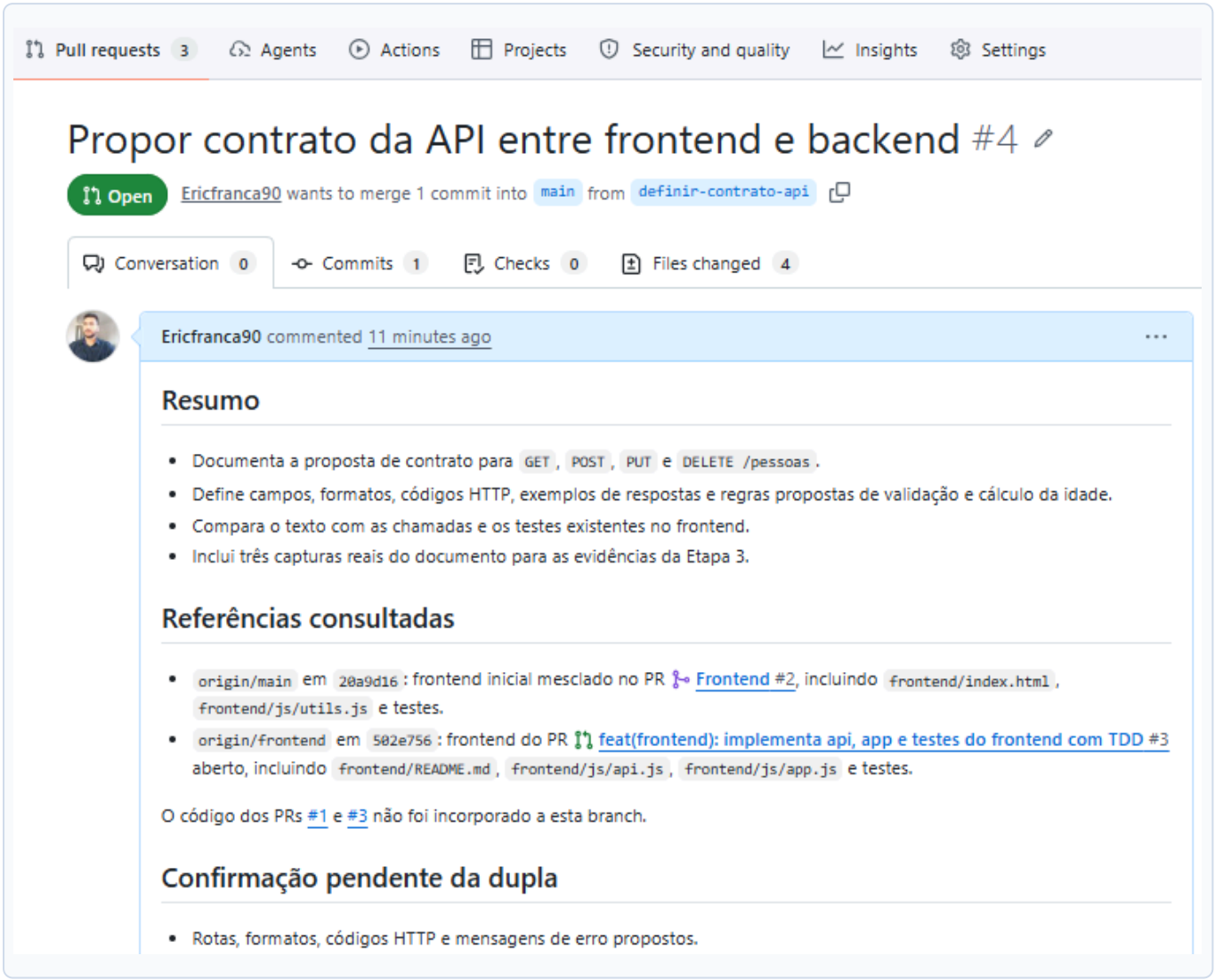
O terminal mostra a branch limpa e o commit da proposta de contrato.



ETAPA 3

Pull request para revisão

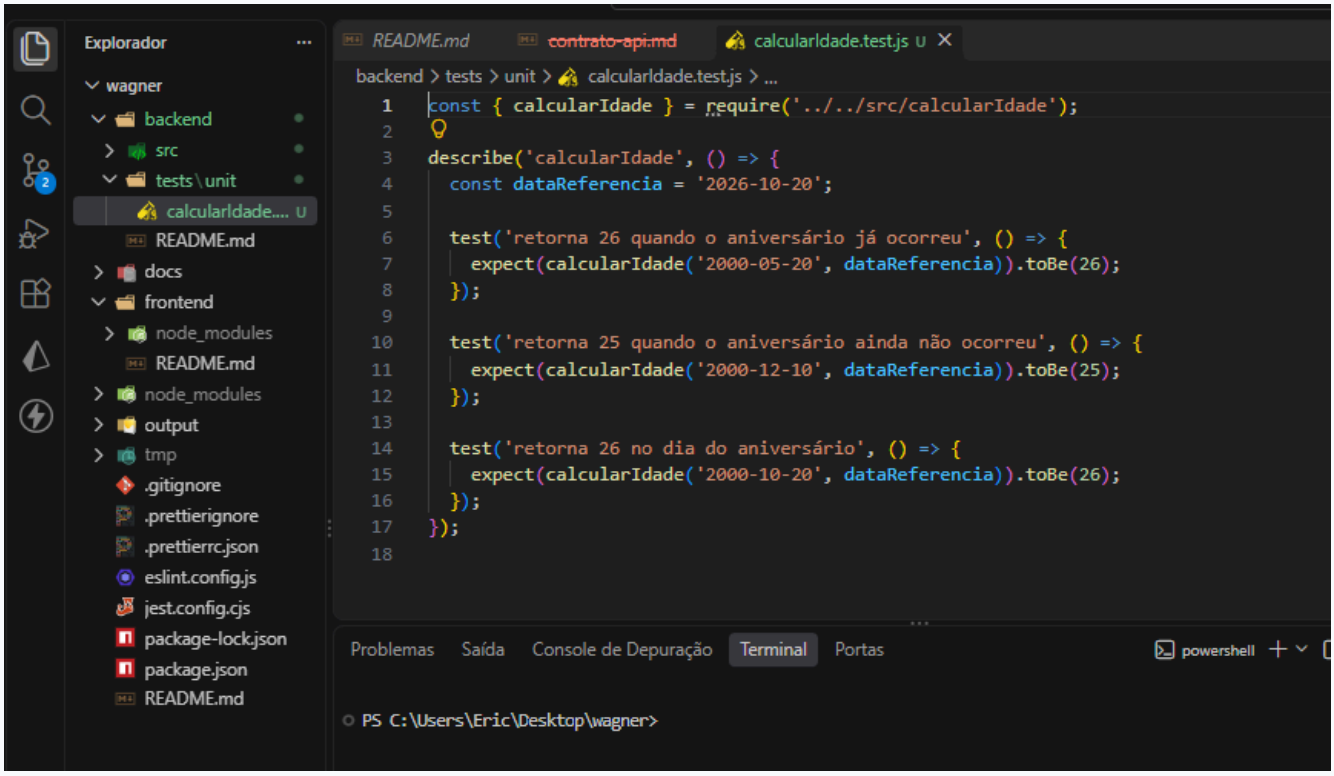
O PR #4 apresenta a proposta à dupla e permanece aberto para revisão.



ETAPA 4 - RED

Testes escritos primeiro

Três cenários usam a referência fixa de 20/10/2026; a implementação ainda não havia sido criada.



ETAPA 4 - RED

Módulo mínimo

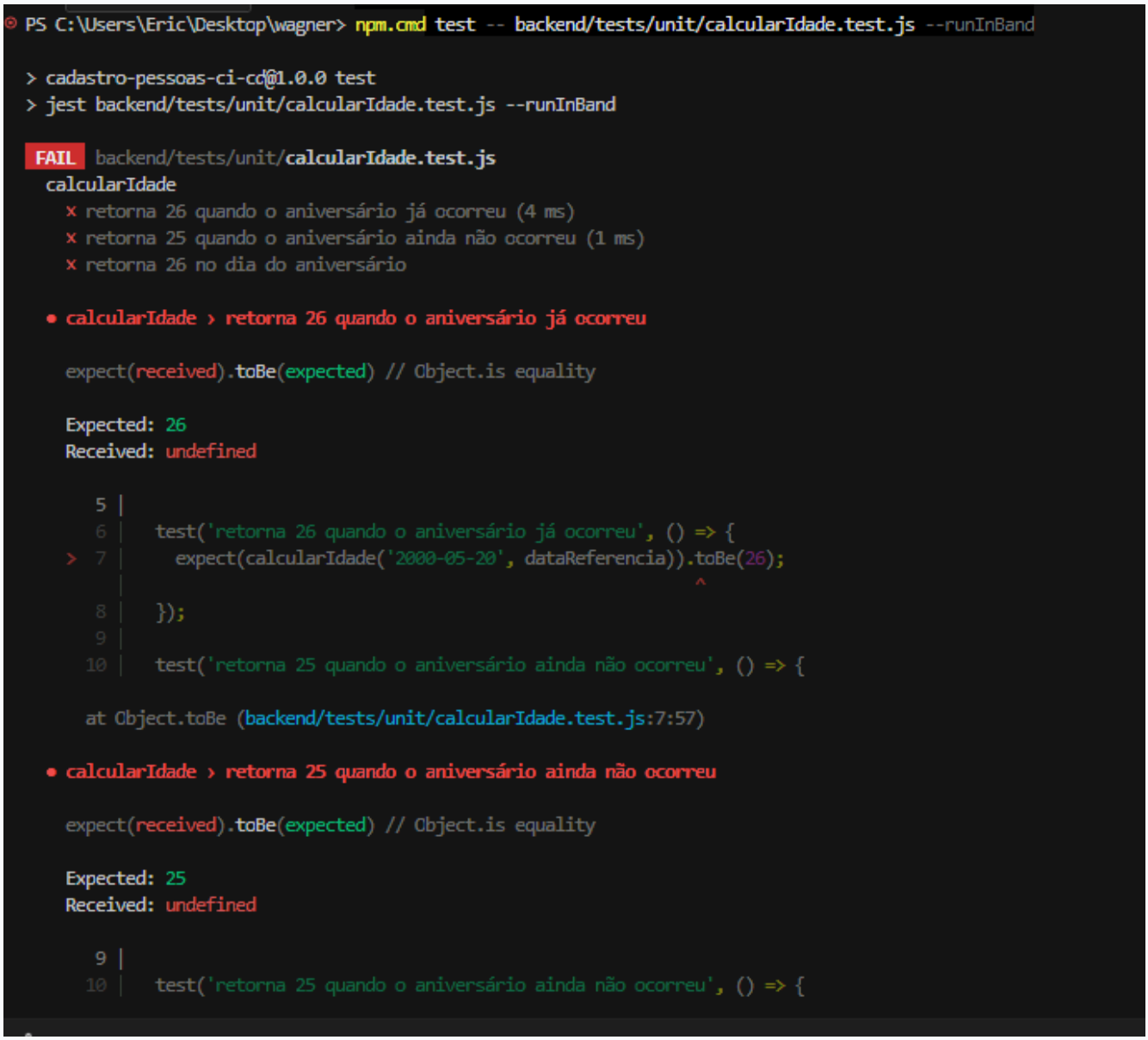
A função exportada retornava apenas undefined, sem algoritmo ou respostas preparadas.



ETAPA 4 - RED

Falha dos testes - parte 1

O Jest executou os casos; os resultados recebidos foram undefined, diferentes das idades esperadas.

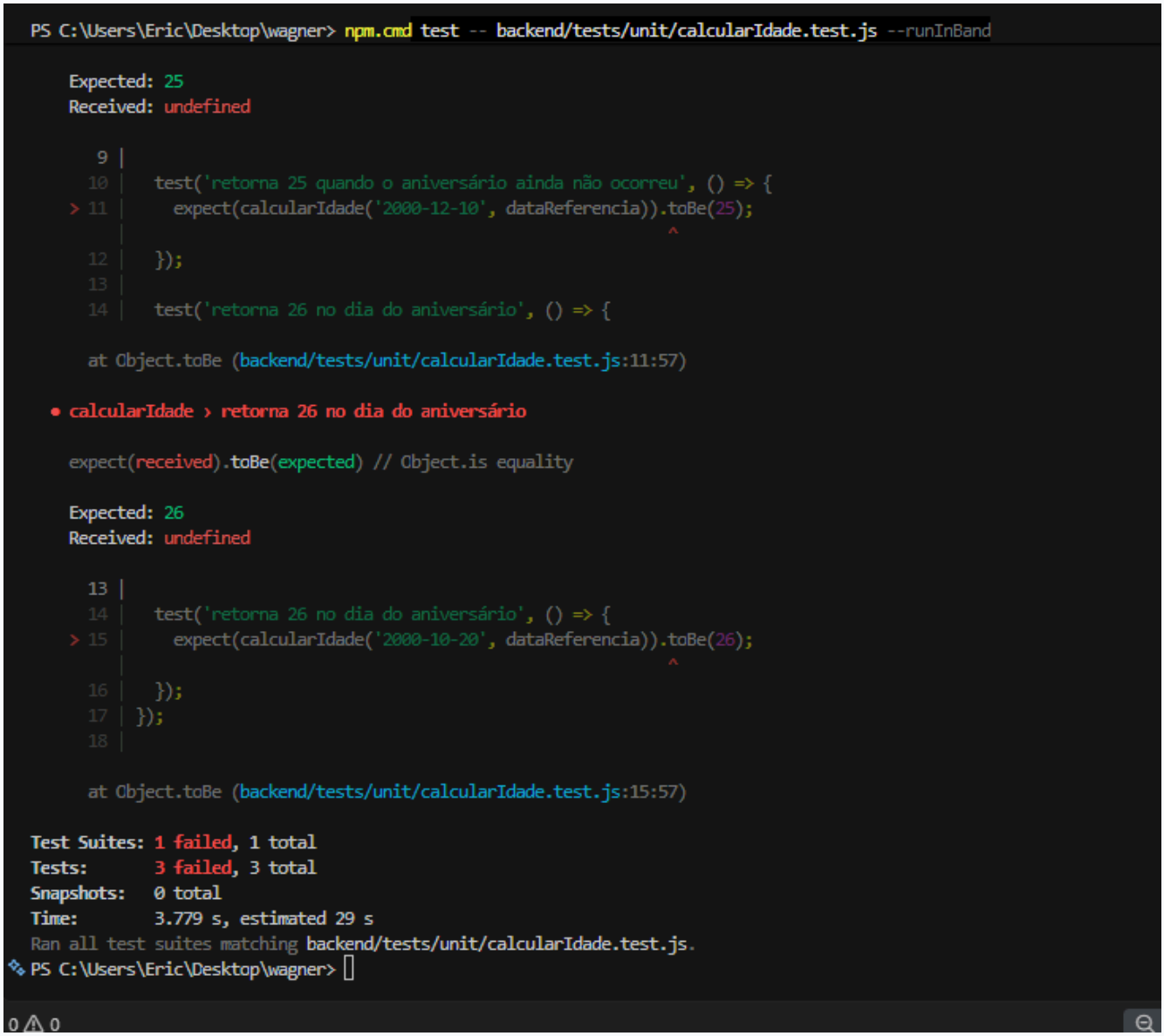


Evidência 18A

ETAPA 4 - RED

Falha dos testes - parte 2

A continuação da saída registra o terceiro caso e o resumo: 3 testes falharam.

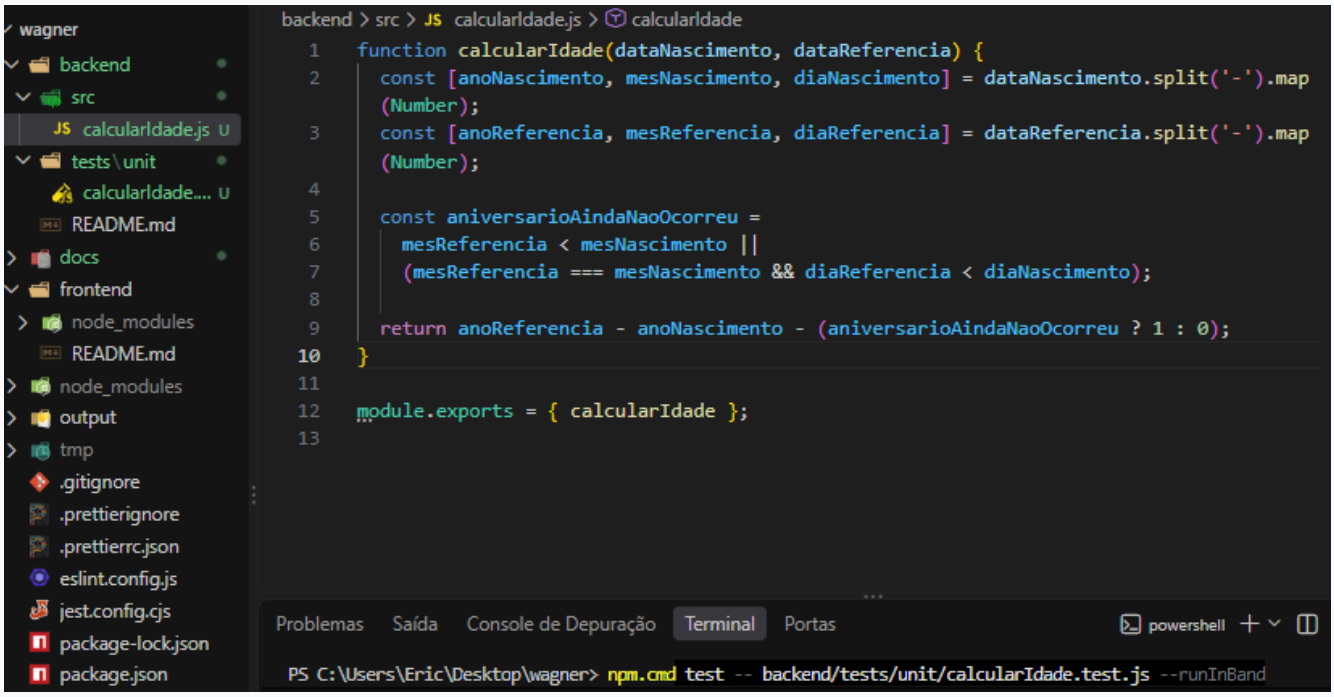


Evidência 18B

ETAPA 4 - GREEN

Cálculo implementado

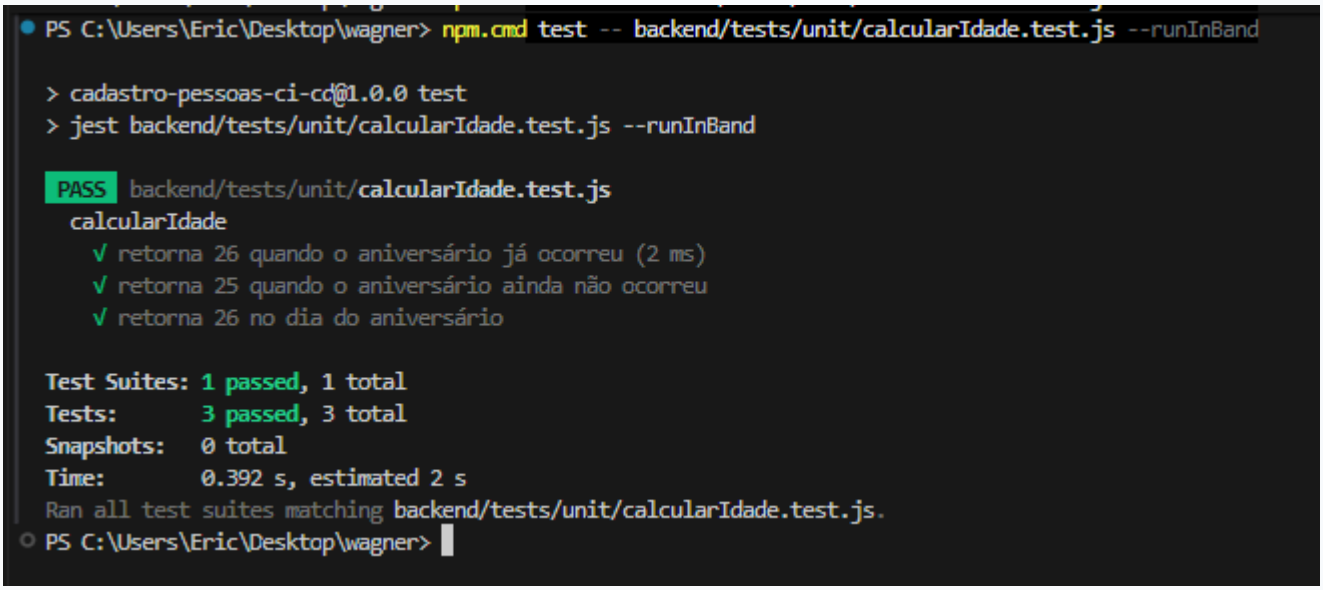
O backend compara ano, mês e dia para retornar a idade em anos completos, usando a data de referência recebida.



ETAPA 4 - GREEN

Testes passando

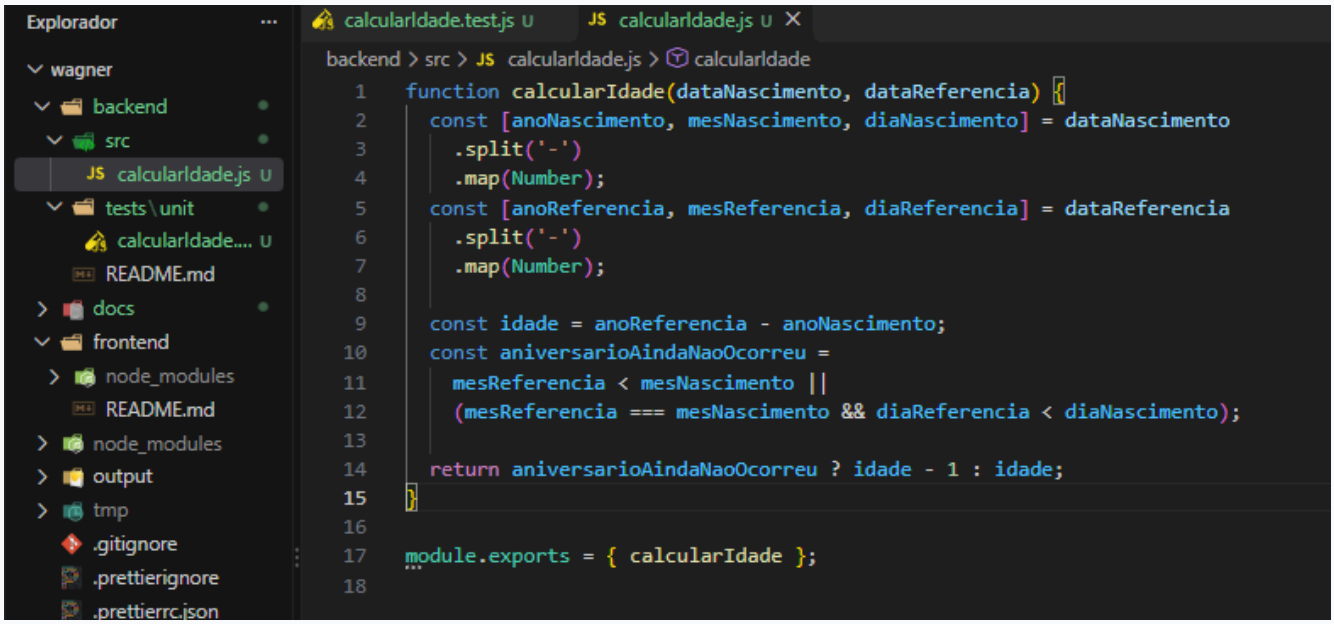
Depois da implementação, os três testes unitários passaram.



ETAPA 4 - REFACTOR

Código reorganizado

A idade-base foi separada da regra de aniversário ainda não ocorrido, sem acrescentar comportamento.



ETAPA 4 - REFACTOR

Resultados preservados

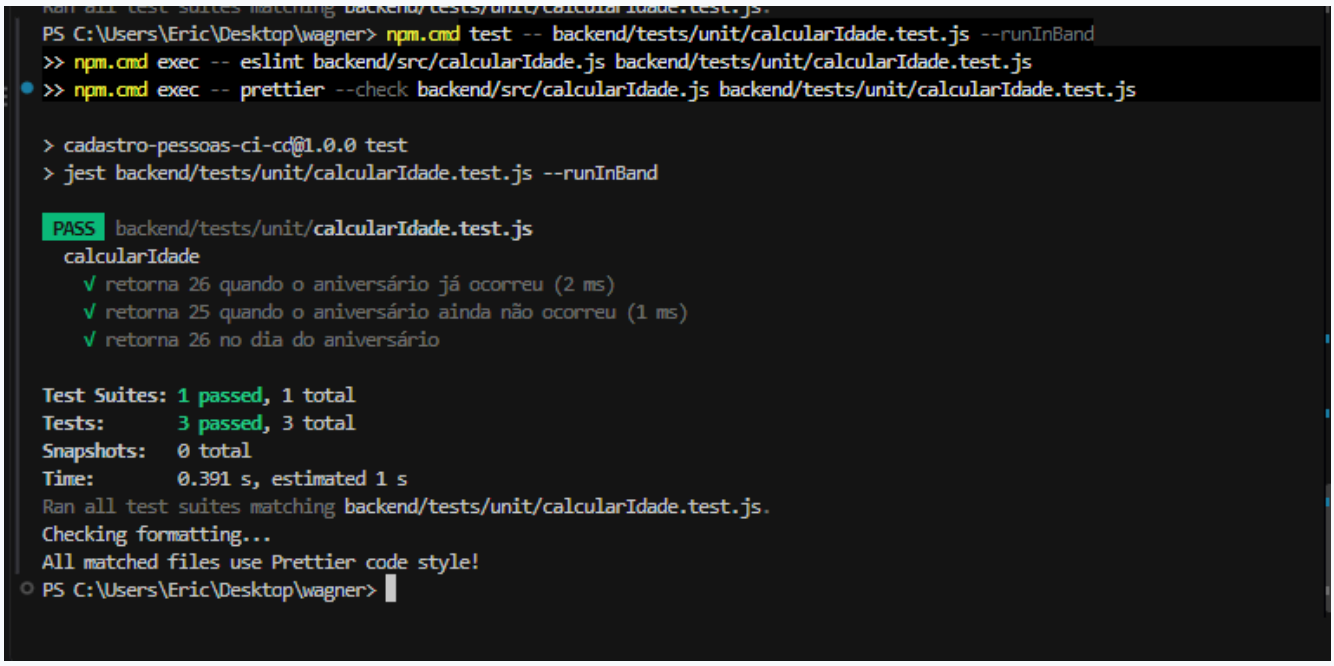
Após a refatoração, os mesmos três testes continuaram passando.



ETAPA 4 - VERIFICAÇÕES

Testes e análise dos arquivos novos

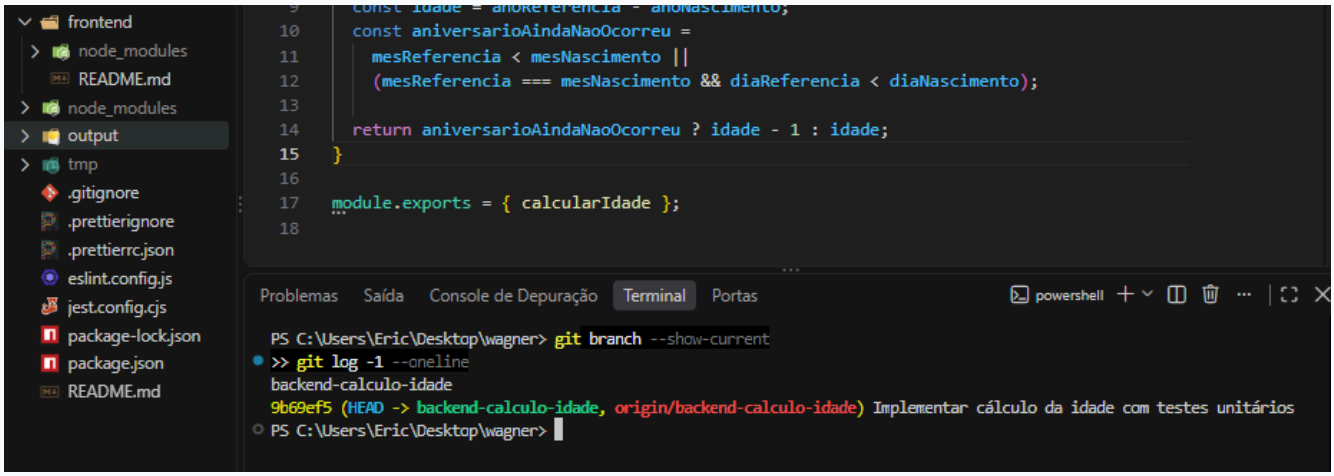
Os três testes, o ESLint e o Prettier dos dois arquivos deste ciclo passaram. A verificação geral de formatação continua pendente.



ETAPA 4 - GIT

Branch e commit de código

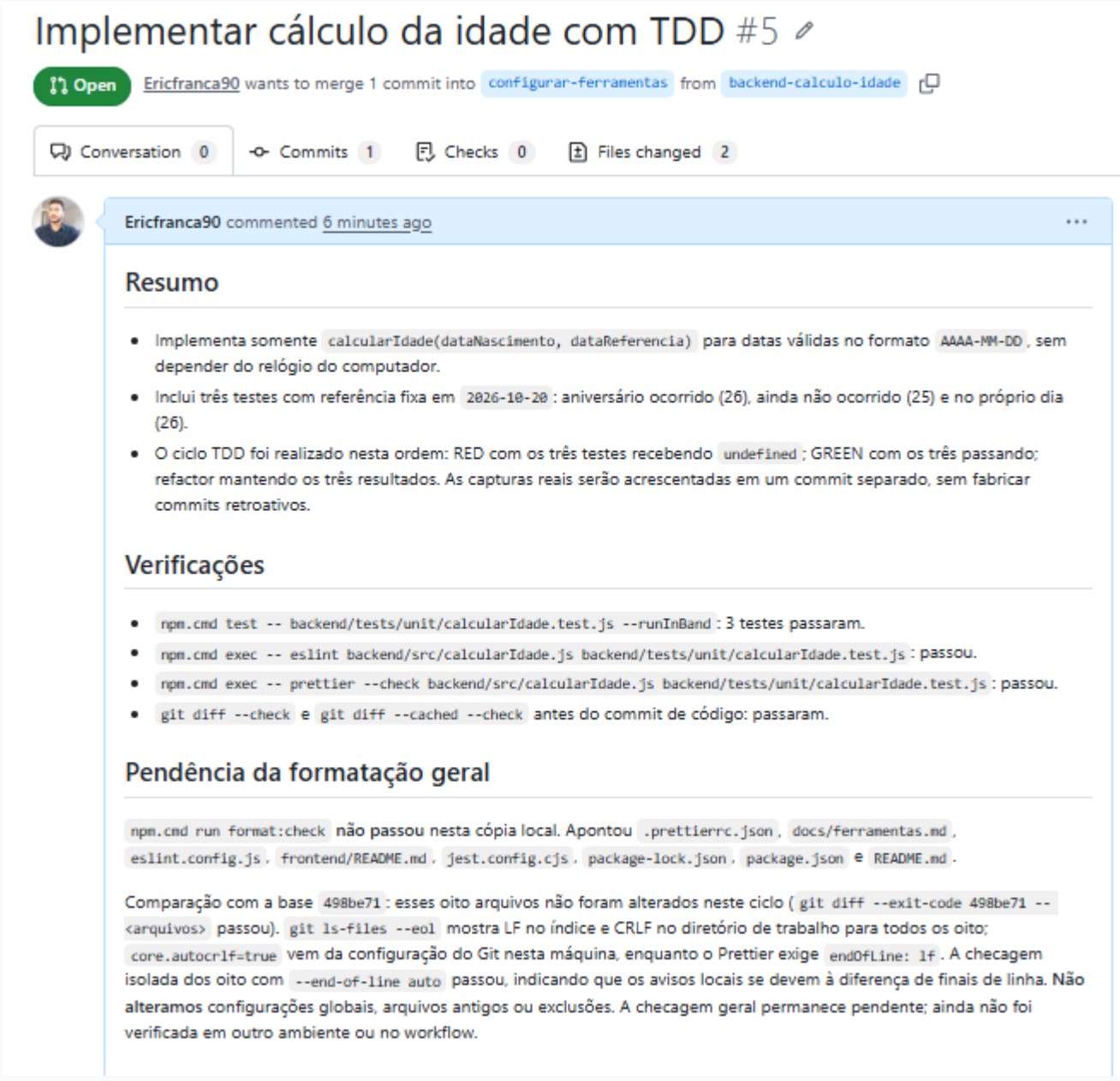
O código e os testes foram registrados juntos, sem criar commits retroativos das fases TDD.



ETAPA 4 - GIT

Pull request para revisão

O PR #5 permanece aberto, da branch backend-calculo-idade para configurar-ferramentas, da qual depende.



Implementar cálculo da idade com TDD #5

Open Ericfranca90 wants to merge 1 commit into `configurar-ferramentas` from `backend-calculo-idade`

Conversation 0 · Commits 1 · Checks 0 · Files changed 2

Ericfranca90 commented 6 minutes ago

Resumo

- Implementa somente `calcularIdade(dataNascimento, dataReferencia)` para datas válidas no formato `AAAA-MM-DD`, sem depender do relógio do computador.
- Inclui três testes com referência fixa em `2026-10-20`: aniversário ocorrido (26), ainda não ocorrido (25) e no próprio dia (26).
- O ciclo TDD foi realizado nesta ordem: RED com os três testes recebendo `undefined`; GREEN com os três passando; refactor mantendo os três resultados. As capturas reais serão acrescentadas em um commit separado, sem fabricar commits retroativos.

Verificações

- `npm.cmd test -- backend/tests/unit/calcularIdade.test.js --runInBand`: 3 testes passaram.
- `npm.cmd exec -- eslint backend/src/calcularIdade.js backend/tests/unit/calcularIdade.test.js`: passou.
- `npm.cmd exec -- prettier --check backend/src/calcularIdade.js backend/tests/unit/calcularIdade.test.js`: passou.
- `git diff --check` e `git diff --cached --check` antes do commit de código: passaram.

Pendência da formatação geral

`npm.cmd run format:check` não passou nesta cópia local. Apontou `.prettierrc.json`, `docs/ferramentas.md`, `eslint.config.js`, `frontend/README.md`, `jest.config.cjs`, `package-lock.json`, `package.json` e `README.md`.

Comparação com a base `498be71`: esses oito arquivos não foram alterados neste ciclo (`git diff --exit-code 498be71 -- <arquivos>` passou). `git ls-files --eol` mostra LF no índice e CRLF no diretório de trabalho para todos os oito; `core.autocrlf=true` vem da configuração do Git nesta máquina, enquanto o Prettier exige `endOfLine: lf`. A checagem isolada dos oito com `--end-of-line auto` passou, indicando que os avisos locais se devem à diferença de finais de linha. Não alteramos configurações globais, arquivos antigos ou exclusões. A checagem geral permanece pendente; ainda não foi verificada em outro ambiente ou no workflow.

ETAPA 5 - RED

Testes escritos primeiro

Sete cenários usam dados fictícios e a referência fixa de 20/10/2026.



```
const { validarPessoa } = require('../../src/validarPessoa');

describe('validarPessoa', () => {
  const dataReferencia = '2026-10-20';

  function pessoaValida() {
    return {
      cpf: '01234567890',
      nome: 'Ana Souza',
      dataNascimento: '2000-05-20',
    };
  }

  test('retorna null para um cadastro válido', () => {
    expect(validarPessoa(pessoaValida(), dataReferencia)).toBeNull();
  });

  test('rejeita CPF com menos de 11 dígitos', () => {
    const dados = { ...pessoaValida(), cpf: '0123456789' };
    expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
  });

  test('rejeita CPF com 11 caracteres contendo letra', () => {
    const dados = { ...pessoaValida(), cpf: '0123456789A' };
    expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
  });

  test('rejeita nome contendo somente espaços', () => {
    const dados = { ...pessoaValida(), nome: '   ' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Informe o nome.');
  });

  test('rejeita data de nascimento vazia', () => {
    const dados = { ...pessoaValida(), dataNascimento: '' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Informe a data de nascimento.');
  });

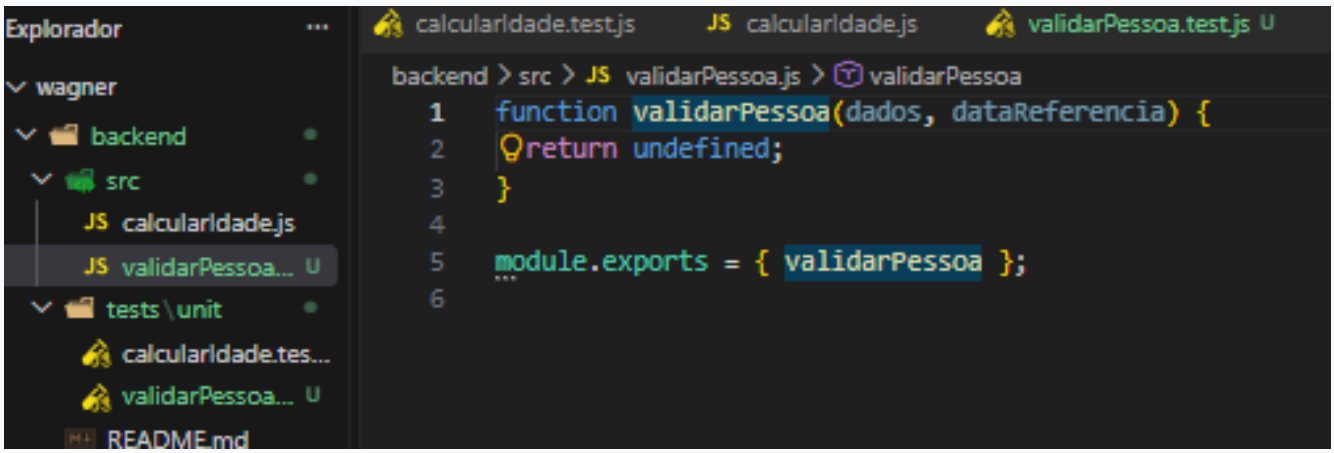
  test('rejeita data de nascimento inexistente', () => {
    const dados = { ...pessoaValida(), dataNascimento: '2026-02-30' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Data de nascimento inválida.');
  });

  test('rejeita nascimento posterior à data de referência', () => {
    const dados = { ...pessoaValida(), dataNascimento: '2026-10-21' };
    expect(validarPessoa(dados, dataReferencia)).toBe(
      'A data de nascimento não pode ser no futuro.',
    );
  });
});
```

ETAPA 5 - RED

Módulo mínimo

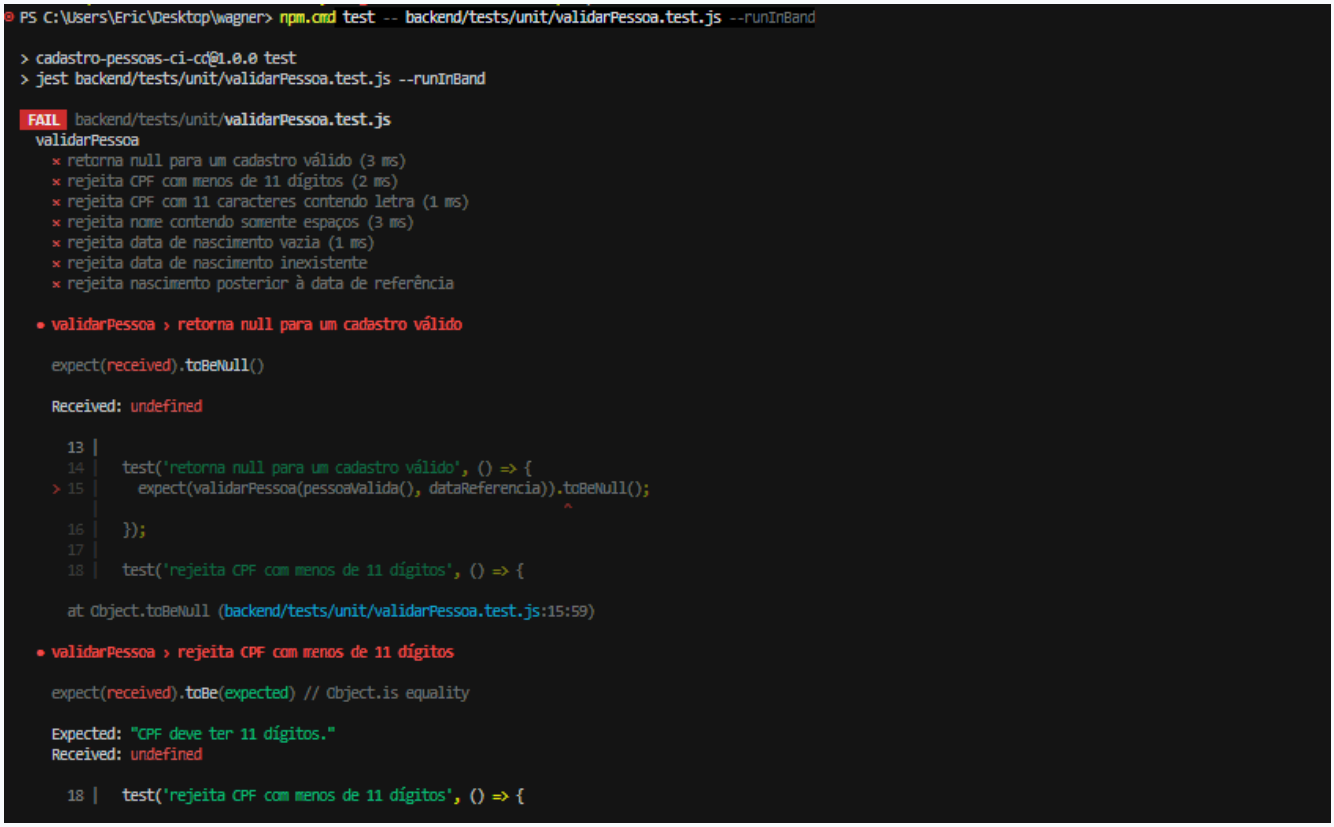
A função exportada ainda retornava undefined, sem validar qualquer campo.



ETAPA 5 - RED

Falhas dos testes - parte 1

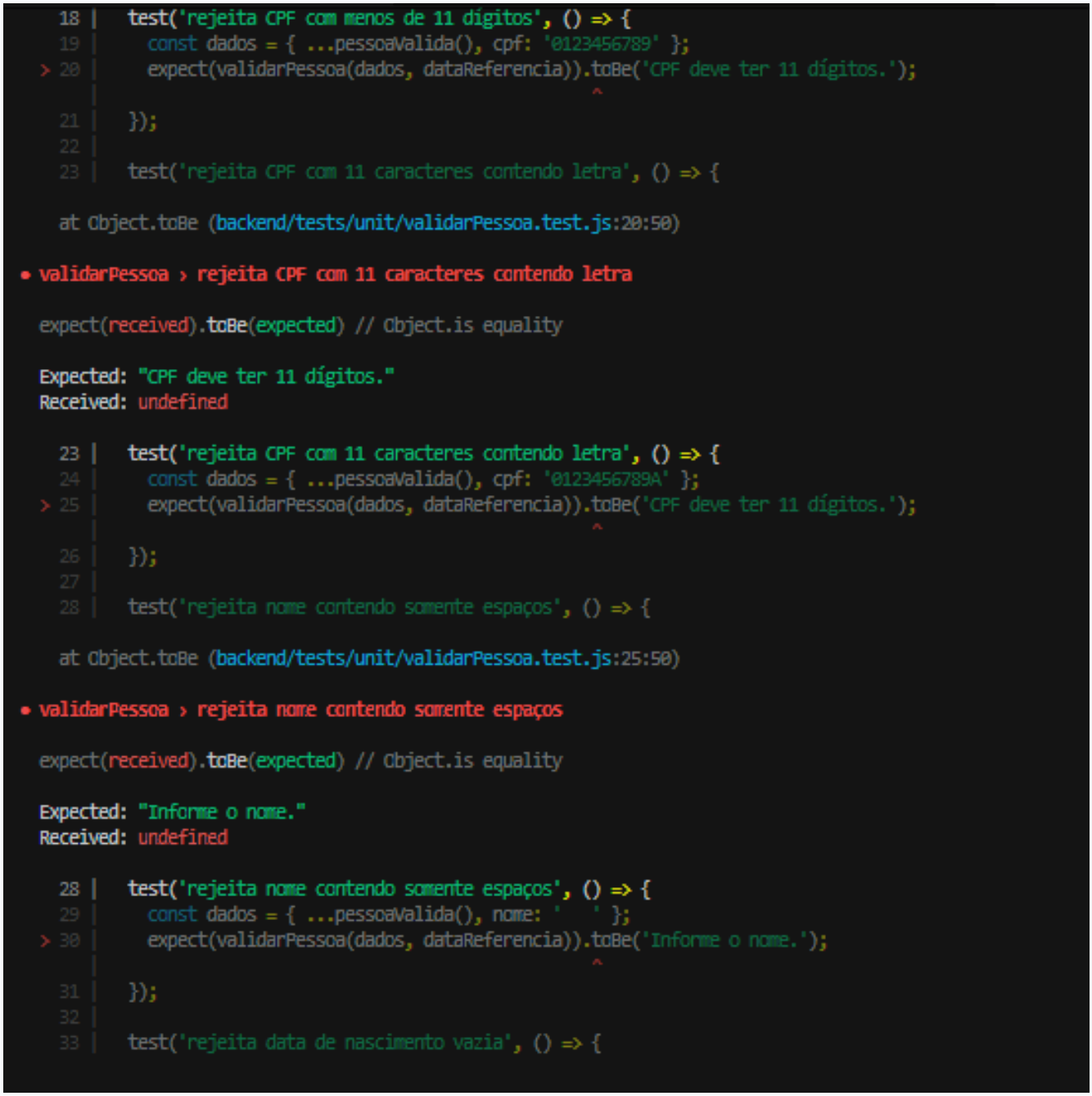
O Jest executou os sete testes; as primeiras falhas mostram o resultado undefined.



ETAPA 5 - RED

Falhas dos testes - parte 2

A continuação registra a rejeição esperada para CPF e nome inválidos.



```
  18 |   test('rejeita CPF com menos de 11 dígitos', () => {
  19 |     const dados = { ...pessoaValida(), cpf: '0123456789' };
> 20 |     expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
     |                                                  ^
  21 |   });
  22 |
  23 |   test('rejeita CPF com 11 caracteres contendo letra', () => {

  at Object.toBe (backend/tests/unit/validarPessoa.test.js:20:50)

● validarPessoa › rejeita CPF com 11 caracteres contendo letra

  expect(received).toBe(expected) // Object.is equality

  Expected: "CPF deve ter 11 dígitos."
  Received: undefined

  23 |   test('rejeita CPF com 11 caracteres contendo letra', () => {
  24 |     const dados = { ...pessoaValida(), cpf: '0123456789A' };
> 25 |     expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
     |                                                  ^
  26 |   });
  27 |
  28 |   test('rejeita nome contendo somente espaços', () => {

  at Object.toBe (backend/tests/unit/validarPessoa.test.js:25:50)

● validarPessoa › rejeita nome contendo somente espaços

  expect(received).toBe(expected) // Object.is equality

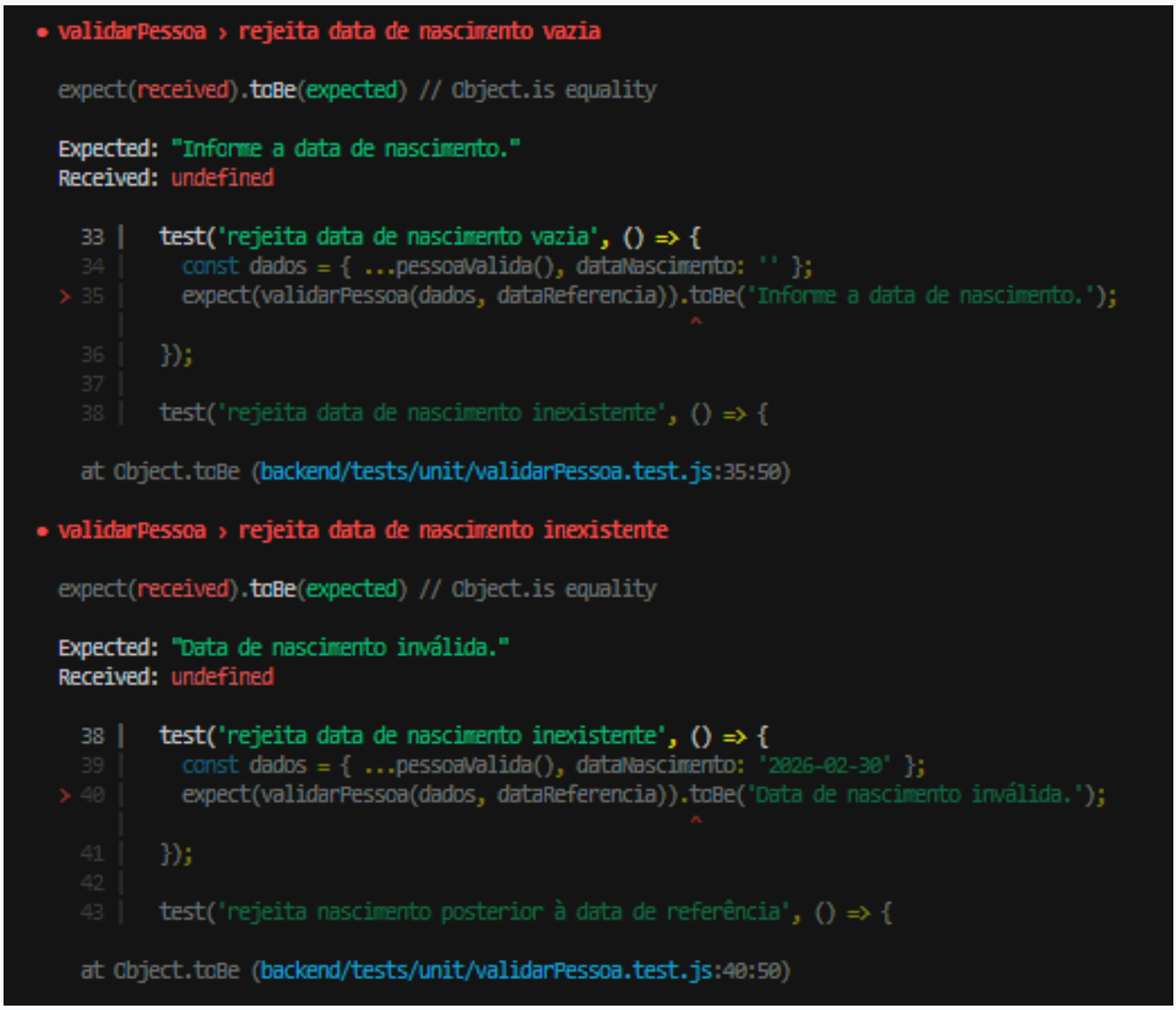
  Expected: "Informe o nome."
  Received: undefined

  28 |   test('rejeita nome contendo somente espaços', () => {
  29 |     const dados = { ...pessoaValida(), nome: '   ' };
> 30 |     expect(validarPessoa(dados, dataReferencia)).toBe('Informe o nome.');
     |                                                  ^
  31 |   });
  32 |
  33 |   test('rejeita data de nascimento vazia', () => {
```

ETAPA 5 - RED

Falhas dos testes - parte 3

Os casos de nascimento vazio e data inexistente também falharam por undefined.



```
● validarPessoa › rejeita data de nascimento vazia

  expect(received).toBe(expected) // Object.is equality

  Expected: "Informe a data de nascimento."
  Received: undefined

    33 |   test('rejeita data de nascimento vazia', () => {
    34 |     const dados = { ...pessoaValida(), dataNascimento: '' };
  > 35 |     expect(validarPessoa(dados, dataReferencia)).toBe('Informe a data de nascimento.');
       |                                                  ^
    36 |   });
    37 |
    38 |   test('rejeita data de nascimento inexistente', () => {

    at Object.toBe (backend/tests/unit/validarPessoa.test.js:35:50)

● validarPessoa › rejeita data de nascimento inexistente

  expect(received).toBe(expected) // Object.is equality

  Expected: "Data de nascimento inválida."
  Received: undefined

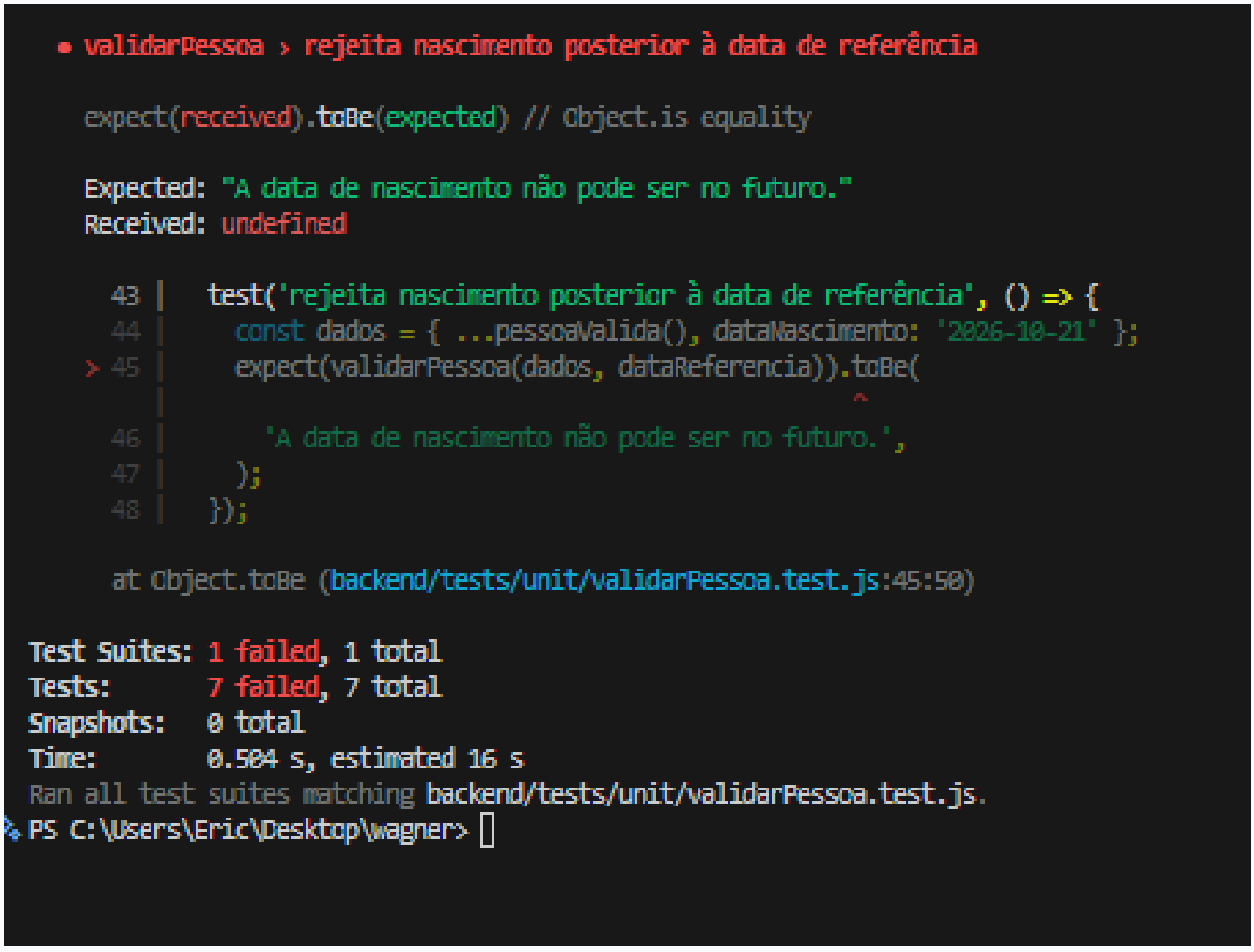
    38 |   test('rejeita data de nascimento inexistente', () => {
    39 |     const dados = { ...pessoaValida(), dataNascimento: '2026-02-30' };
  > 40 |     expect(validarPessoa(dados, dataReferencia)).toBe('Data de nascimento inválida.');
       |                                                  ^
    41 |   });
    42 |
    43 |   test('rejeita nascimento posterior à data de referência', () => {

    at Object.toBe (backend/tests/unit/validarPessoa.test.js:40:50)
```

ETAPA 5 - RED

Falhas dos testes - parte 4

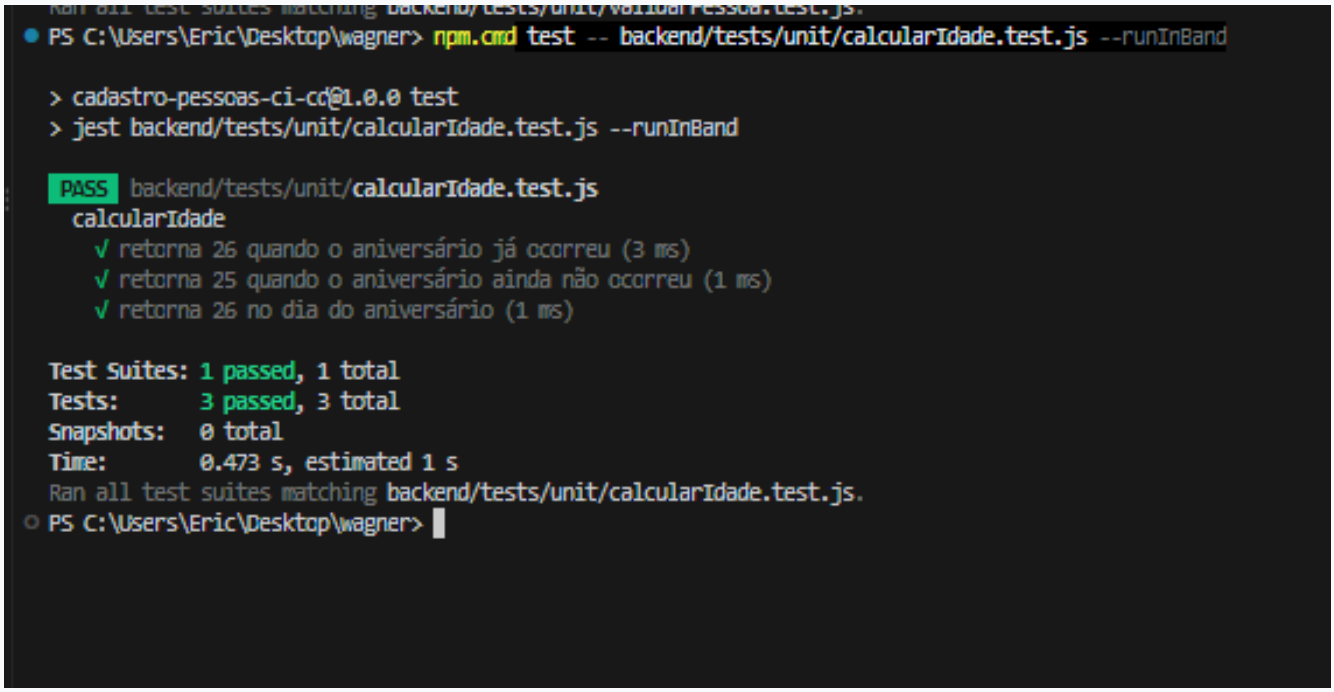
O caso de data futura falhou, e o resumo confirma sete testes falhando.



ETAPA 5 - RED

Testes anteriores preservados

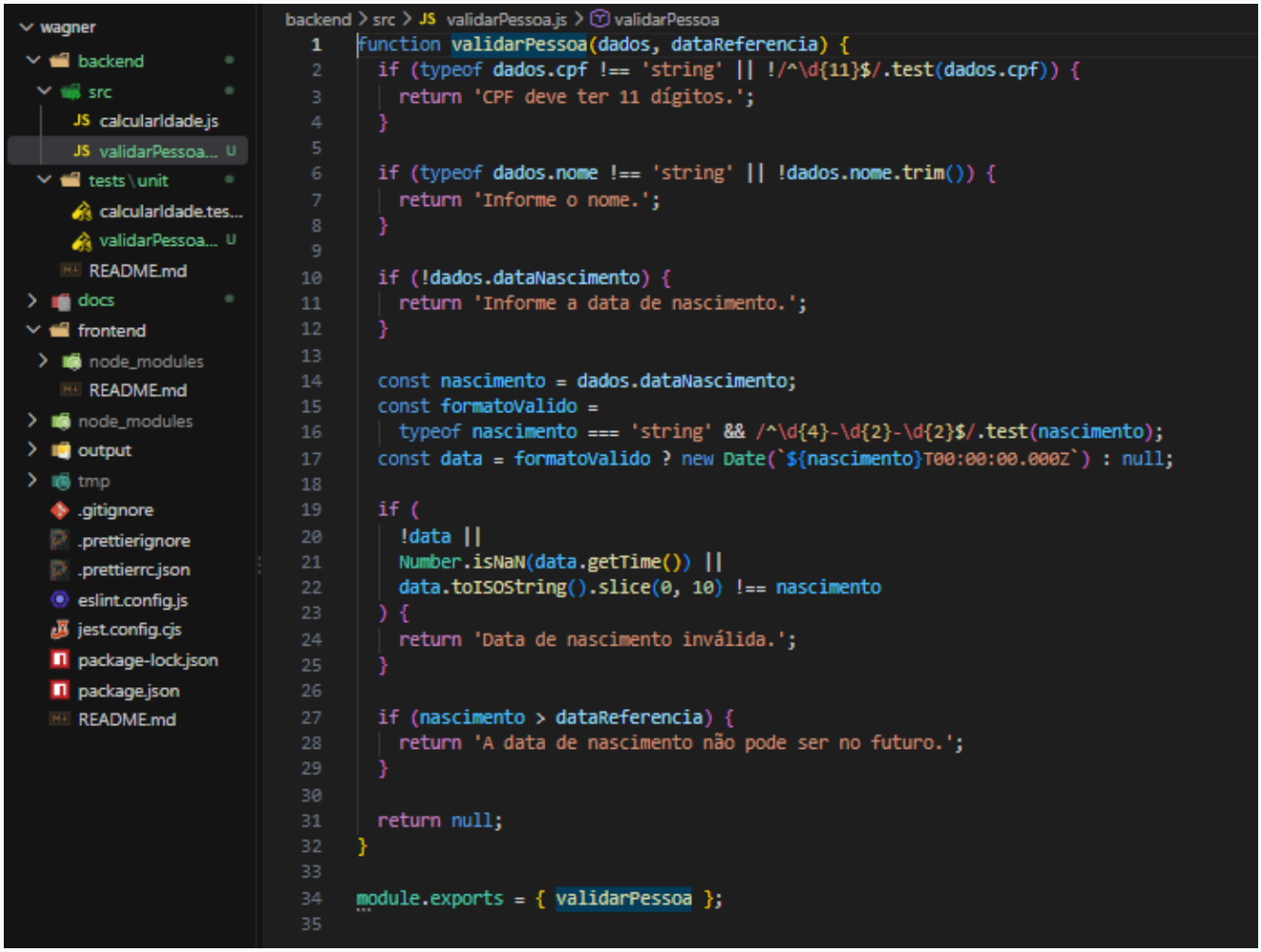
Os três testes do cálculo da idade continuaram passando durante o RED das validações.



ETAPA 5 - GREEN

Validação implementada

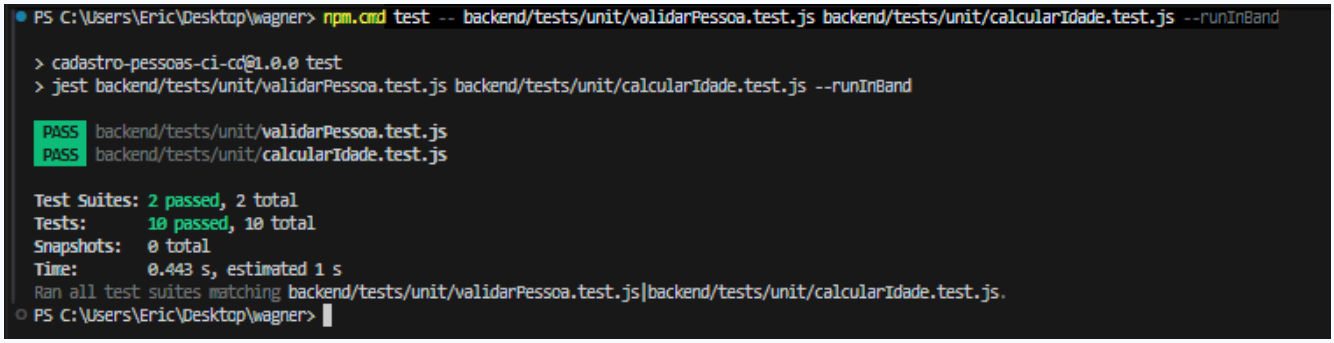
O backend passou a verificar CPF, nome e nascimento, sem consultar serviços externos.



ETAPA 5 - GREEN

Testes passando

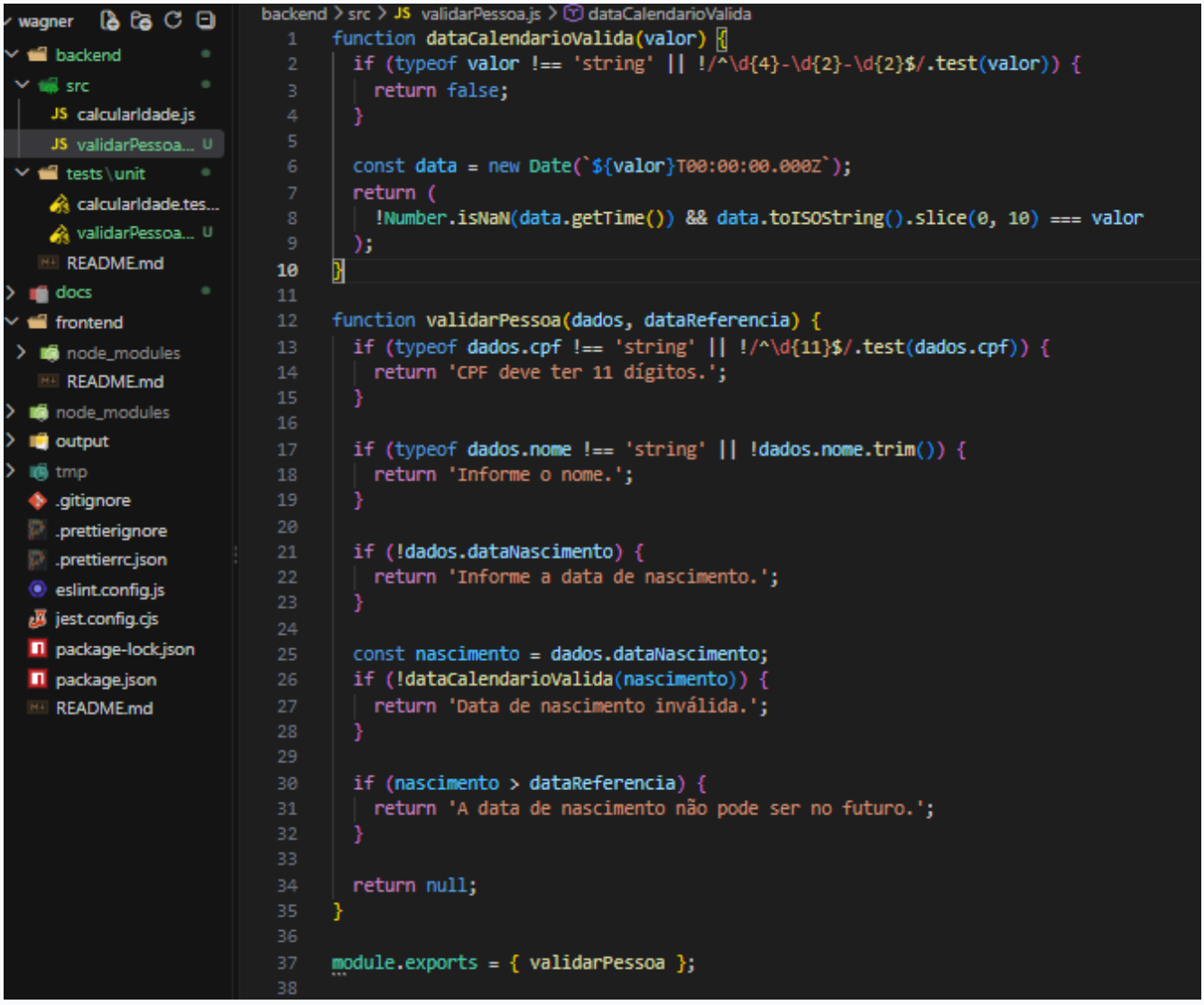
Os sete testes novos e os três testes anteriores passaram após a implementação.



ETAPA 5 - REFACTOR

Código reorganizado

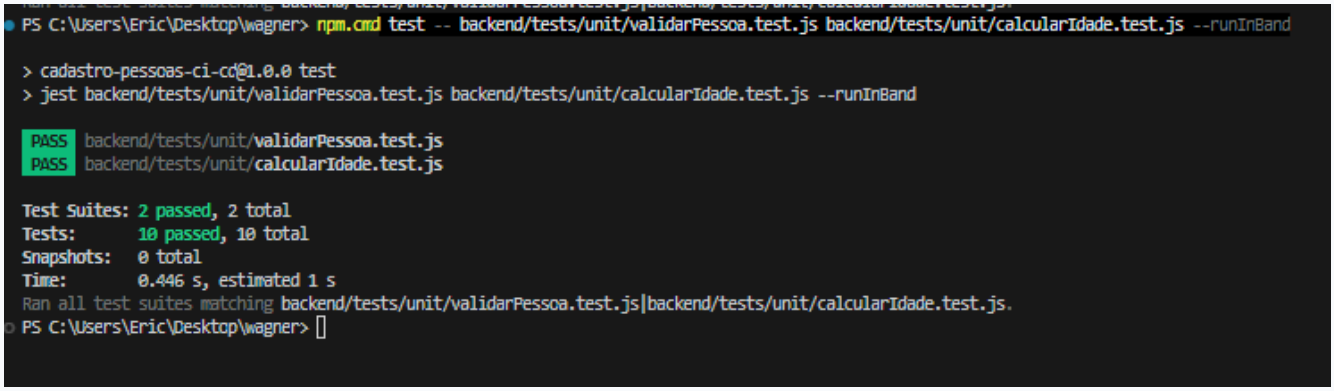
A conferência de data de calendário foi separada, sem mudar as regras testadas.



ETAPA 5 - REFACTOR

Resultados preservados

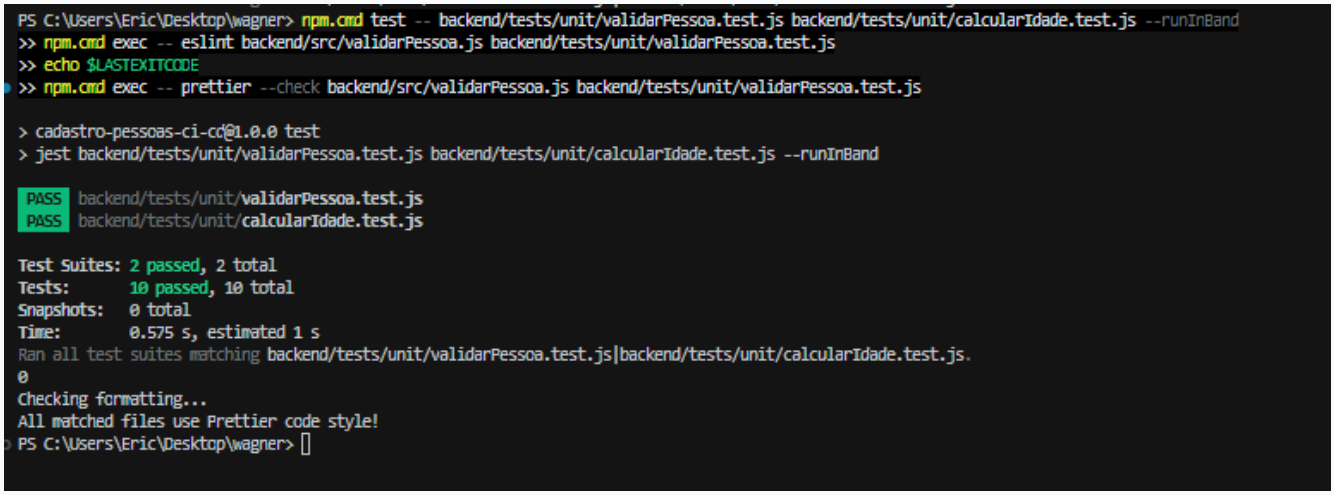
Depois da reorganização, os mesmos dez testes continuaram passando.



ETAPA 5 - VERIFICAÇÕES

Testes e qualidade dos arquivos novos

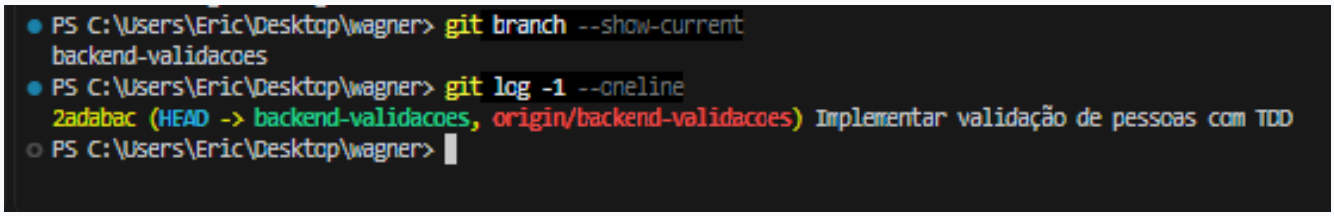
Dez testes passaram; ESLint e Prettier dos arquivos deste ciclo foram aprovados. A formatação geral continua pendente.



ETAPA 5 - GIT

Branch e commit de código

A branch backend-validacoes recebeu o commit 2adabac com o módulo e os testes.



ETAPA 5 - GIT

Pull request para revisão

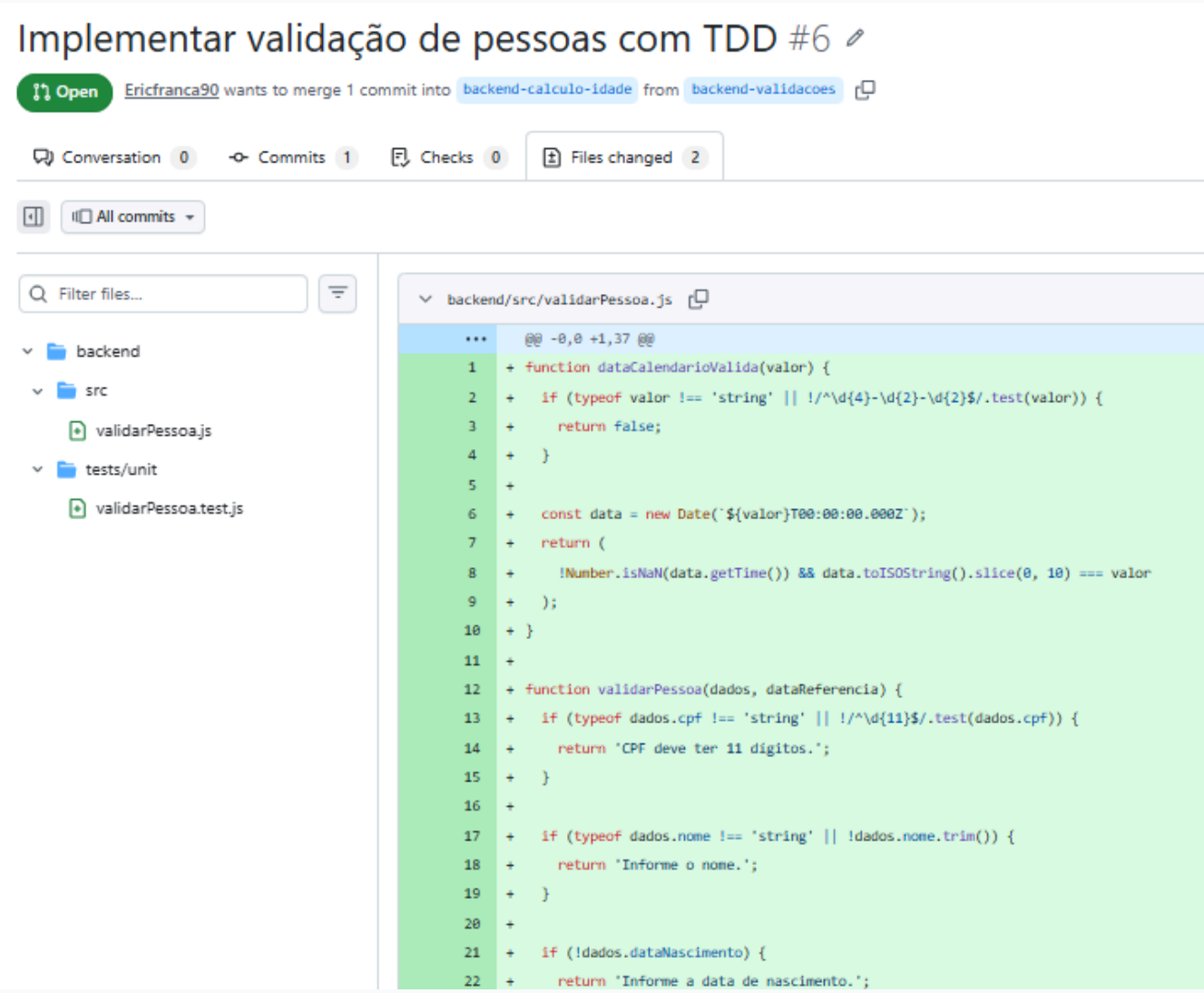
O PR #6 está aberto, de backend-validacoes para backend-calculo-idade, e depende do PR #5.



ETAPA 5 - GIT

Comparação do pull request

A comparação publicada mostra somente o módulo de validação e seus testes, sem reapresentar o cálculo da idade.



INTEGRAÇÃO E ENTREGA CONTÍNUA

Etapa 6
CRUD e API

Projeto Cadastro de Pessoas em JavaScript

> Continuação do relatório acumulado: desenvolvimento do backend com TDD, CRUD, armazenamento JSON e testes. As páginas anteriores preservam as etapas 1 a 5.

Resultado deste bloco: 43 testes passaram; ESLint e Prettier dos arquivos envolvidos passaram.
Versionamento: commit 0b7bb30 e PR #7 aberto sobre backend-validacoes, dependente do PR #6.
Pendência técnica: a verificação geral de formatação aponta cinco arquivos anteriores com finais de linha locais diferentes.
Próximas etapas de Eric: integrar o frontend já desenvolvido, configurar Docker e GitHub Actions.
Data desta atualização: 22 de outubro de 2026.

ETAPA 6 - RED

Teste do primeiro cadastro

O teste foi escrito antes da rota POST, com arquivo temporário e data de referência fixa.



```
describe('POST /pessoas', () => {
  let diretorioTeste;
  let arquivoDados;

  beforeEach(() => {
    diretorioTeste = fs.mkdtempSync(
      path.join(os.tmpdir(), 'cadastro-pessoas-teste-'),
    );
    arquivoDados = path.join(diretorioTeste, 'pessoas.json');
  });

  afterEach(() => {
    if (fs.existsSync(arquivoDados)) {
      fs.unlinkSync(arquivoDados);
    }
    fs.rmdirSync(diretorioTeste);
  });

  test('cadastra a pessoa, calcula a idade e persiste os dados sem a idade', async () => {
    const app = criarApp({
      arquivoDados,
      obterDataReferencia: () => '2026-10-20',
    });

    const resposta = await request(app).post('/pessoas').send({
      cpf: '01234567890',
      nome: '  Ana Souza  ',
      dataNascimento: '2000-05-20',
    });

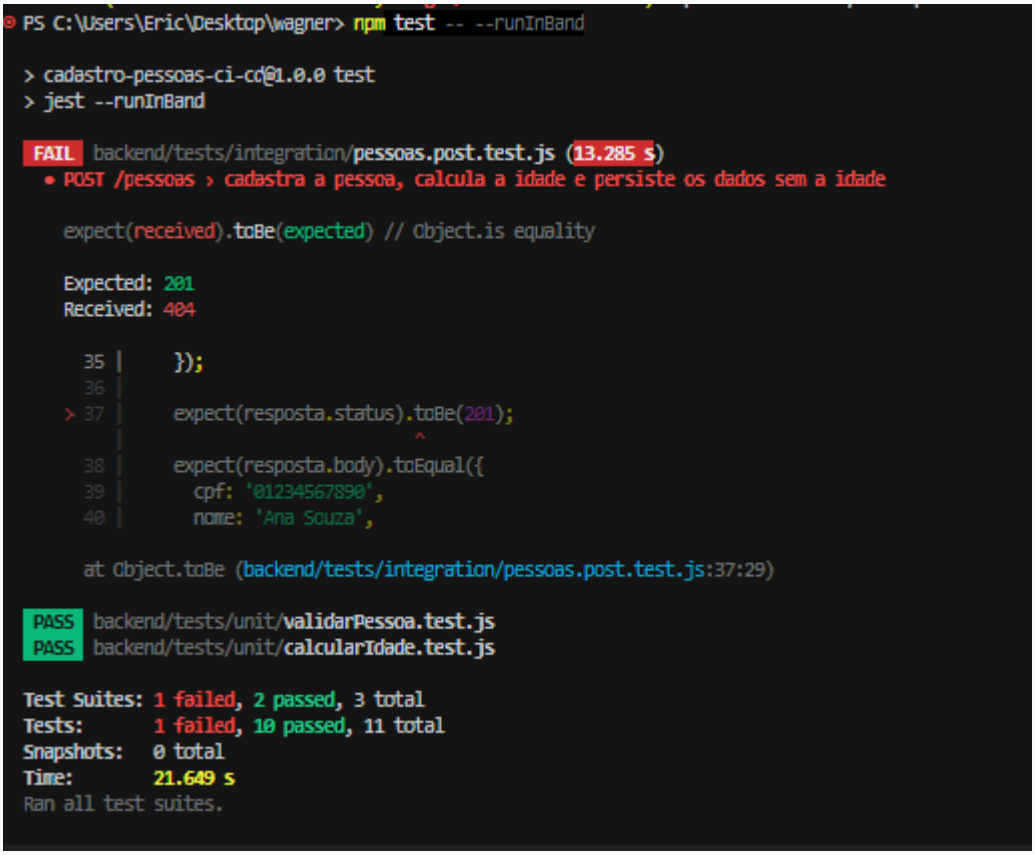
    expect(resposta.status).toBe(201);
    expect(resposta.body).toEqual({
      cpf: '01234567890',
      nome: 'Ana Souza',
      dataNascimento: '2000-05-20',
      idade: 26,
    });

    const cadastros = JSON.parse(fs.readFileSync(arquivoDados, 'utf8'));
    expect(cadastros).toEqual([
      {
        cpf: '01234567890',
        nome: 'Ana Souza',
        dataNascimento: '2000-05-20',
      },
    ]);
  });
```

ETAPA 6 - RED

Falha de comportamento

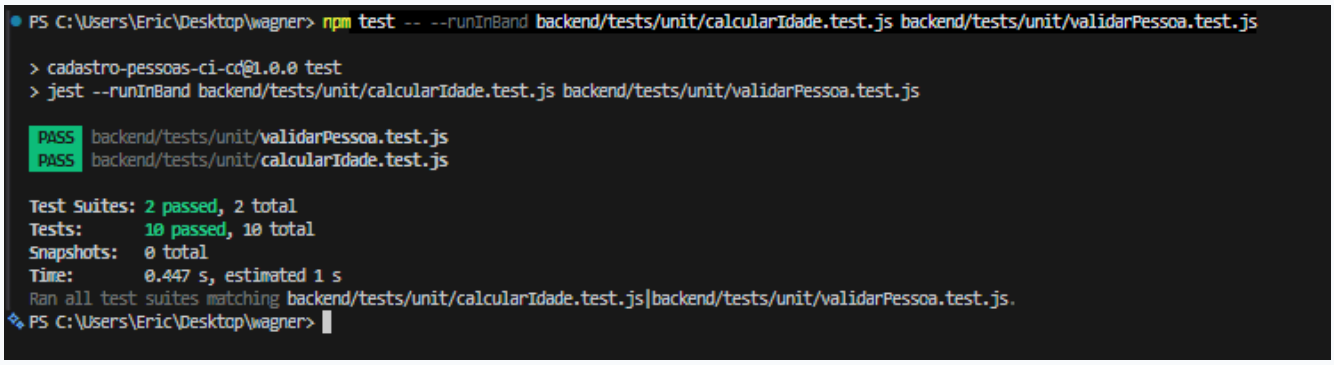
O POST recebeu 404 quando o teste esperava 201; os dez testes anteriores continuaram passando.



ETAPA 6 - RED

Testes anteriores preservados

A execução isolada confirmou que os dez testes de idade e validação continuavam passando.



ETAPA 6 - GREEN

Cadastro implementado

A rota POST valida, normaliza o nome, grava o cadastro e retorna a idade calculada.

app.js U

backend > src > app.js > ...

```
const express = require('express');
const { calcularIdade } = require('./calcularIdade');
const { validarPessoa } = require('./validarPessoa');
const { lerPessoas, salvarPessoas } = require('./armazenamentoPessoas');

function criarApp({ arquivoDados, obterDataReferencia }) {
  const app = express();

  app.locals.arquivoDados = arquivoDados;
  app.locals.obterDataReferencia = obterDataReferencia;
  app.use(express.json());

  app.post('/pessoas', (requisicao, resposta) => {
    const entrada = requisicao.body ?? {};
    const pessoa = {
      cpf: entrada.cpf,
      nome:
        typeof entrada.nome === 'string' ? entrada.nome.trim() : entrada.nome,
      dataNascimento: entrada.dataNascimento,
    };
    const dataReferencia = obterDataReferencia();
    const erro = validarPessoa(pessoa, dataReferencia);

    if (erro) {
      return resposta.status(400).json({ erro });
    }

    const pessoas = lerPessoas(arquivoDados);
    pessoas.push(pessoa);
    salvarPessoas(arquivoDados, pessoas);

    return resposta.status(201).json({
      ...pessoa,
      idade: calcularIdade(pessoa.dataNascimento, dataReferencia),
    });
  });

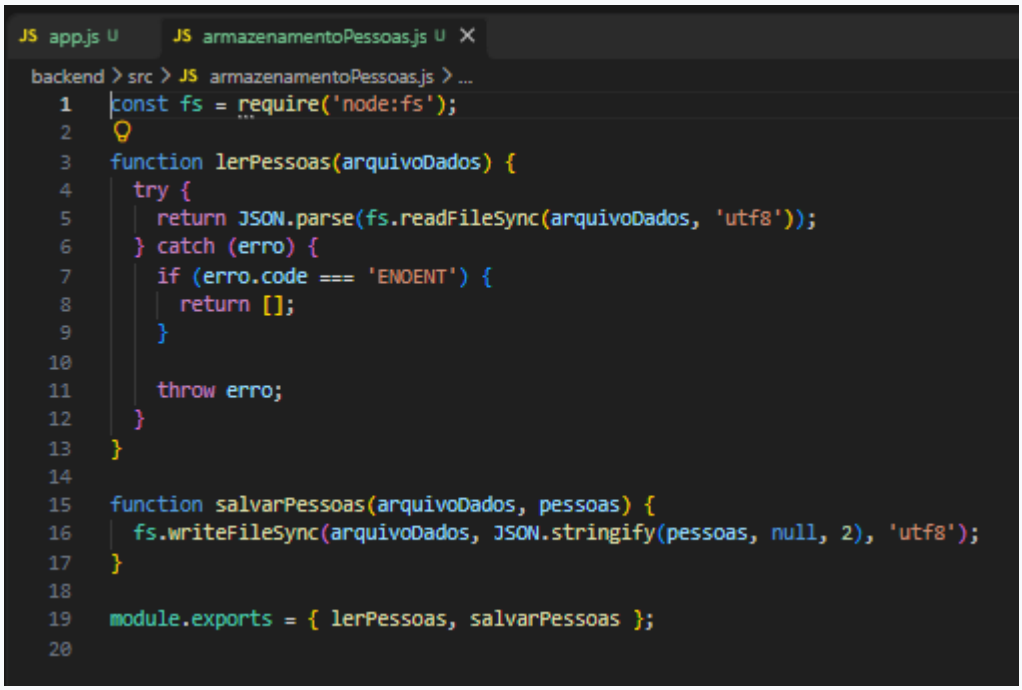
  return app;
}

module.exports = { criarApp };
```

ETAPA 6 - GREEN

Armazenamento JSON

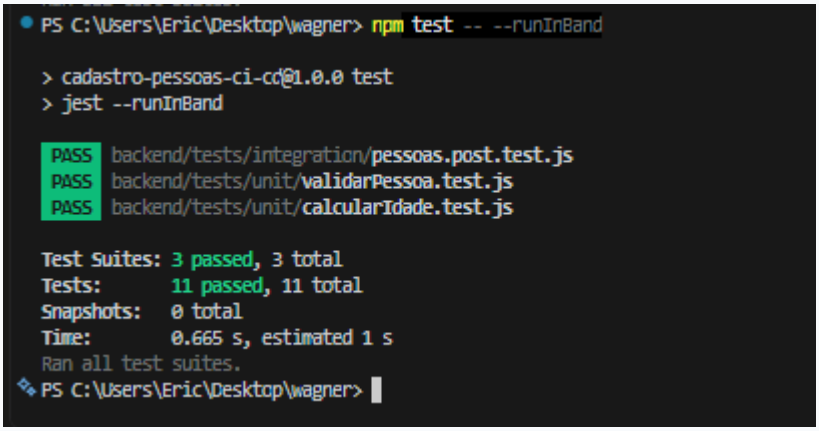
A leitura e a gravação usam arquivo JSON local; a idade não é persistida.



ETAPA 6 - GREEN

Primeiro ciclo aprovado

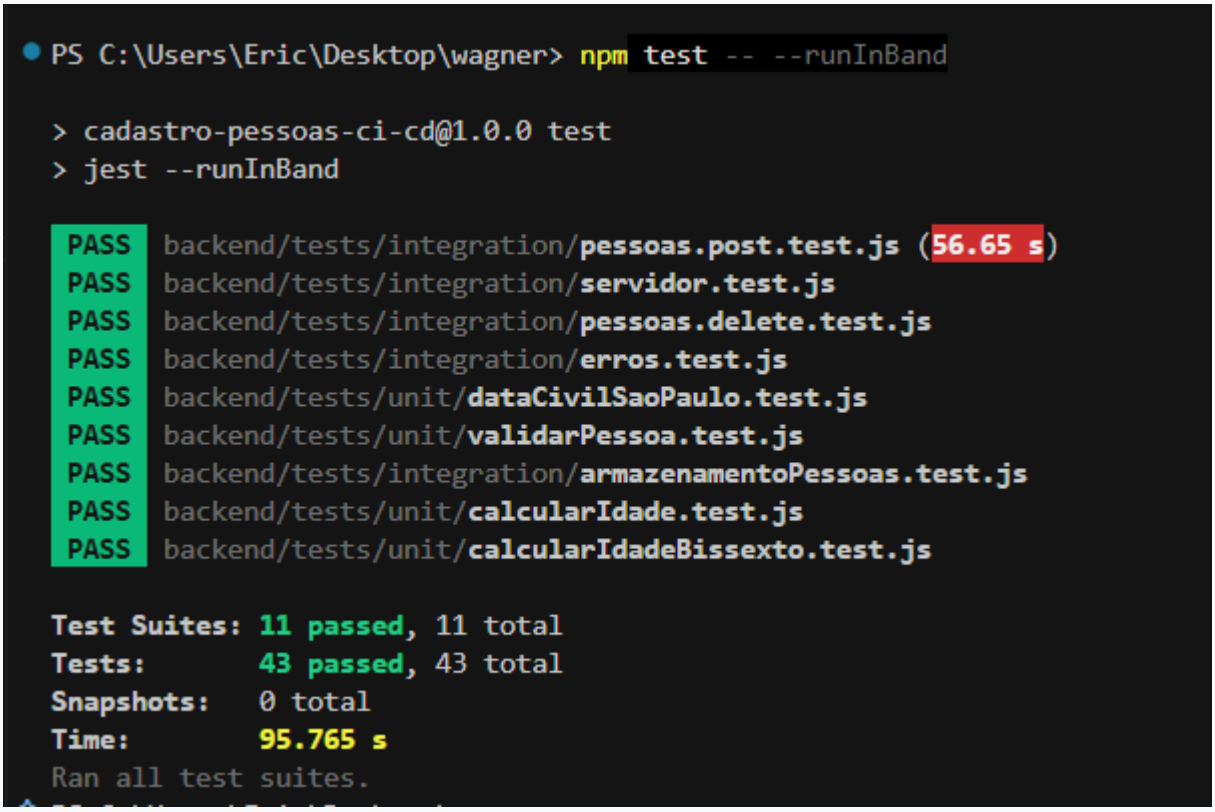
Depois da implementação do POST, onze testes passaram: o novo teste e os dez anteriores.



ETAPA 6 - TESTES

Suíte completa do backend

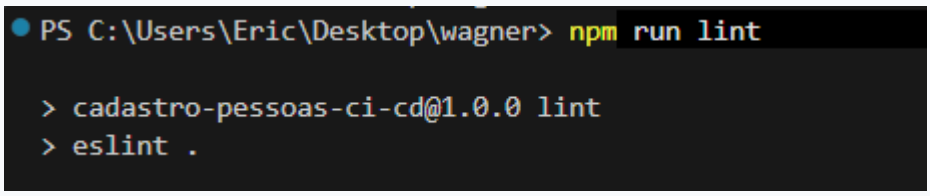
Após os ciclos de listagem, edição, exclusão, armazenamento e erros, 43 testes passaram.



ETAPA 6 - QUALIDADE

Análise estática

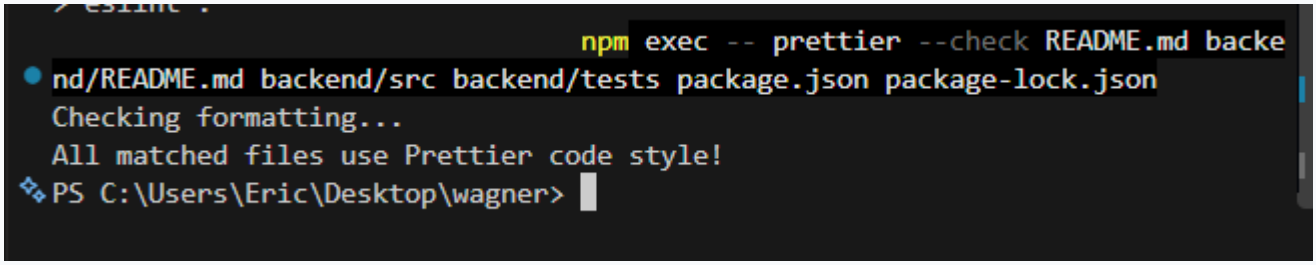
O ESLint foi executado para o projeto; a verificação concluída não apontou erros.



ETAPA 6 - QUALIDADE

Formatação dos arquivos deste bloco

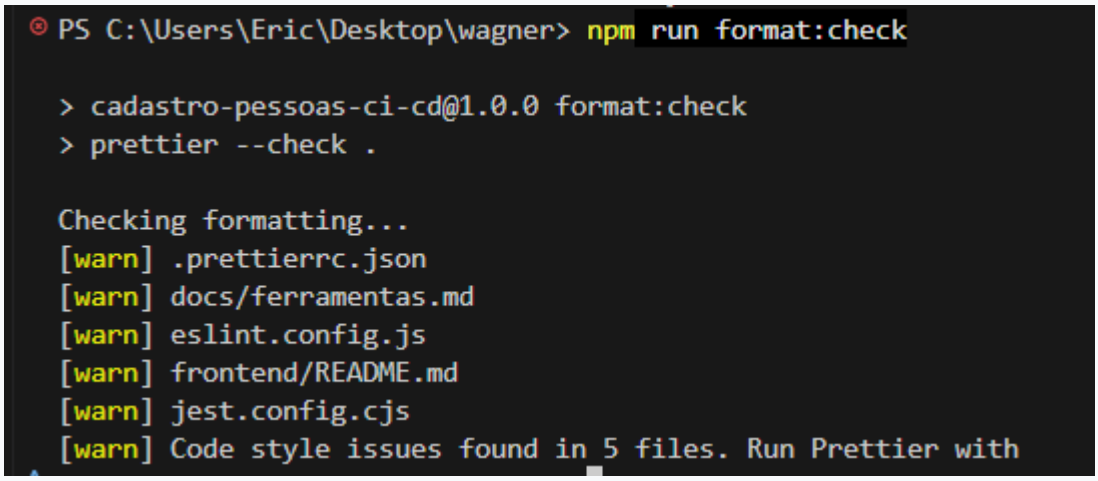
O Prettier confirmou o padrão de código nos arquivos alterados e criados para o backend.



ETAPA 6 - PENDÊNCIA

Formatação geral

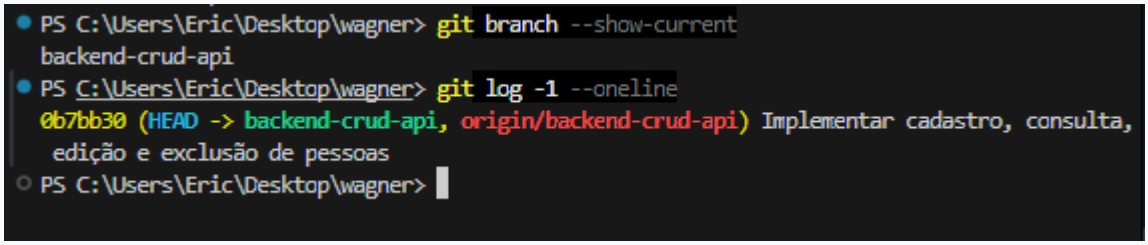
A verificação geral ainda aponta cinco arquivos anteriores; esse resultado não foi apresentado como aprovação.



ETAPA 6 - GIT

Branch e commit

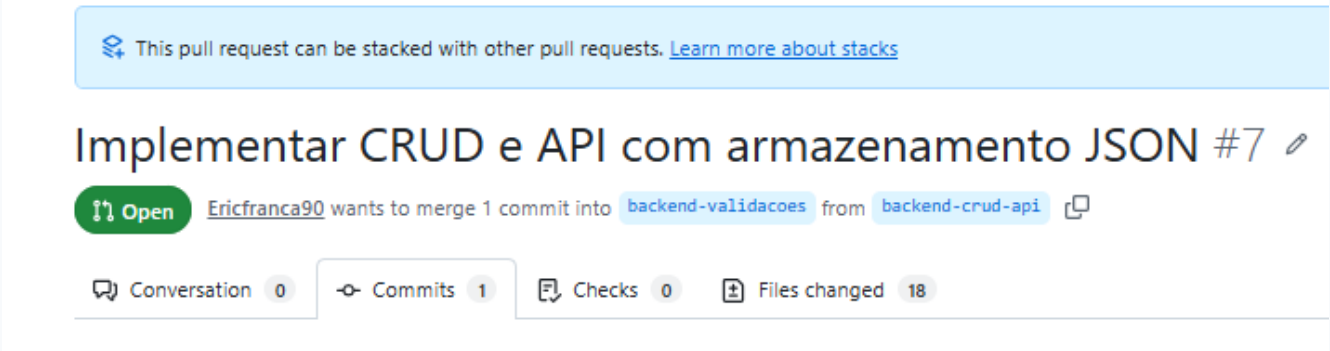
A branch backend-crud-api registra o commit 0b7bb30 com o código e os testes.



ETAPA 6 - GIT

Pull request aberto

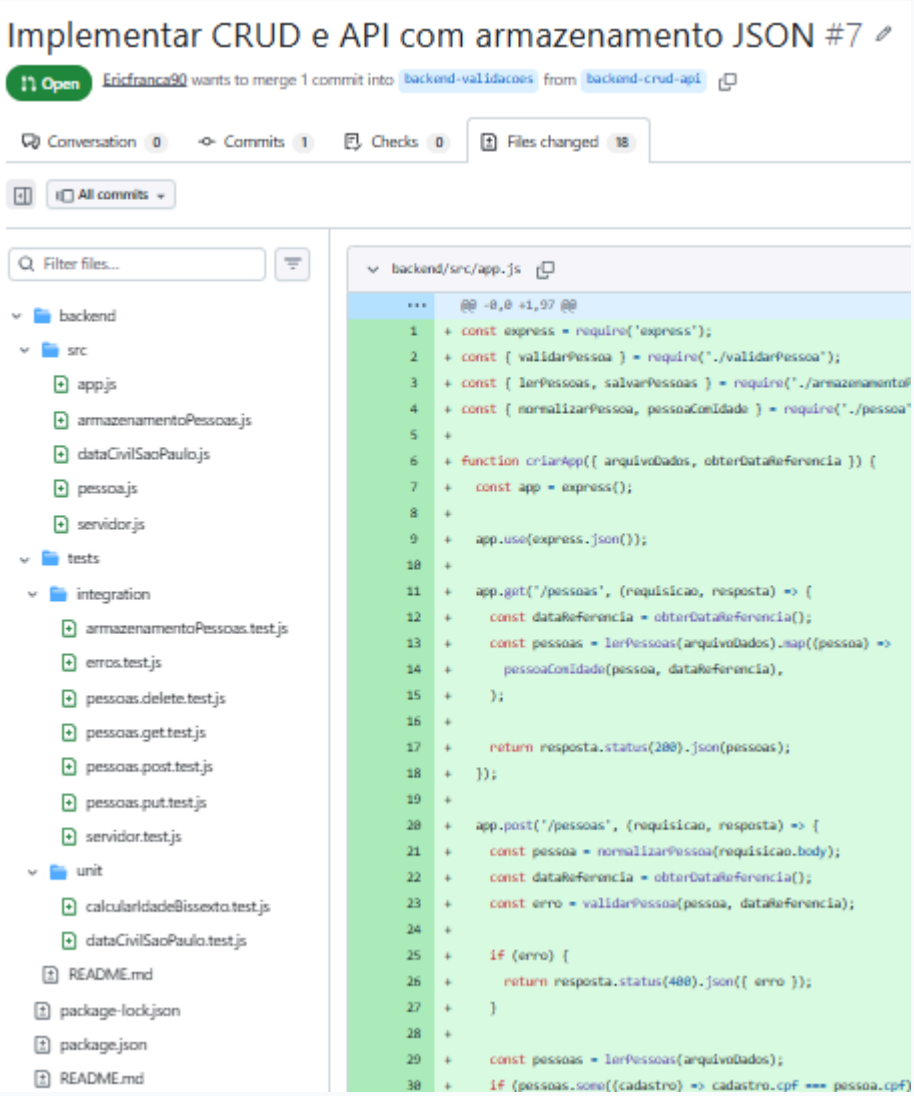
O PR #7 foi aberto de backend-crud-api para backend-validacoes e depende do PR #6.



ETAPA 6 - GIT

Comparação do pull request

A aba Files changed mostra os 18 arquivos deste bloco, sem alterações no frontend.

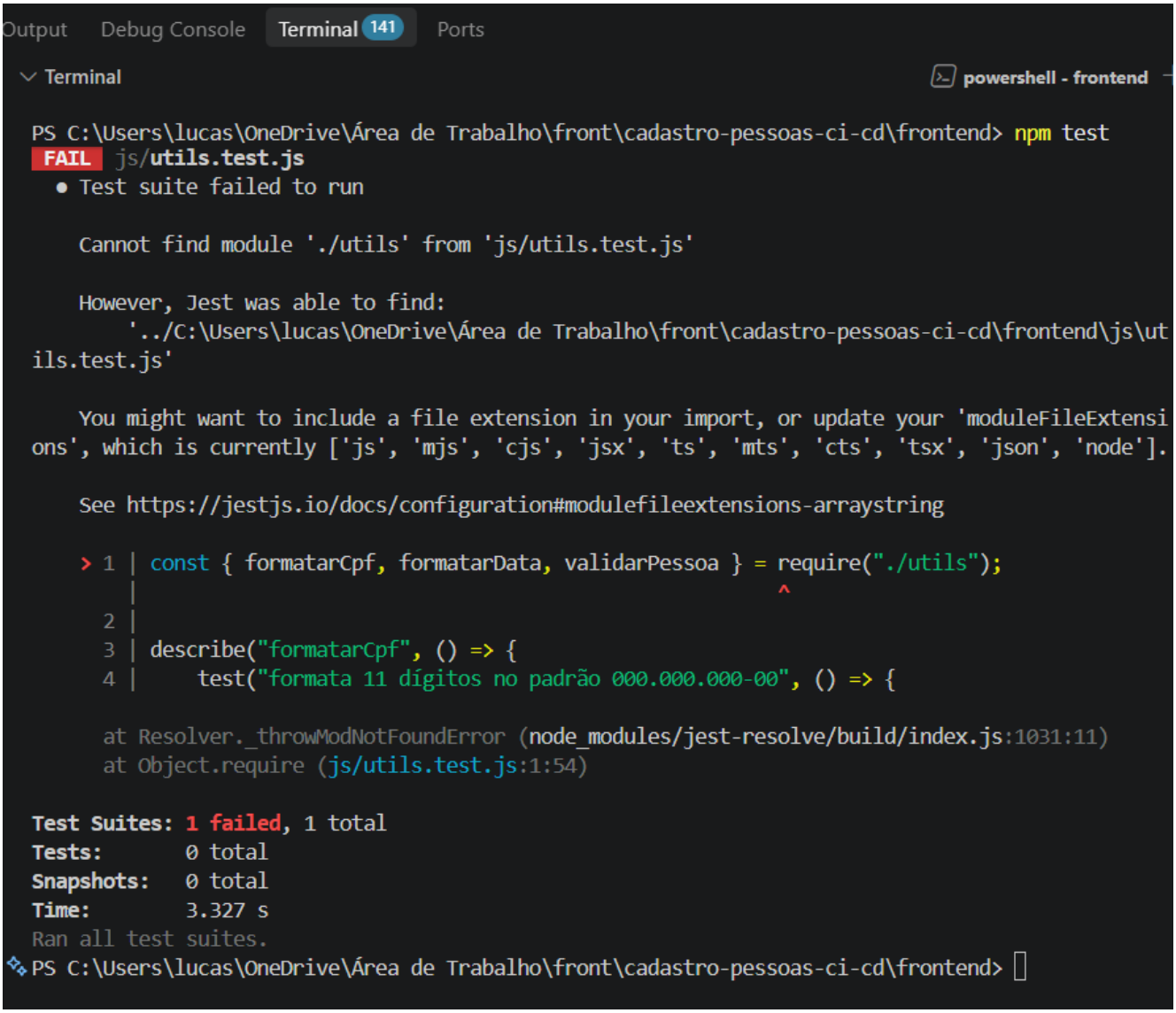


Cadastro de Pessoas

Evidências do Frontend

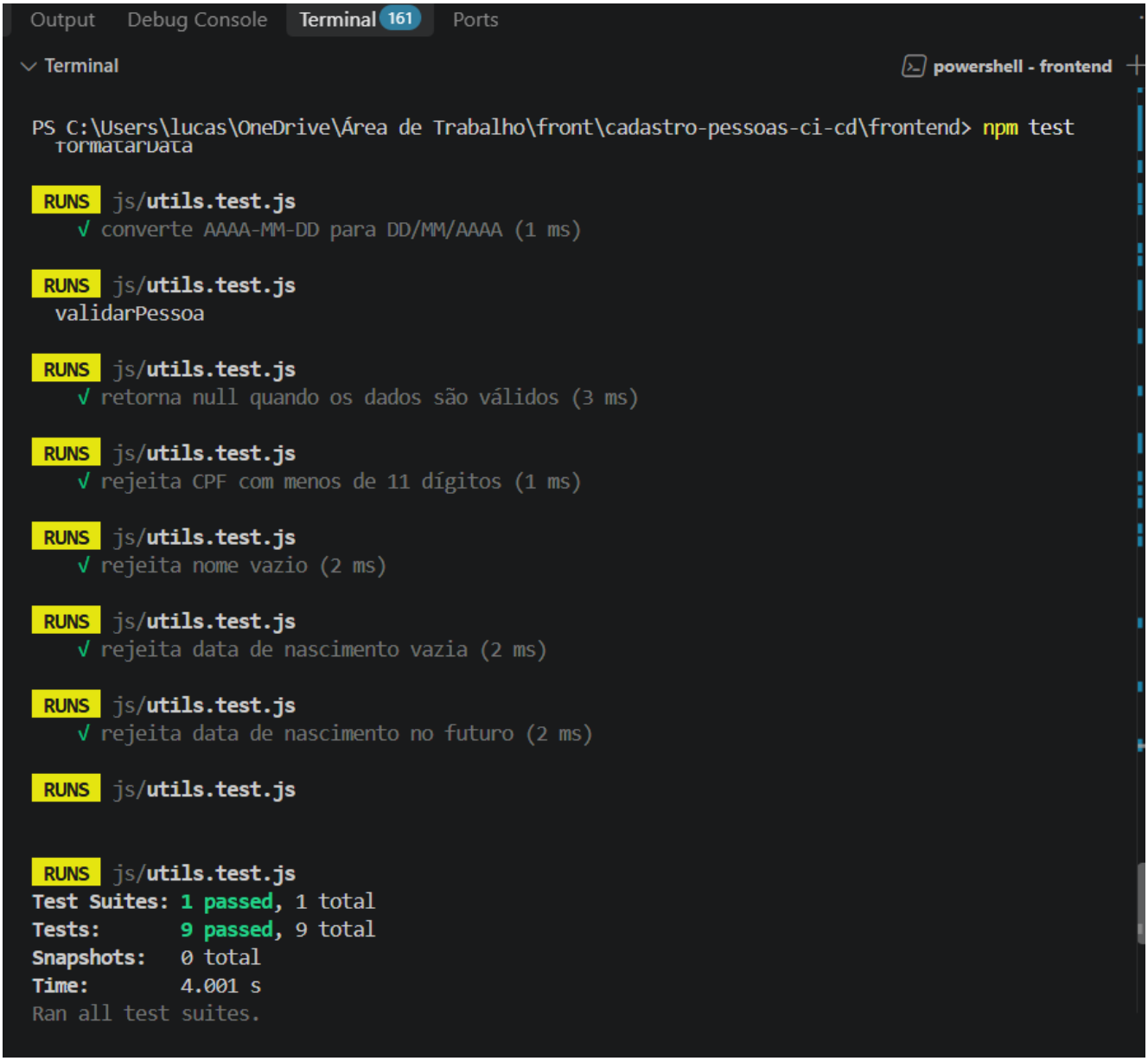
Integração e Entrega Contínua

utils.js - Red



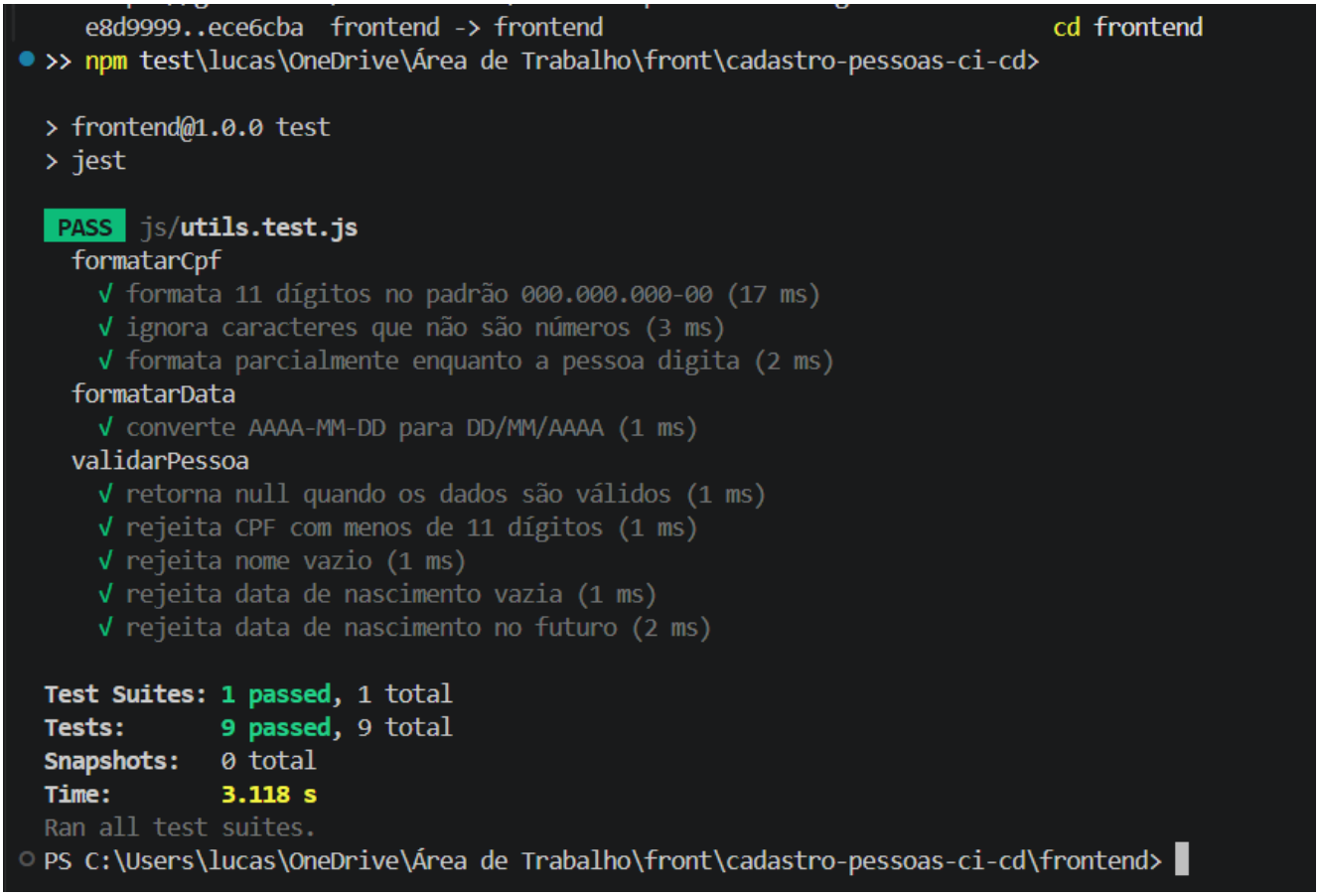
Ciclo TDD, etapa Red: testes de formatarCpf, formatarData e validarPessoa falhando, pois o arquivo utils.js ainda não foi criado.

utils.js - Green



Ciclo TDD, etapa Green: os 9 testes de formatarCpf, formatarData e validarPessoa passando após a criação do utils.js.

utils.js - Refactor



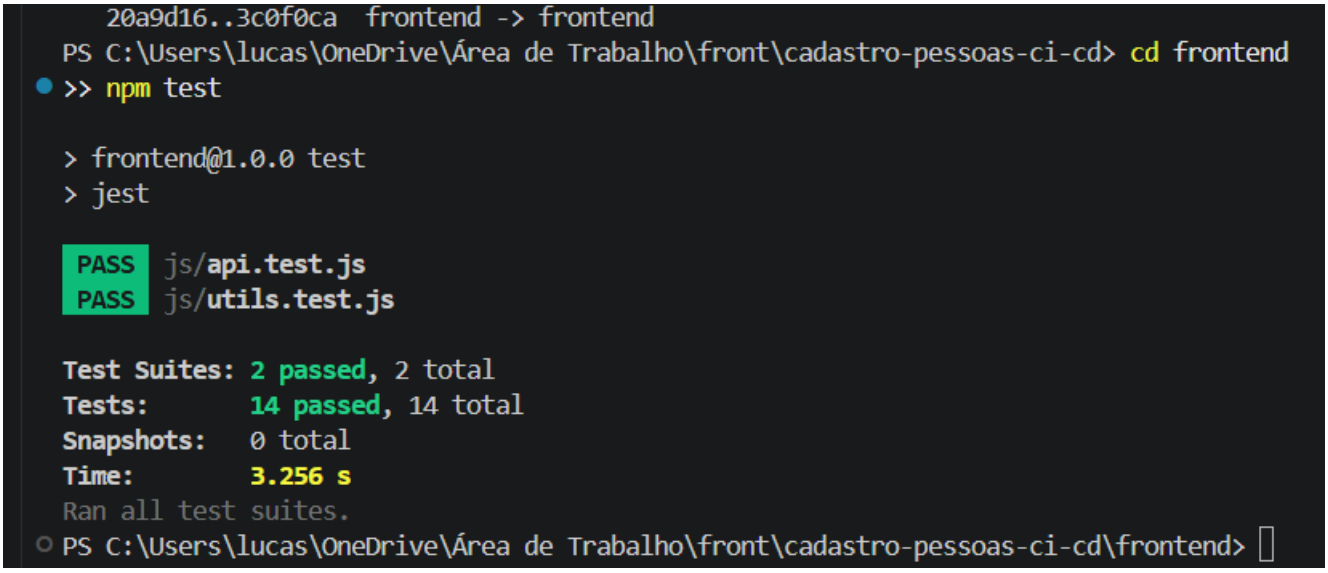
Ciclo TDD, etapa Refactor: os 9 testes continuam passando após a refatoração do utils.js (helper somenteDigitos e constante TAMANHO_CPF).

api.js - Red



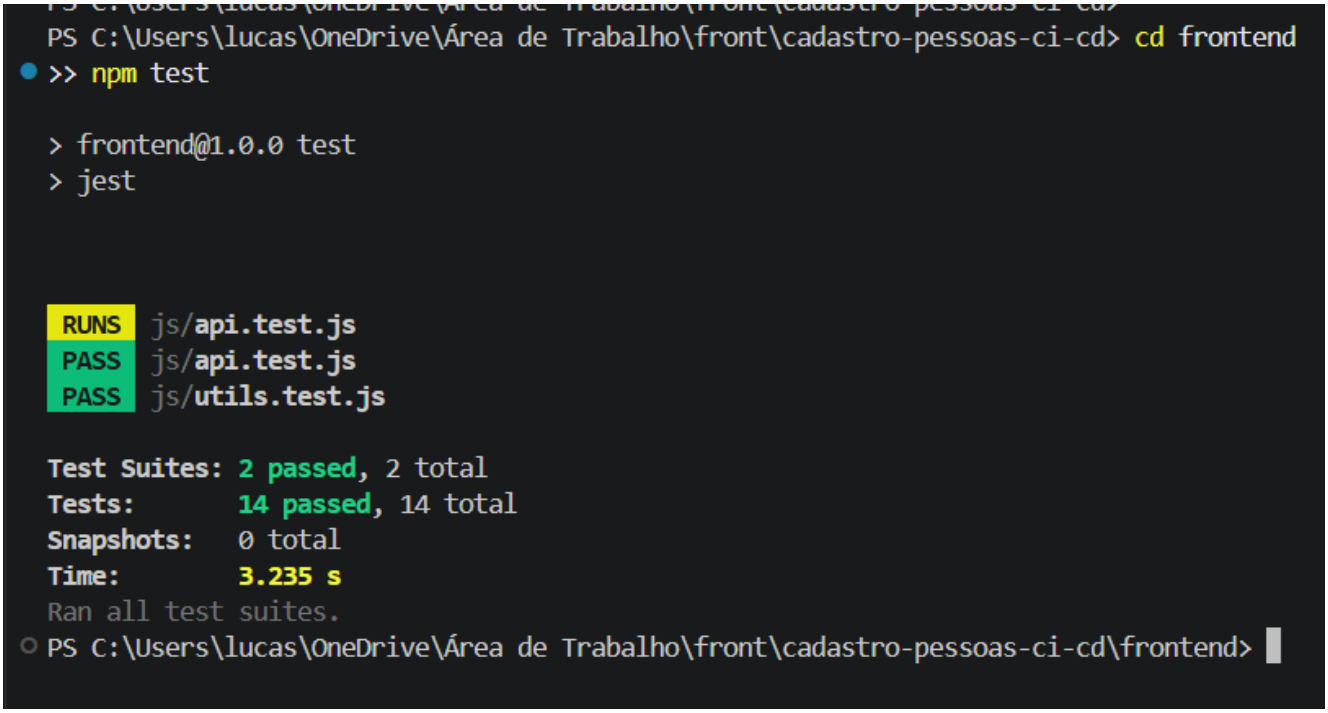
Ciclo TDD (api.js), etapa Red: o teste api.test.js falha porque o módulo api.js ainda não existe. Os 9 testes do utils continuam passando.

api.js - Green



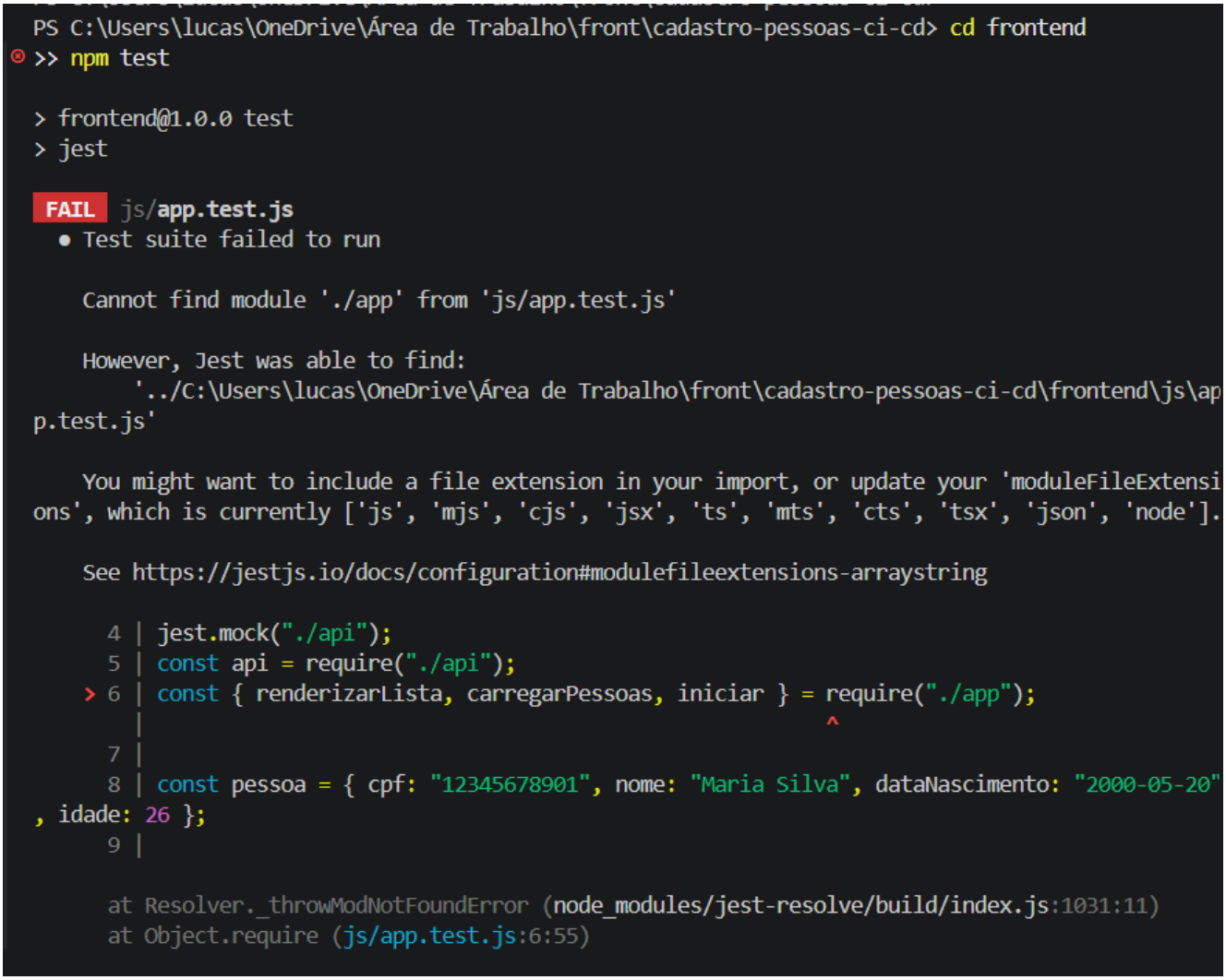
Ciclo TDD (api.js), etapa Green: os 5 testes de api.test.js passam após a criação do api.js, somando 14 testes aprovados com os do utils.

api.js - Refactor



Ciclo TDD (api.js), etapa Refactor: os 14 testes continuam passando após a refatoração do api.js (função enviarJson reaproveitada em POST e PUT).

app.js - Red



Ciclo TDD (app.js), etapa Red: o teste app.test.js, que cobre a listagem, o cadastro, a edição e a exclusão, falha porque o módulo app.js ainda não existe.

app.js - Green

```
>> npm test\lucas\OneDrive\Área de Trabalho\front\cadastro-pessoas-ci-cd>

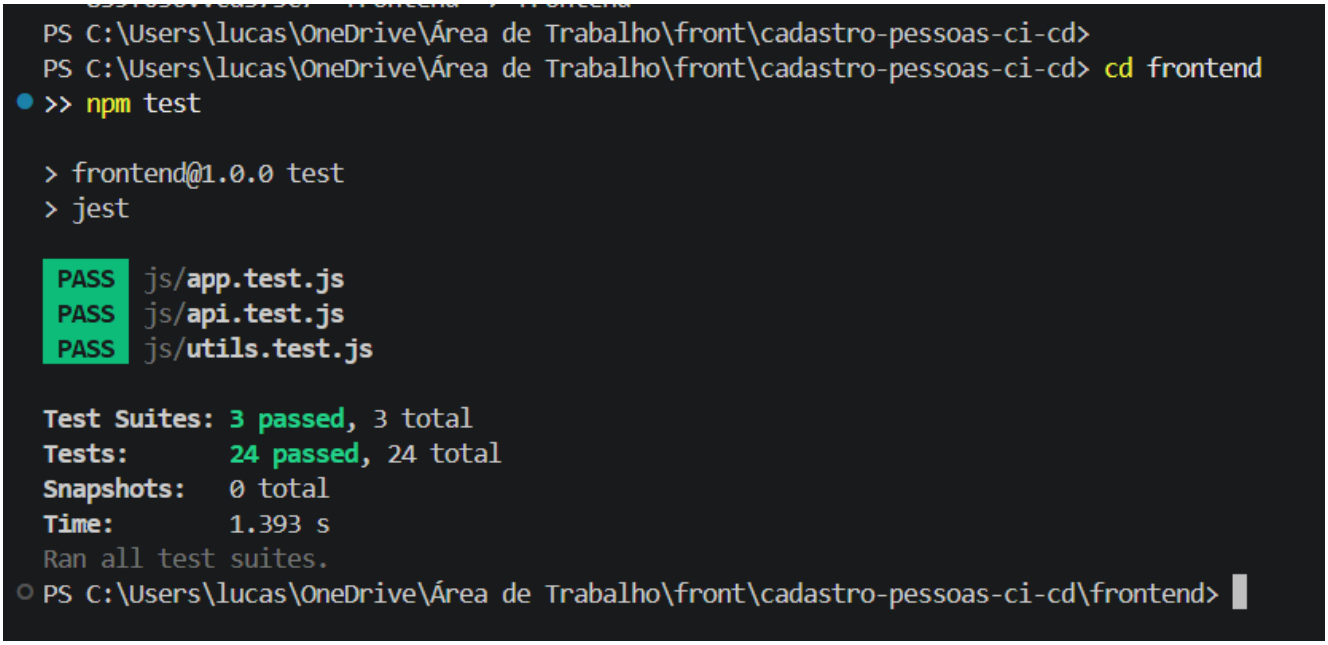
> frontend@1.0.0 test
> jest

 PASS  js/app.test.js
 PASS  js/api.test.js
 PASS  js/utils.test.js

Test Suites: 3 passed, 3 total
Tests:       24 passed, 24 total
Snapshots:   0 total
Time:        1.481 s
Ran all test suites.
PS C:\Users\lucas\OneDrive\Área de Trabalho\front\cadastro-pessoas-ci-cd\frontend>
```

Ciclo TDD (app.js), etapa Green: os testes de app.test.js (listagem, cadastro, edição e exclusão) passam após a criação do app.js, somando 24 testes aprovados nos 3 arquivos.

app.js - Refactor



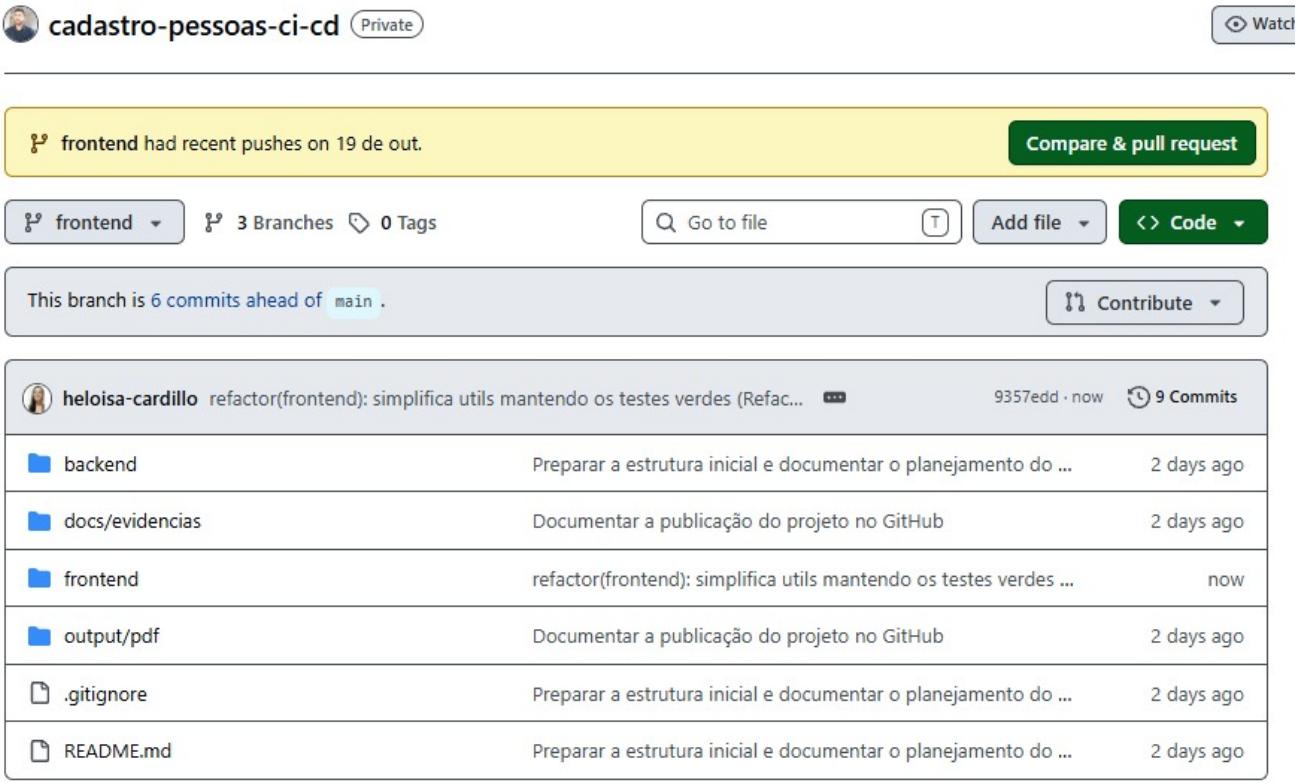
Ciclo TDD (app.js), etapa Refactor: os 24 testes continuam passando após a refatoração do app.js (funções lerFormulario, limparMensagem e salvarPessoa extraídas).

Interface



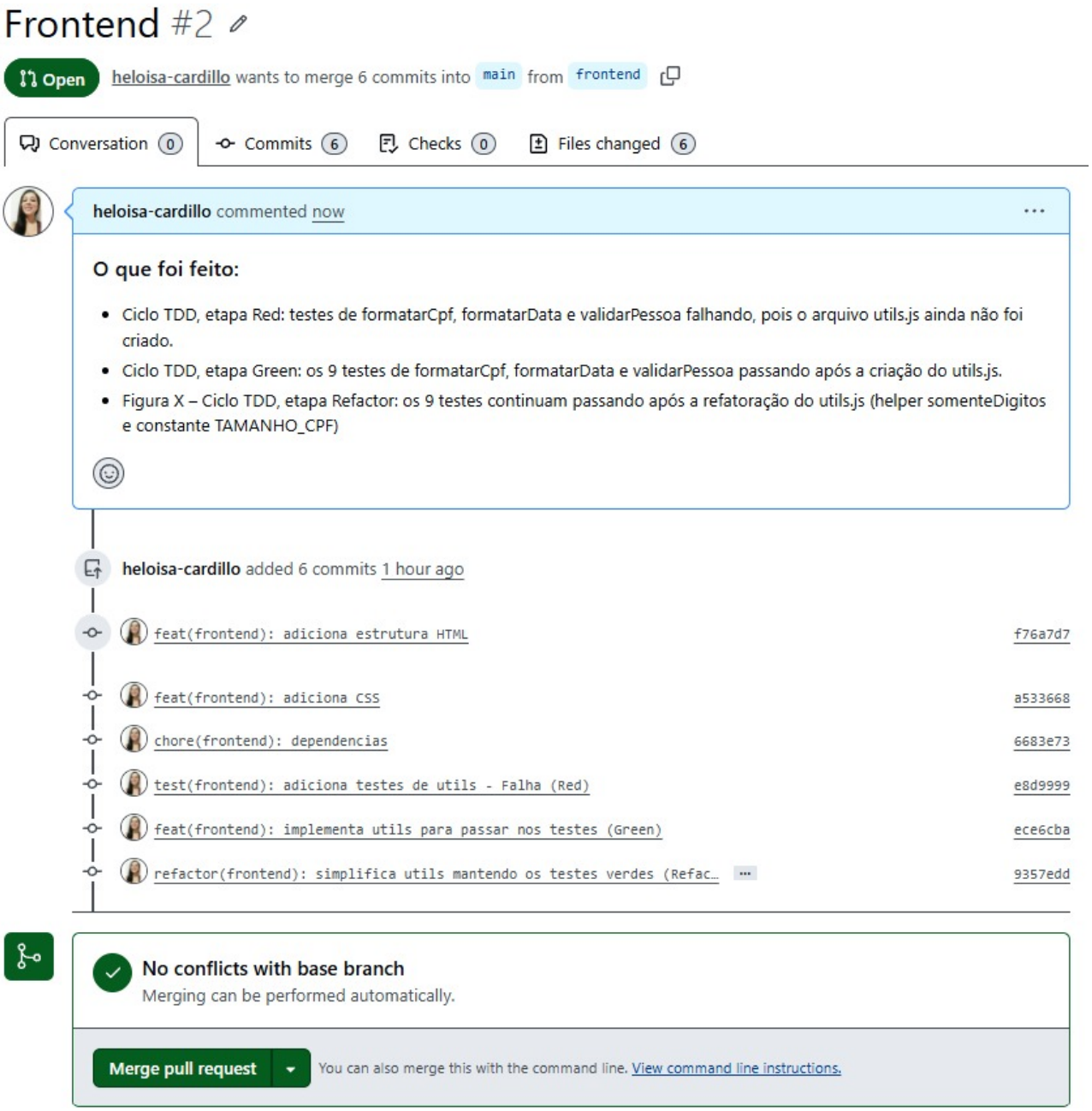
Frontend no navegador: tela de Cadastro de Pessoas com a máscara de CPF funcionando e a validação passando. A mensagem "Failed to fetch" aparece porque o backend ainda não está disponível para receber o cadastro.

GitHub - Branch



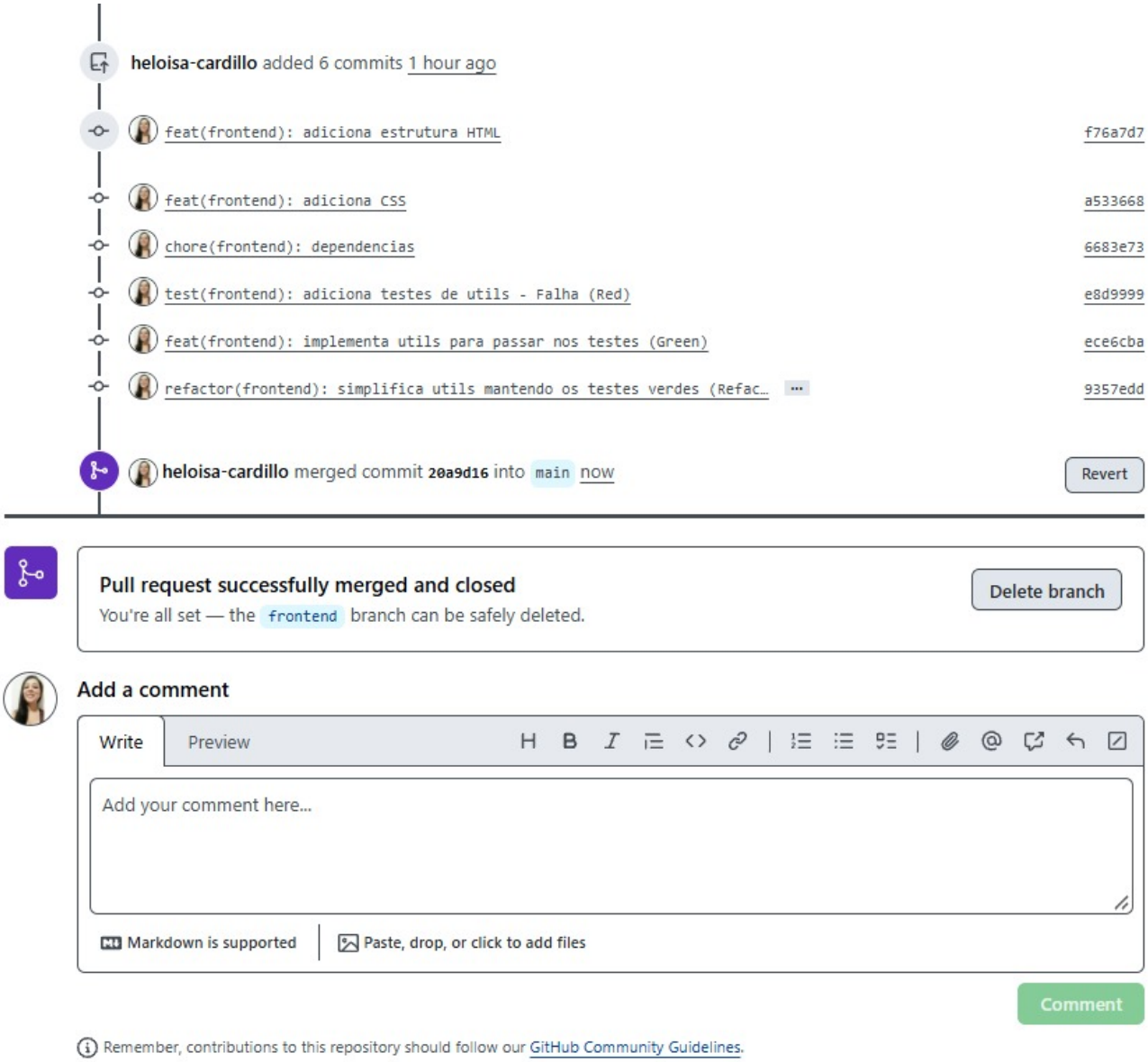
GitHub: branch frontend criada e enviada ao repositório, com 6 commits à frente da main.

GitHub - Pull request aberto



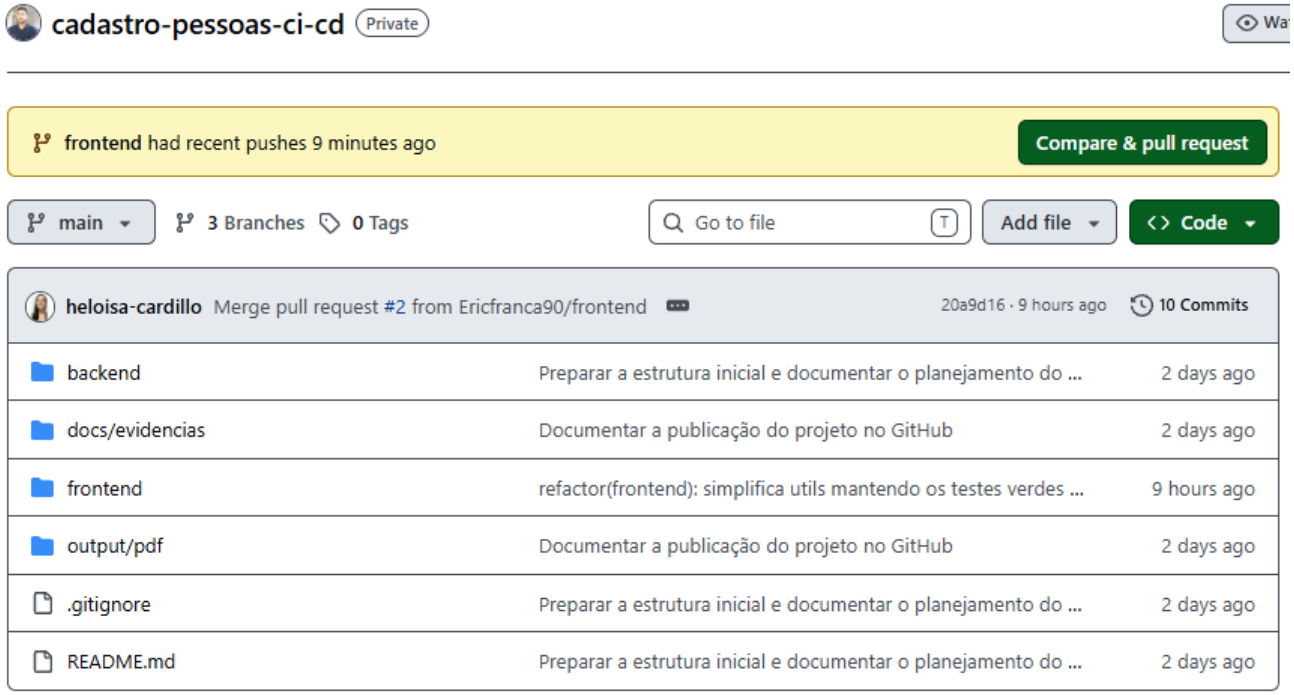
GitHub: pull request #2 (Frontend) aberto da branch frontend para a main, com os 6 commits da estrutura, do CSS e do ciclo TDD do utils, sem conflitos para o merge.

GitHub - Pull request mergeado



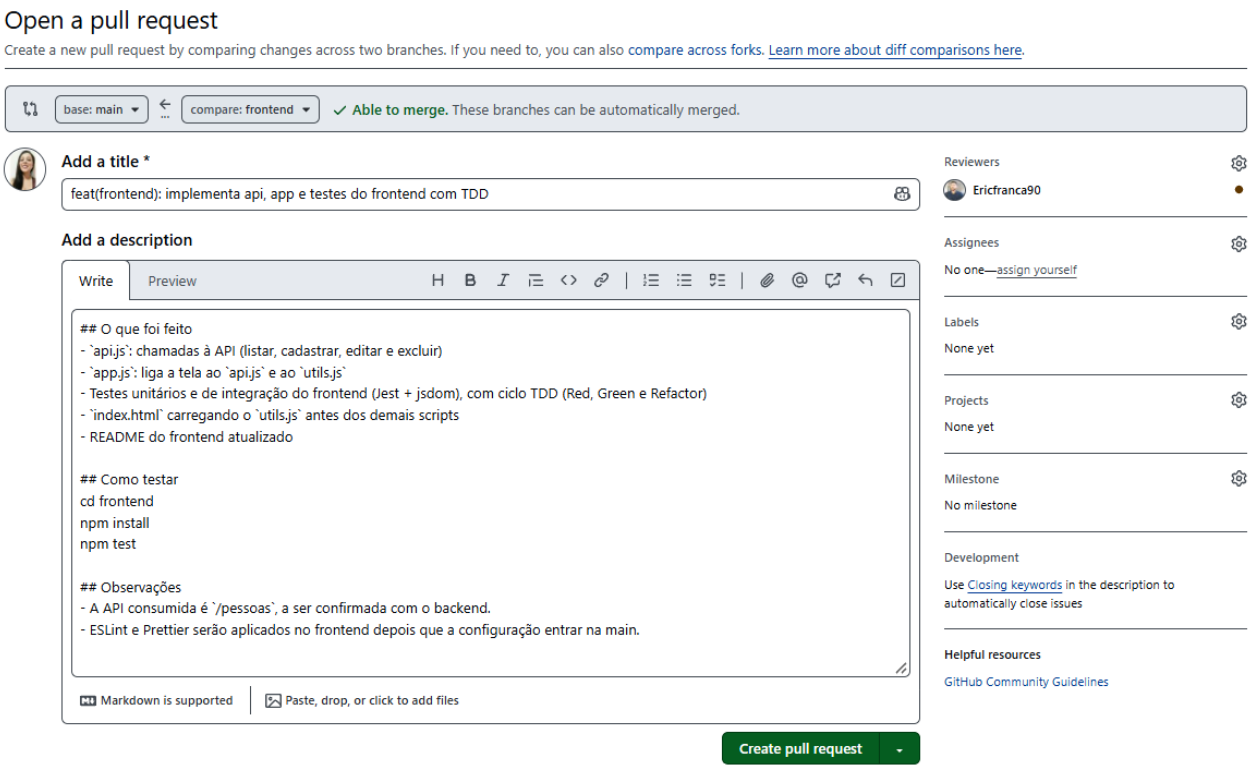
GitHub: pull request #2 mergeado na main e fechado com sucesso.

GitHub - Novo pull request



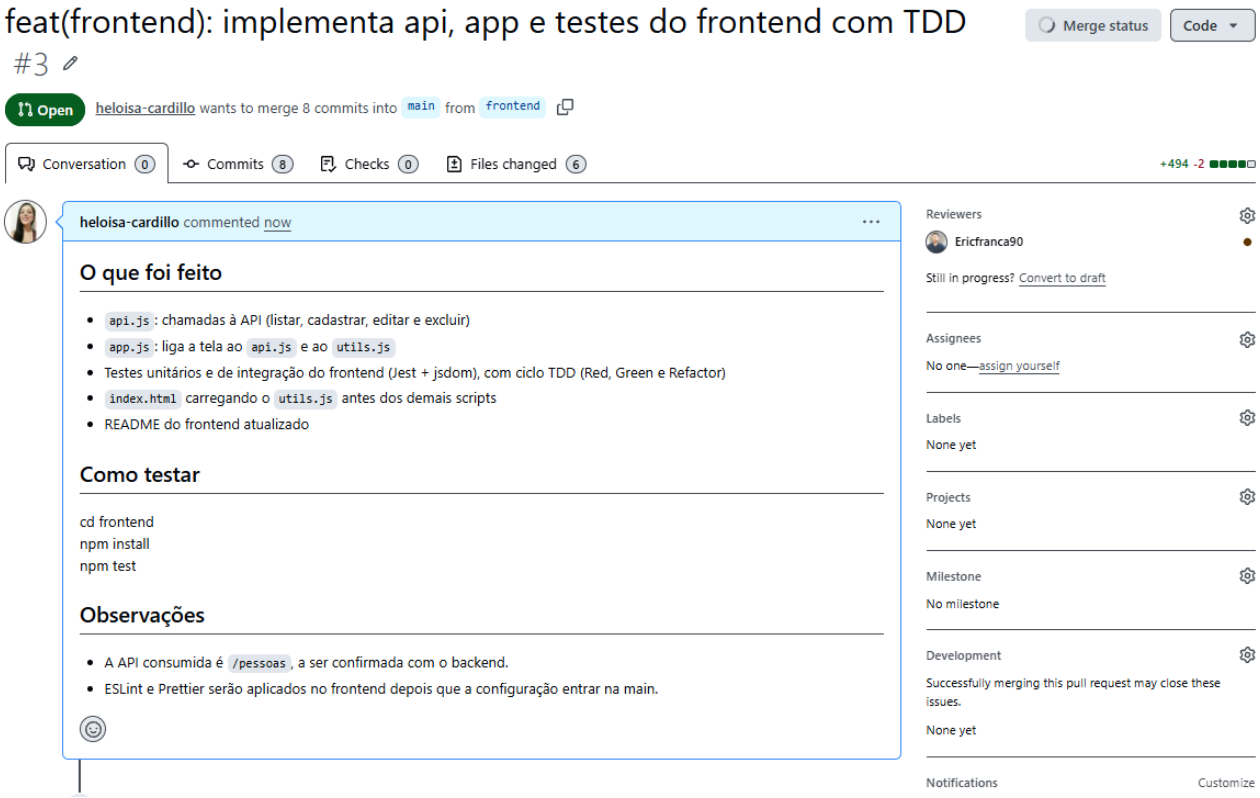
GitHub: repositório com o aviso de novos pushes na branch frontend e o botão Compare & pull request.

GitHub - Abertura do pull request



GitHub: abertura do pull request de frontend para a main, com título, descrição, Eric como revisor e a indicação de que as branches podem ser mescladas automaticamente.

GitHub - Pull request #3 aberto



GitHub: pull request #3 aberto da branch frontend para a main, com 8 commits e 6 arquivos alterados, aguardando a revisão do Eric.

ETAPA 7

Integração frontend/backend

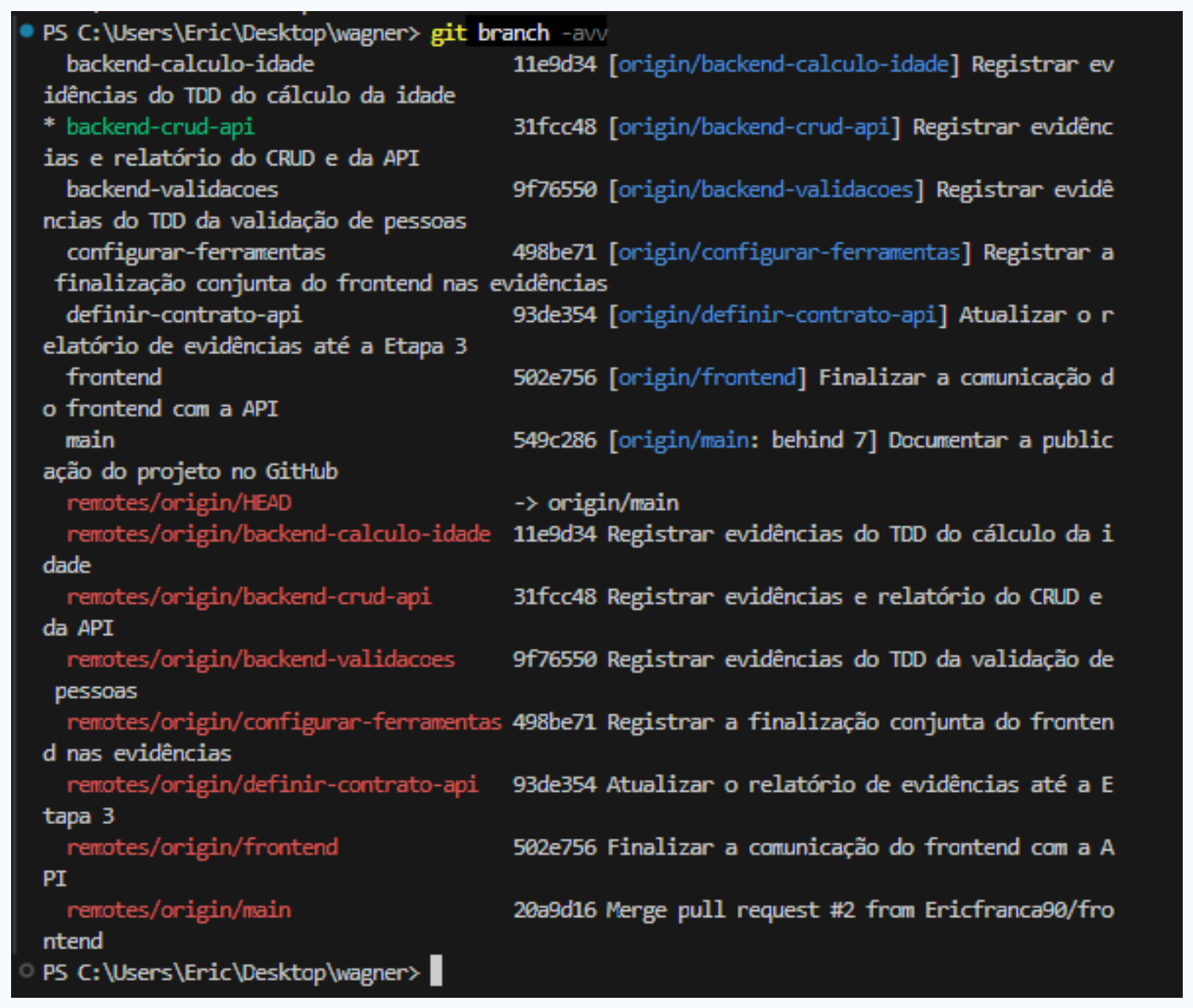
- O frontend da Heloisa foi incorporado ao histórico do backend por um commit de merge, preservando a autoria anterior.
- O Express disponibiliza a página, o CSS e os scripts; a rota /pessoas continua respondendo como API JSON.
- O teste foi escrito antes da implementação: a interface retornou 404 no RED e os três testes passaram no GREEN.
- No navegador, foram verificados cadastro, rejeição de CPF duplicado, edição, persistência após F5 e exclusão, usando JSON de teste isolado.
- Ao fim, passaram 46 testes do backend, 24 do frontend, ESLint e Prettier dos arquivos deste ciclo.
- O PR #8 permanece aberto e depende do PR #7; o PR #3 do frontend também permanece aberto. Não houve merge de PRs.
- A formatação geral continua pendente em 15 arquivos. Ela não está apresentada como aprovada.

As imagens seguintes são capturas reais da etapa de integração.

ETAPA 7 - INTEGRAÇÃO

Branches antes da integração

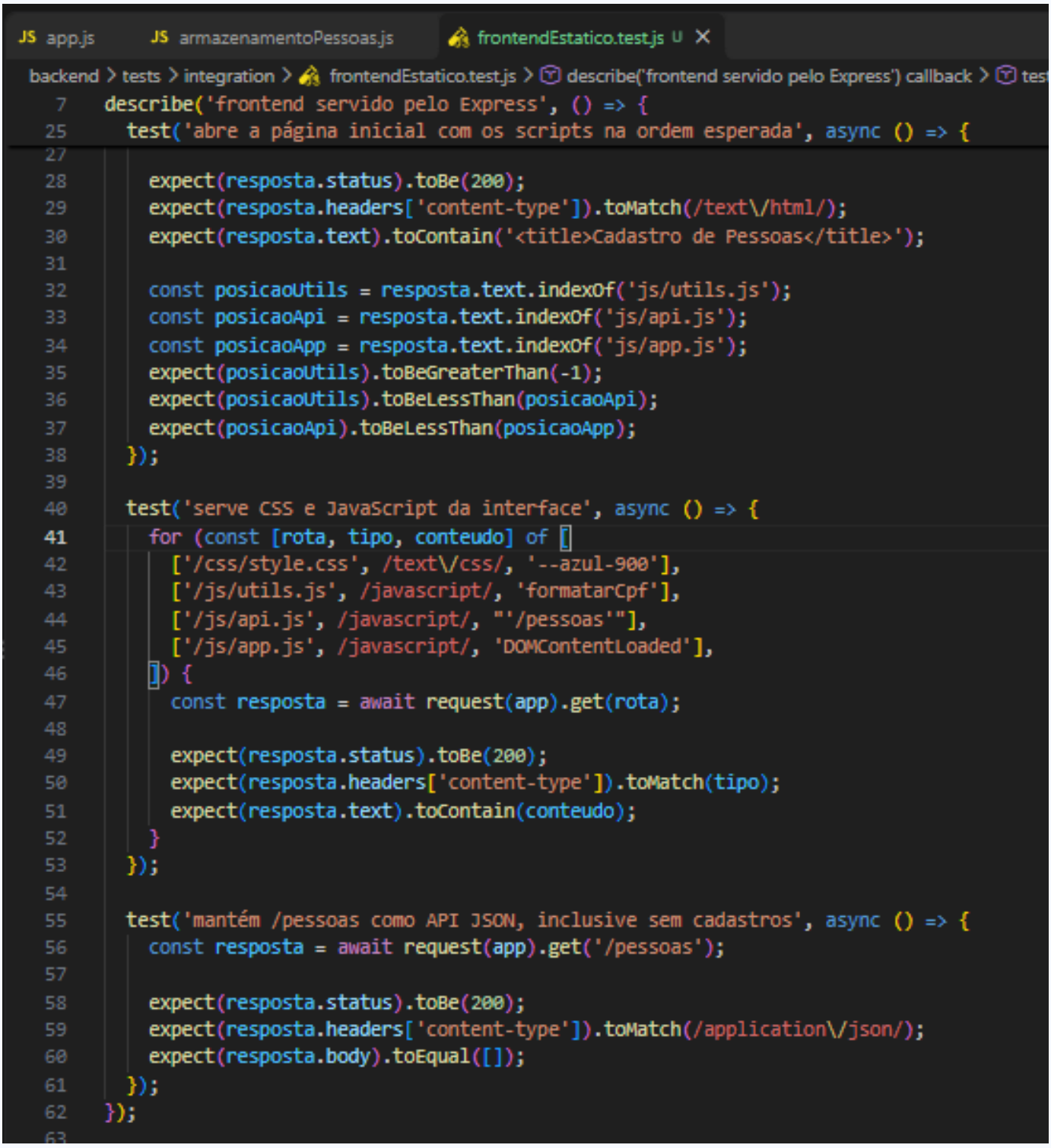
Referências do backend e do frontend conferidas antes de juntar o trabalho.



ETAPA 7 - INTEGRAÇÃO

Teste escrito primeiro

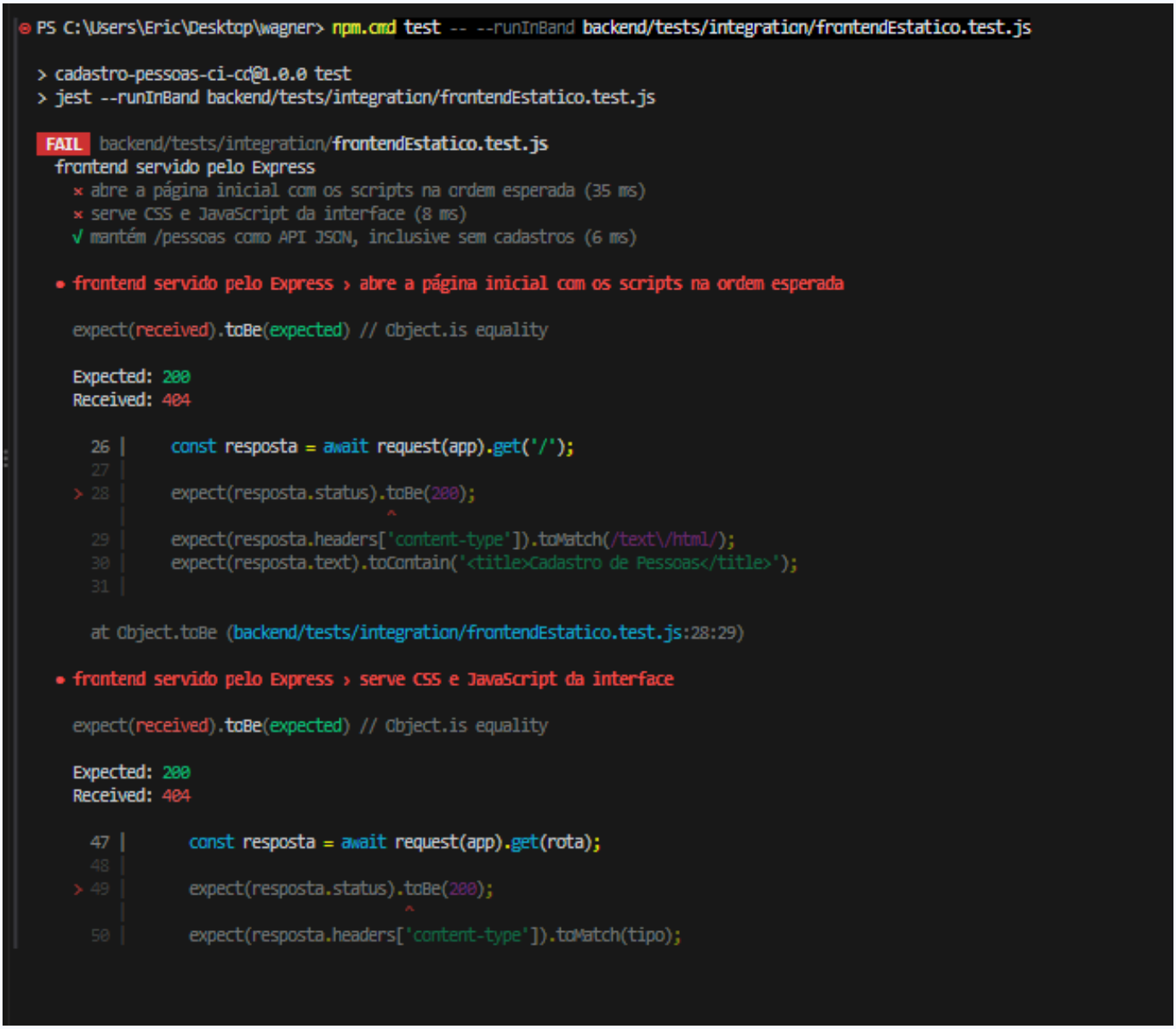
O teste exige a página inicial, os arquivos estáticos e a API JSON.



ETAPA 7 - INTEGRAÇÃO

RED - falhas esperadas

Sem servir a interface, a página e os arquivos estáticos retornaram 404.



ETAPA 7 - INTEGRAÇÃO

RED - resultado do teste

Duas falhas por 404 e o teste da API aprovado, sem erro de configuração.



ETAPA 7 - INTEGRAÇÃO

GREEN - código de integração

O Express passa a servir os arquivos do frontend após as rotas da API.

```
app.use(express.static(path.join(__dirname, '..', '..', 'frontend')));

app.use((requisicao, resposta) => {
  return resposta.status(404).json({ erro: 'Rota não encontrada.' });
});

app.use((erro, requisicao, resposta, proximo) => {
  if (resposta.headersSent) {
    return proximo(erro);
  }

  if (erro.type === 'entity.parse.failed') {
    return resposta.status(400).json({ erro: 'JSON inválido.' });
  }

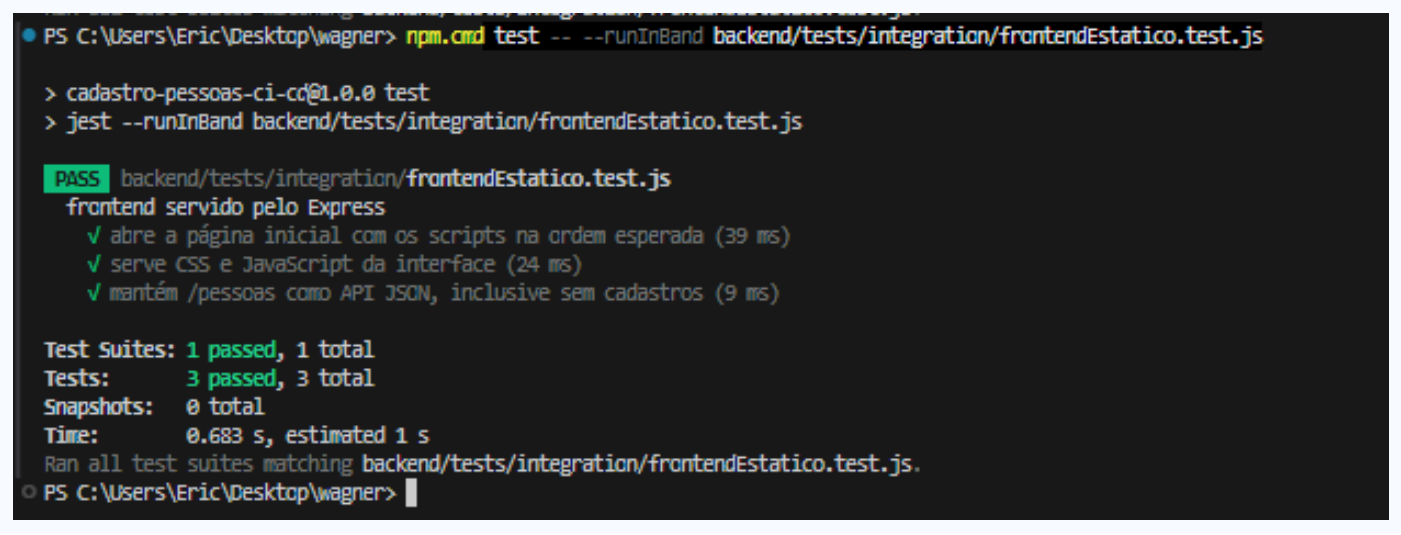
  return resposta.status(500).json({ erro: 'Erro interno do servidor.' });
});

return app;
}
```

ETAPA 7 - INTEGRAÇÃO

GREEN - testes de integração

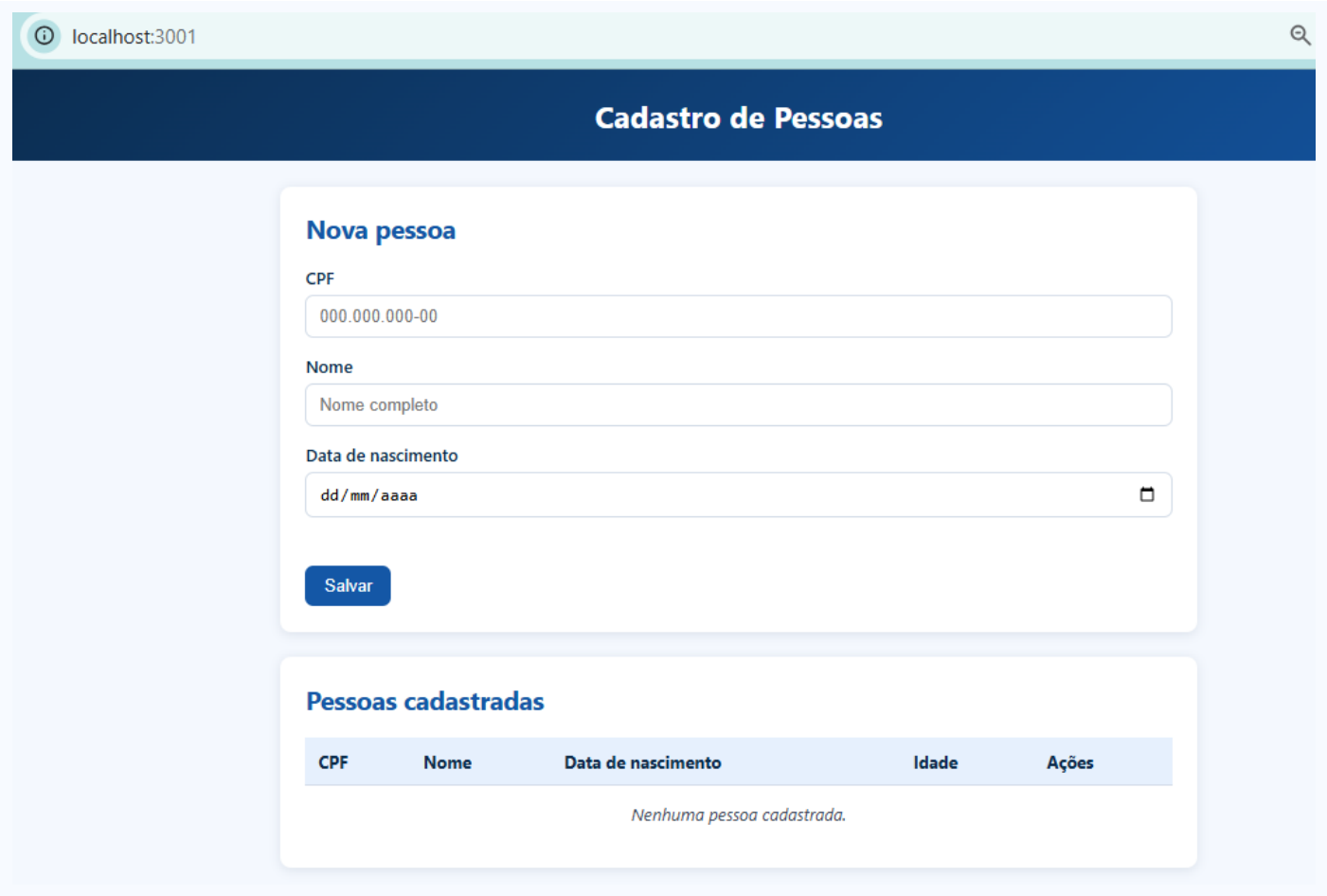
Os três testes da página, dos arquivos estáticos e da API passaram.



ETAPA 7 - INTEGRAÇÃO

Aplicação: lista vazia

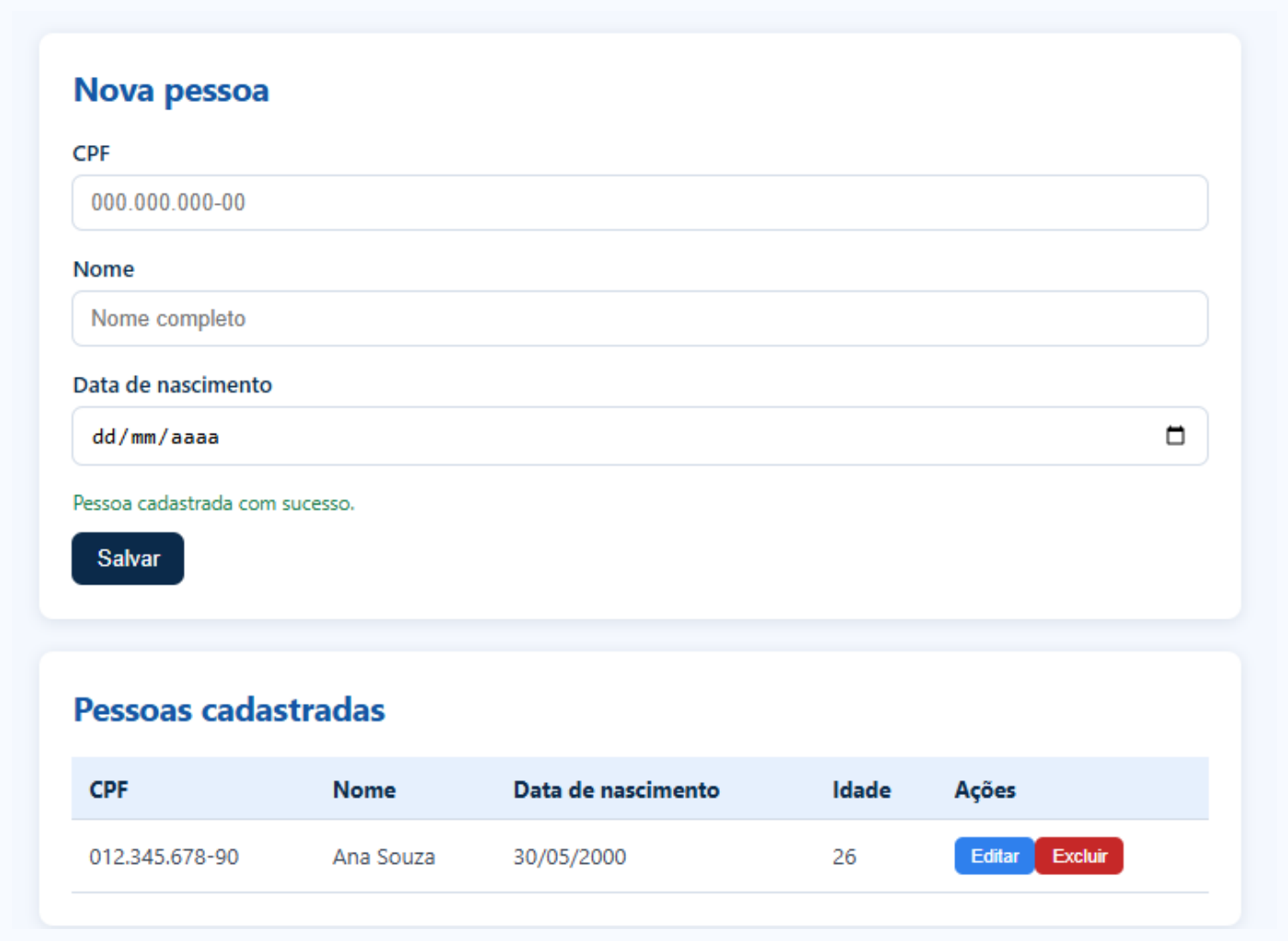
A página carregou no navegador com o formulário e a lista inicial vazia.



ETAPA 7 - INTEGRAÇÃO

Cadastro e idade calculada

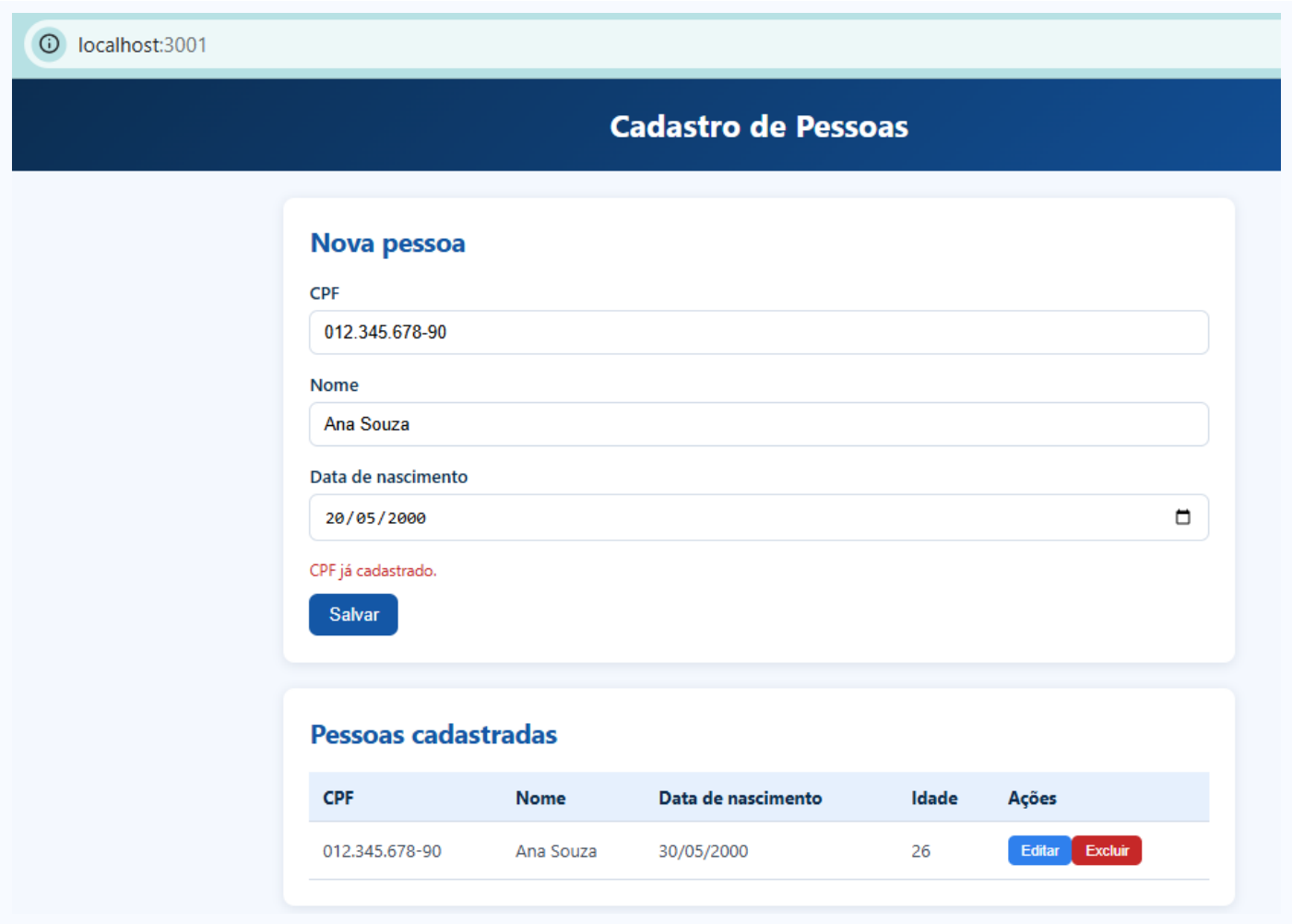
Ana Souza foi cadastrada com dados fictícios e idade de 26 anos.



ETAPA 7 - INTEGRAÇÃO

CPF duplicado recusado

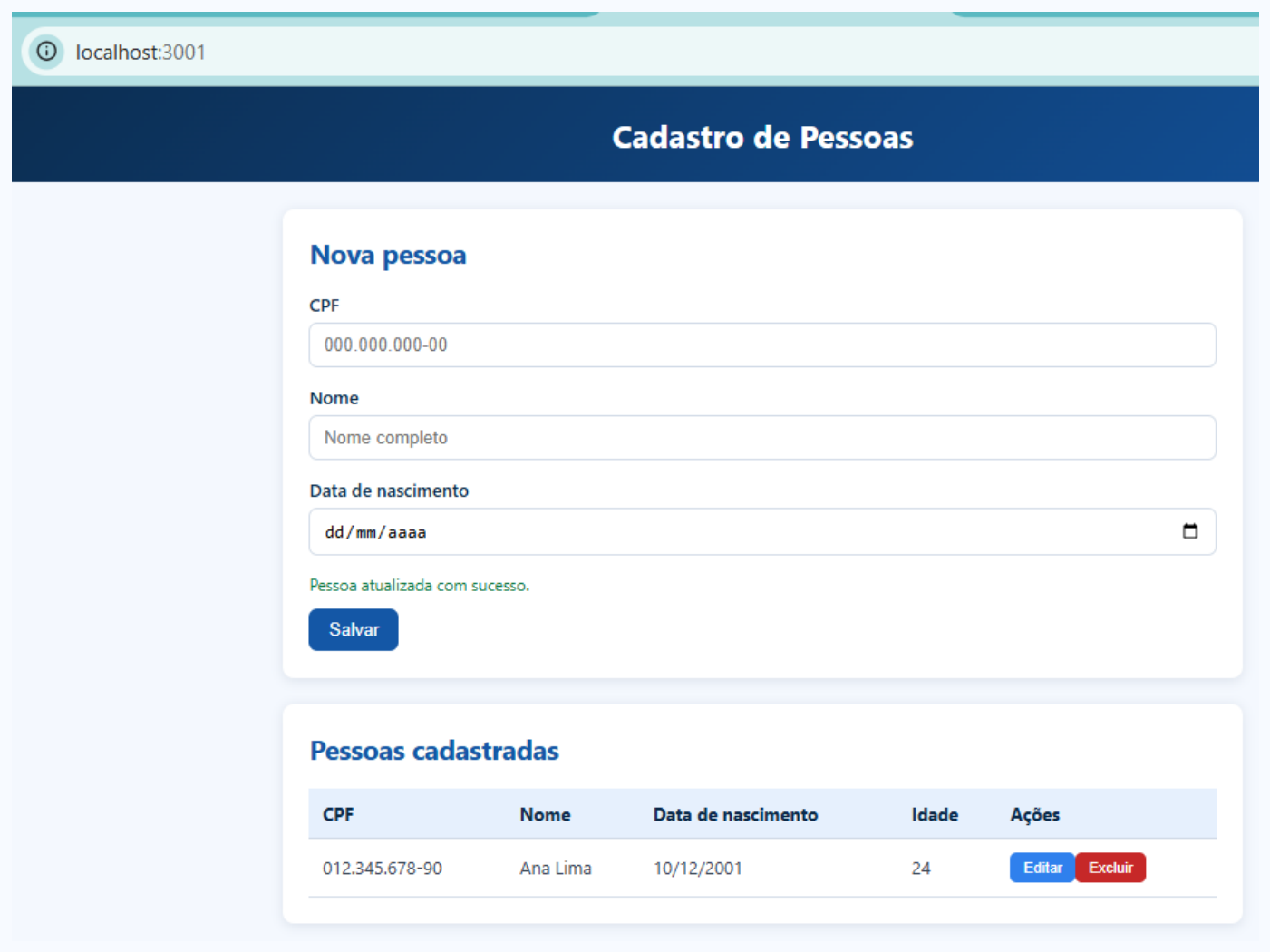
A tentativa repetida exibiu o erro e preservou um único cadastro.



ETAPA 7 - INTEGRAÇÃO

Edição do cadastro

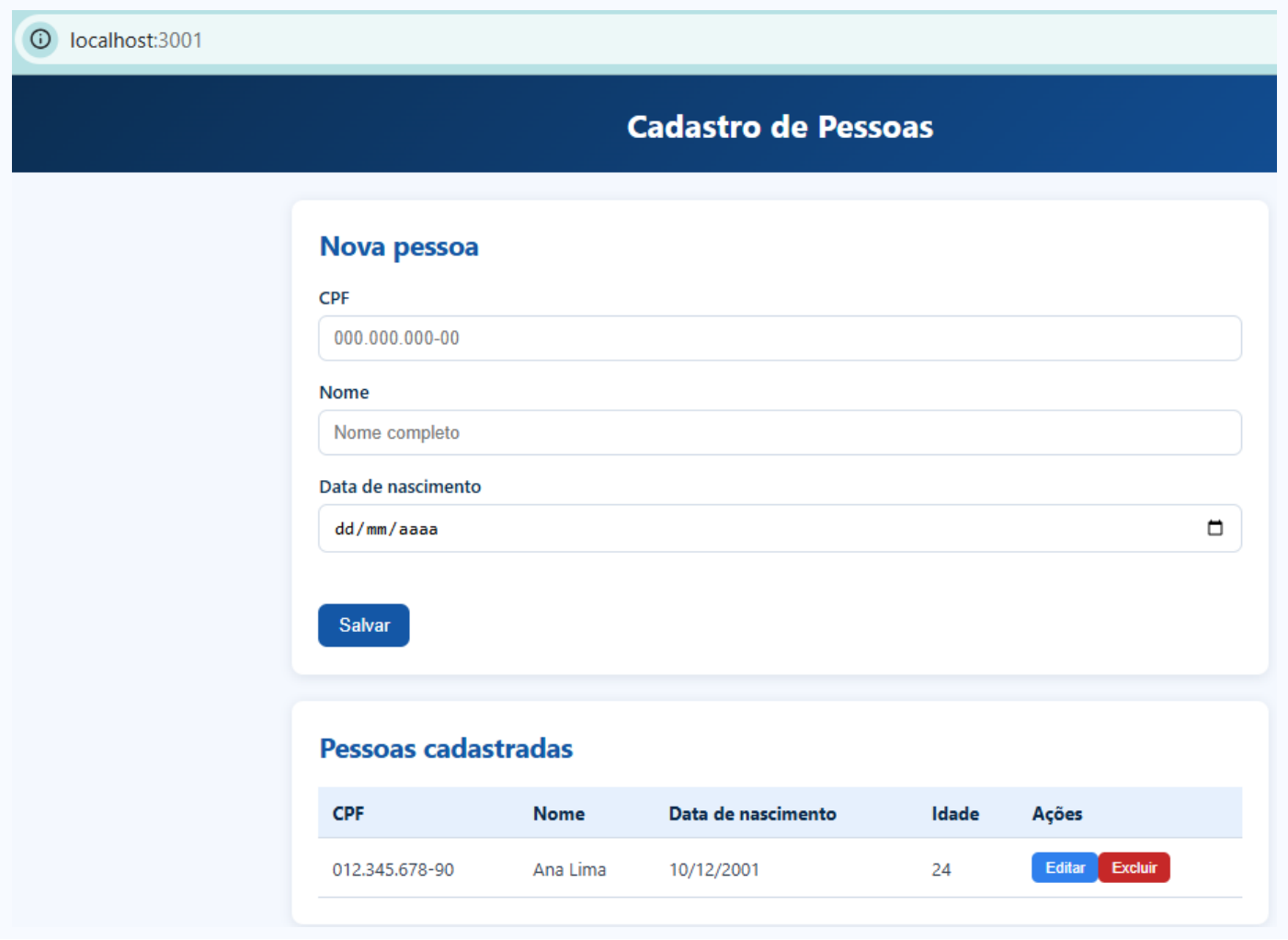
Nome e nascimento foram atualizados; o CPF permaneceu e a idade passou a 24.



ETAPA 7 - INTEGRAÇÃO

Persistência após recarregar

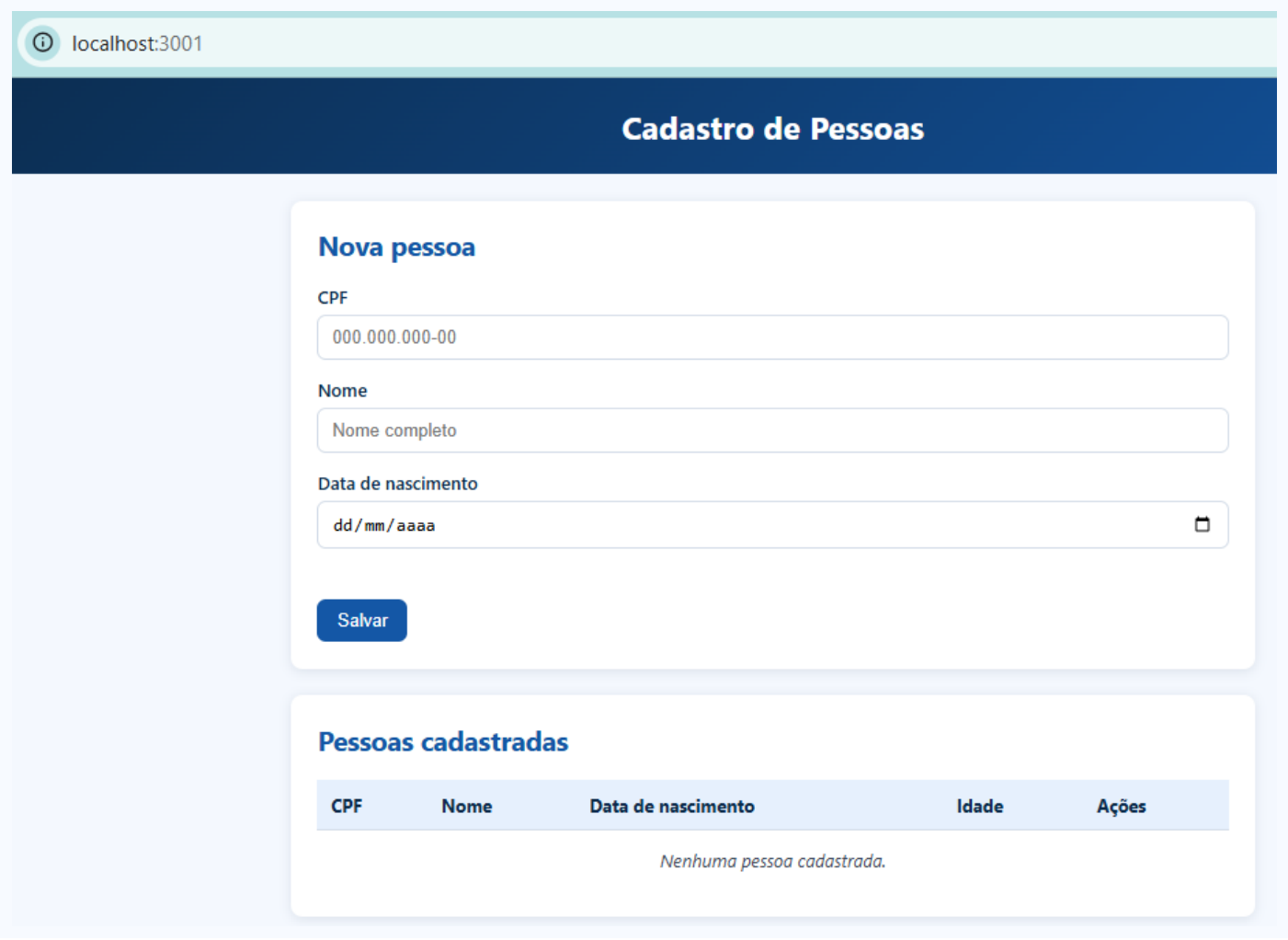
Após F5, o cadastro editado permaneceu na lista.



ETAPA 7 - INTEGRAÇÃO

Exclusão do cadastro

Após excluir, a interface voltou a mostrar a lista vazia.



ETAPA 7 - INTEGRAÇÃO

Testes completos

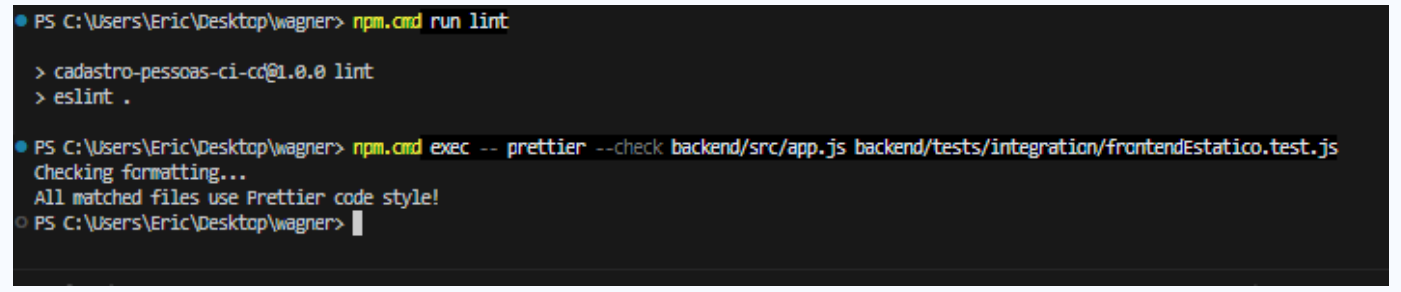
Passaram 46 testes do backend e 24 testes do frontend.



ETAPA 7 - INTEGRAÇÃO

Qualidade deste ciclo

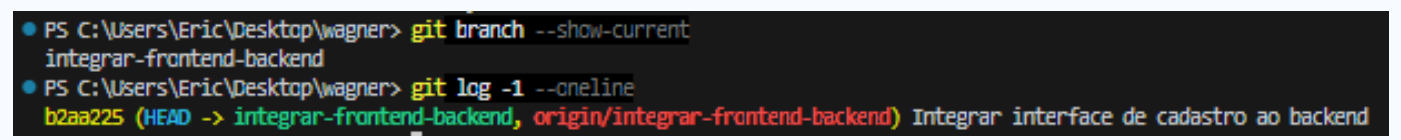
ESLint e Prettier dos arquivos da integração passaram; a checagem geral segue pendente.



ETAPA 7 - INTEGRAÇÃO

Commit de integração

A branch integrar-frontend-backend registra o commit de união do código.



ETAPA 7 - INTEGRAÇÃO

Pull request aberto

O PR #8 compara integrar-frontend-backend com backend-crud-api, sem merge.



Integrar interface de cadastro ao backend #8

Open Ericfranca90 wants to merge 18 commits into `backend-crud-api` from `integrar-frontend-backend`

Conversation 0 · Commits 18 · Checks 0 · Files changed 14

Ericfranca90 commented há 5 minutos

Resumo

- Integra a interface existente da branch `frontend` ao backend, preservando os commits da Heloisa no histórico.
- Disponibiliza a página, CSS e JavaScript pelo Express, mantendo `/pessoas` como API JSON.
- Adiciona testes de integração para a página, os arquivos estáticos e a API.

Verificações

- Backend: 46 testes passaram (12 suítes).
- Frontend: 24 testes passaram (3 suítes).
- ESLint passou; Prettier passou nos dois arquivos novos/alterados do backend.
- Fluxo manual com JSON de teste isolado: cadastro, CPF duplicado, edição, persistência após recarregar e exclusão funcionaram.

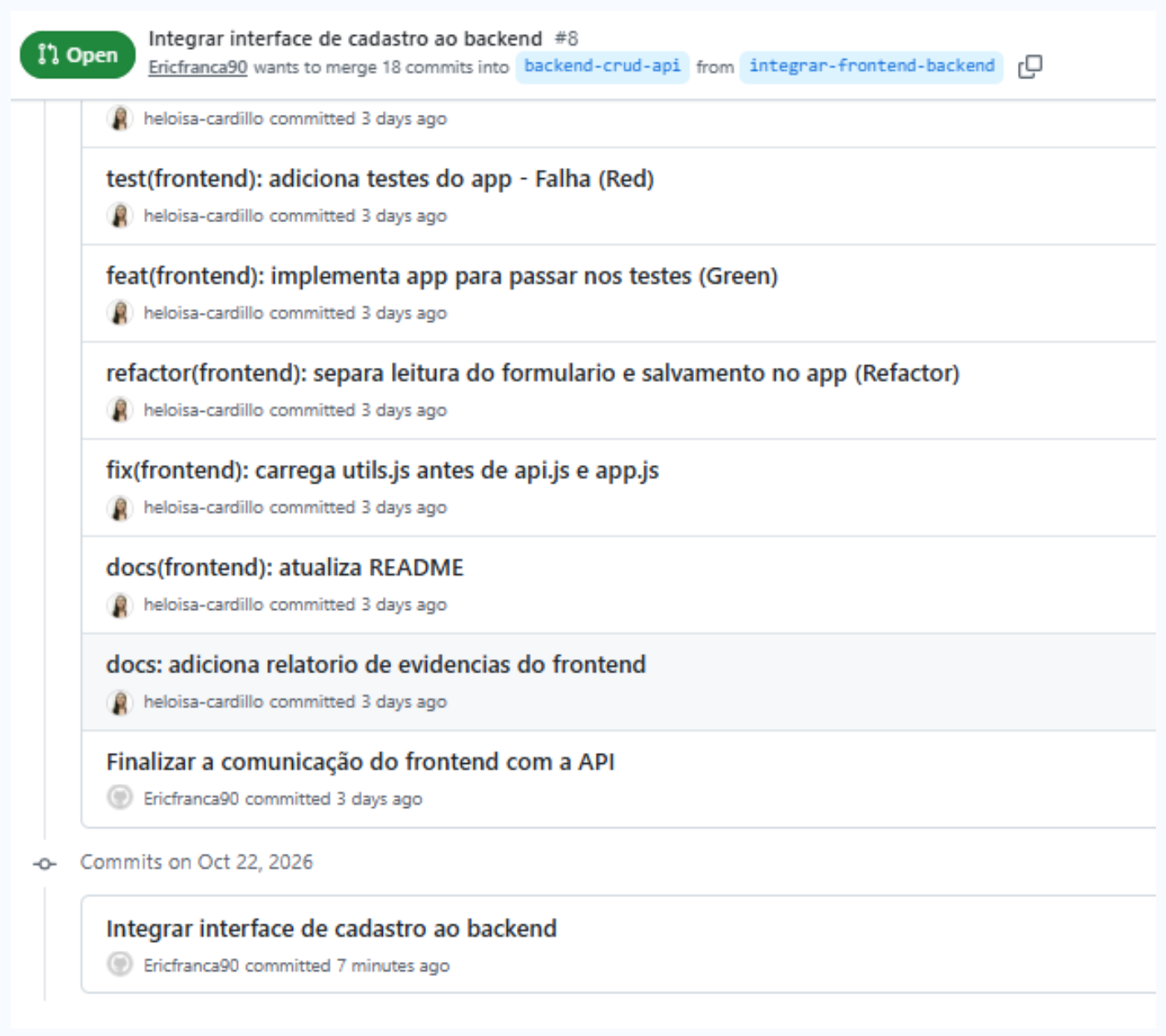
Dependências e pendências

- Base: `backend-crud-api`; este PR depende do PR Implementar CRUD e API com armazenamento JSON #7.
- Incorpora os commits da branch do frontend do PR feat(frontend): implementa api, app e testes do frontend com TDD #3, que permanece aberto. Rever a ordem de integração antes de qualquer merge para evitar duplicação.
- A checagem de formatação do projeto inteiro ainda apresenta 15 avisos; não foi declarada aprovada nem foi feita normalização geral.
- Prints e relatório acumulado serão incluídos em commit documental separado.

ETAPA 7 - INTEGRAÇÃO

Autoria preservada

O histórico do PR mostra os commits da Heloisa e o commit de integração.



ETAPA 8

Formatação e finais de linha

Resumo da manutenção

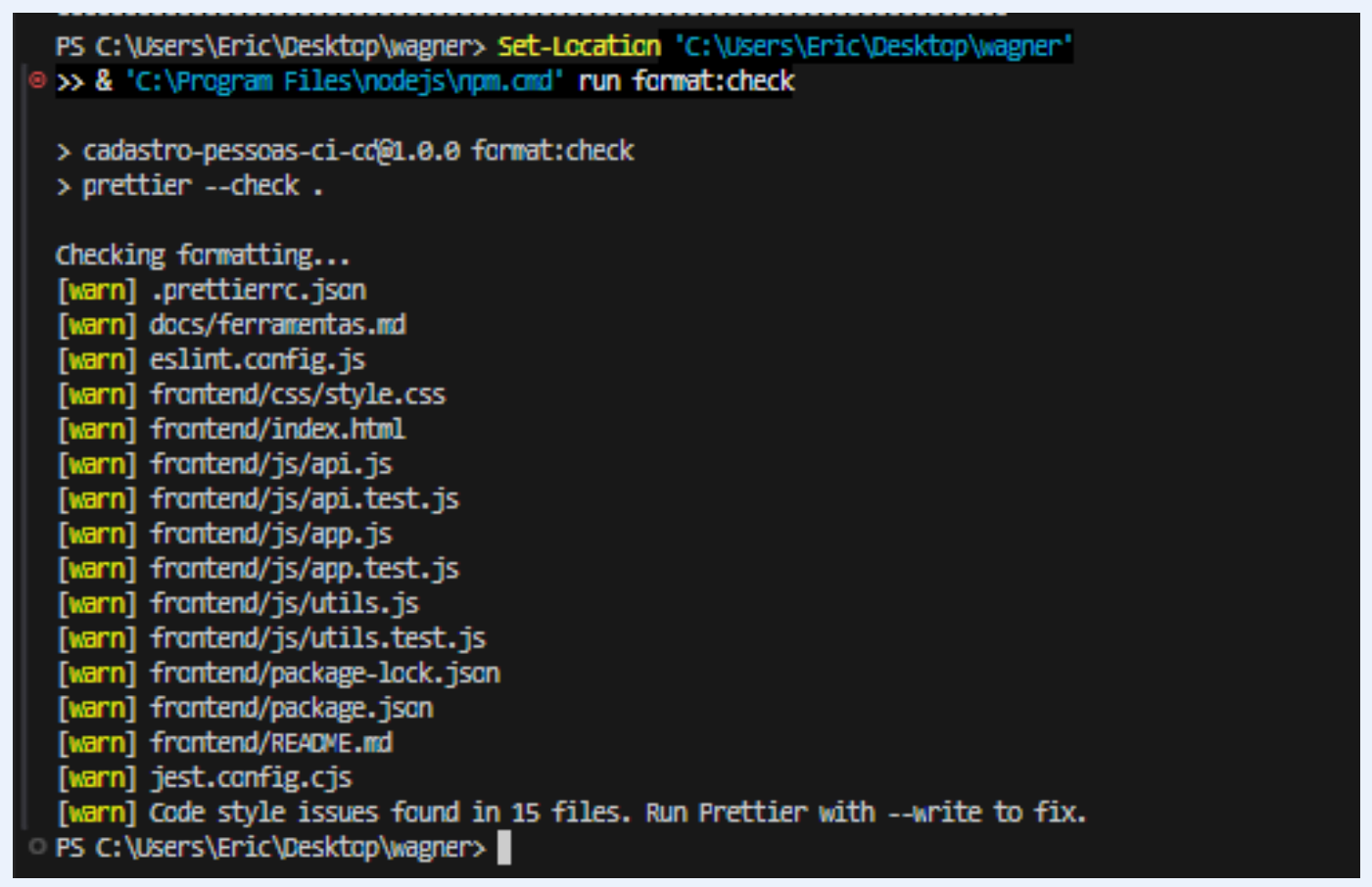
- A checagem geral apontava 15 arquivos. Em sete, a única diferença era LF/CRLF; oito também precisavam de formatação.
- Foi adicionada uma política versionada de LF para texto, preservando imagens e PDFs como binários, sem mudar a configuração global do Git.
- A formatação foi aplicada aos arquivos necessários. A comparação de sintaxe JavaScript e da estrutura HTML não identificou mudança funcional.
- Passaram a checagem geral do Prettier, o ESLint, 46 testes do backend e 24 testes do frontend.
- Em checkout isolado do commit, npm ci reproduziu as dependências pelo lockfile e a checagem geral também passou; os lockfiles não mudaram.
- O commit 66da991 foi publicado no PR #9, aberto contra integrar-frontend-backend e dependente do PR #8. Nenhum PR foi mesclado.

As páginas seguintes mostram capturas reais desta etapa.

ETAPA 8

Diagnóstico: checagem geral

Antes da manutenção, o Prettier apontava 15 arquivos. Esta é uma checagem de formatação, não um RED de teste funcional.



ETAPA 8

Diagnóstico: LF, CRLF e estilo

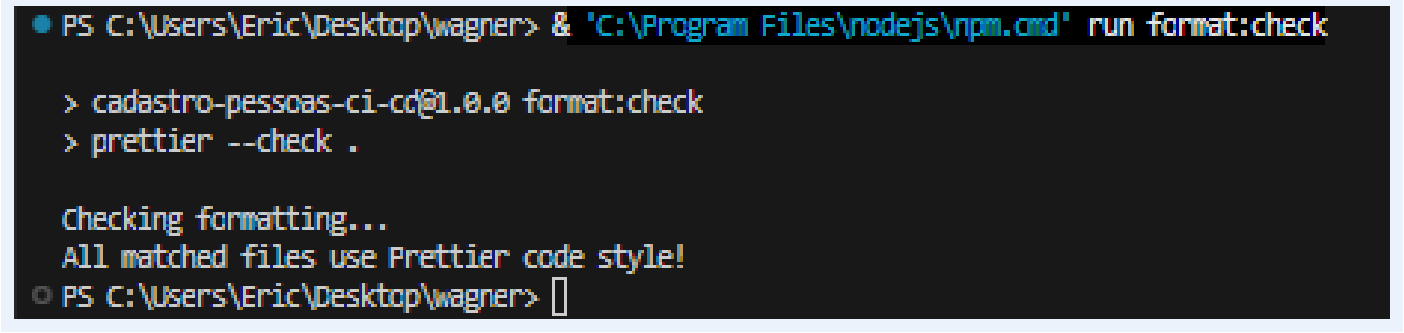
O Git do sistema usava core.autocrlf=true. O índice estava em LF e os arquivos locais em CRLF; ao permitir CRLF só no diagnóstico, oito avisos persistiram.



ETAPA 8

Formatação geral aprovada

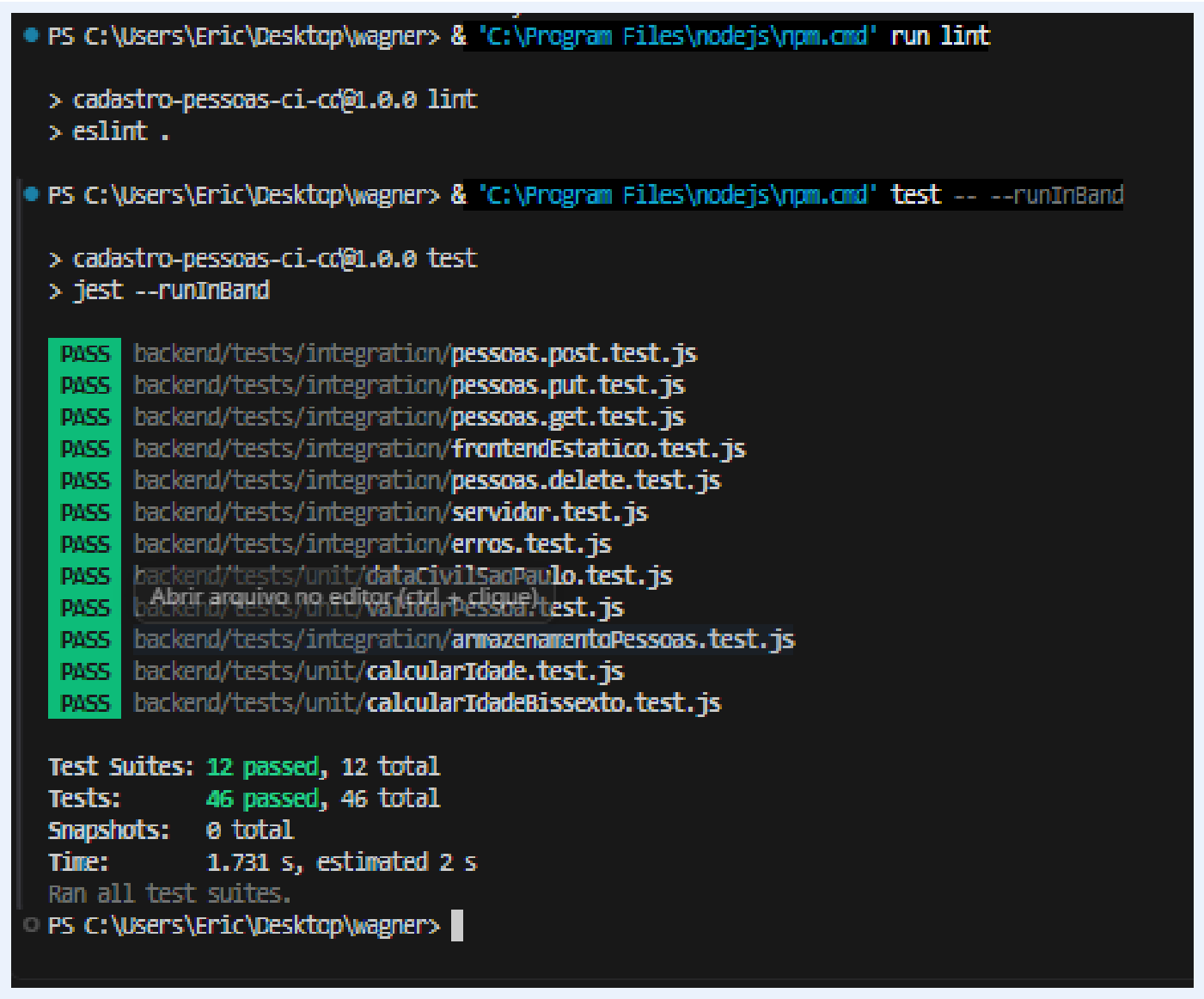
Após a padronização, npm run format:check passou em todo o escopo do projeto definido pelo Prettier.



ETAPA 8

Análise estática e backend

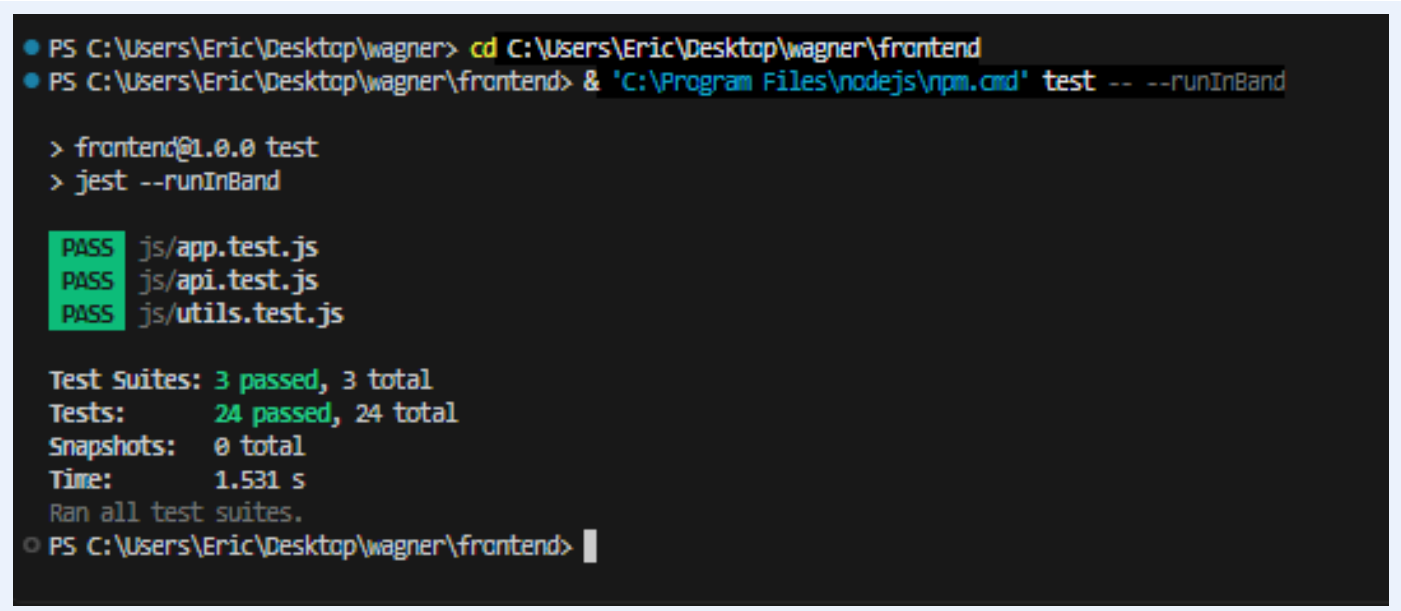
O ESLint terminou sem erros. As 12 suítes e os 46 testes do backend passaram.



ETAPA 8

Testes do frontend

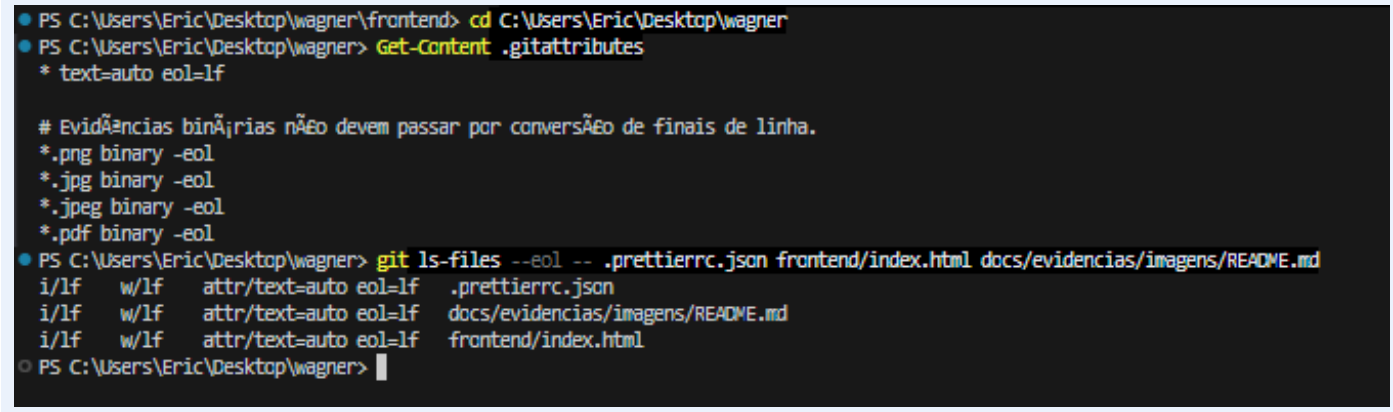
As três suítes e os 24 testes do frontend passaram após a formatação.



ETAPA 8

Política de finais de linha

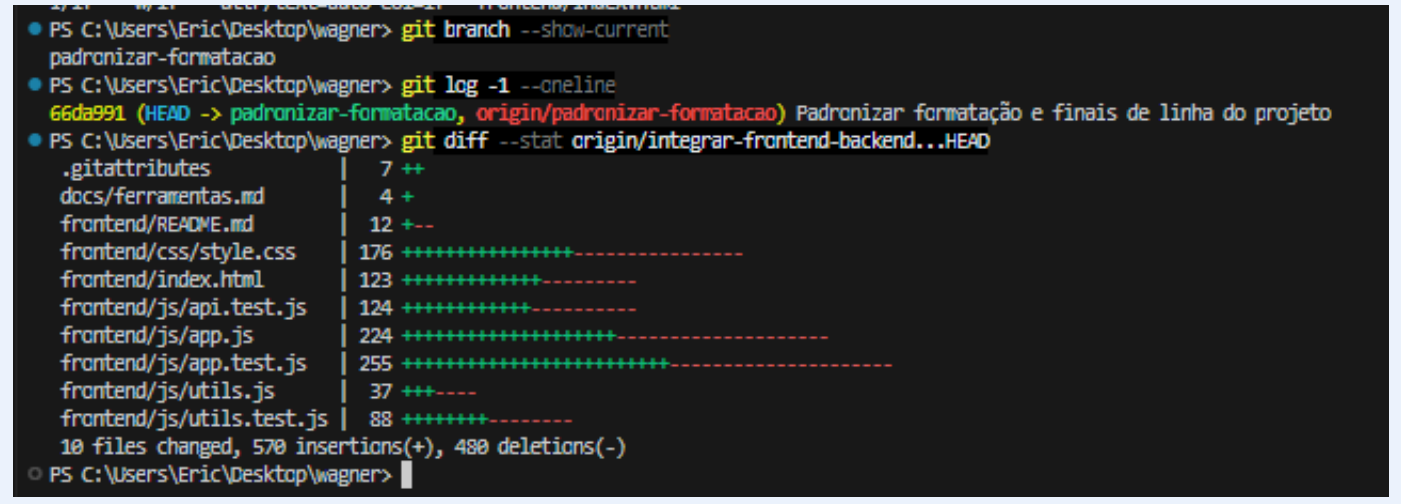
O .gitattributes fixa LF para texto e exclui imagens e PDFs da conversão. Arquivos de texto foram conferidos com git ls-files --eol.



ETAPA 8

Commit da manutenção

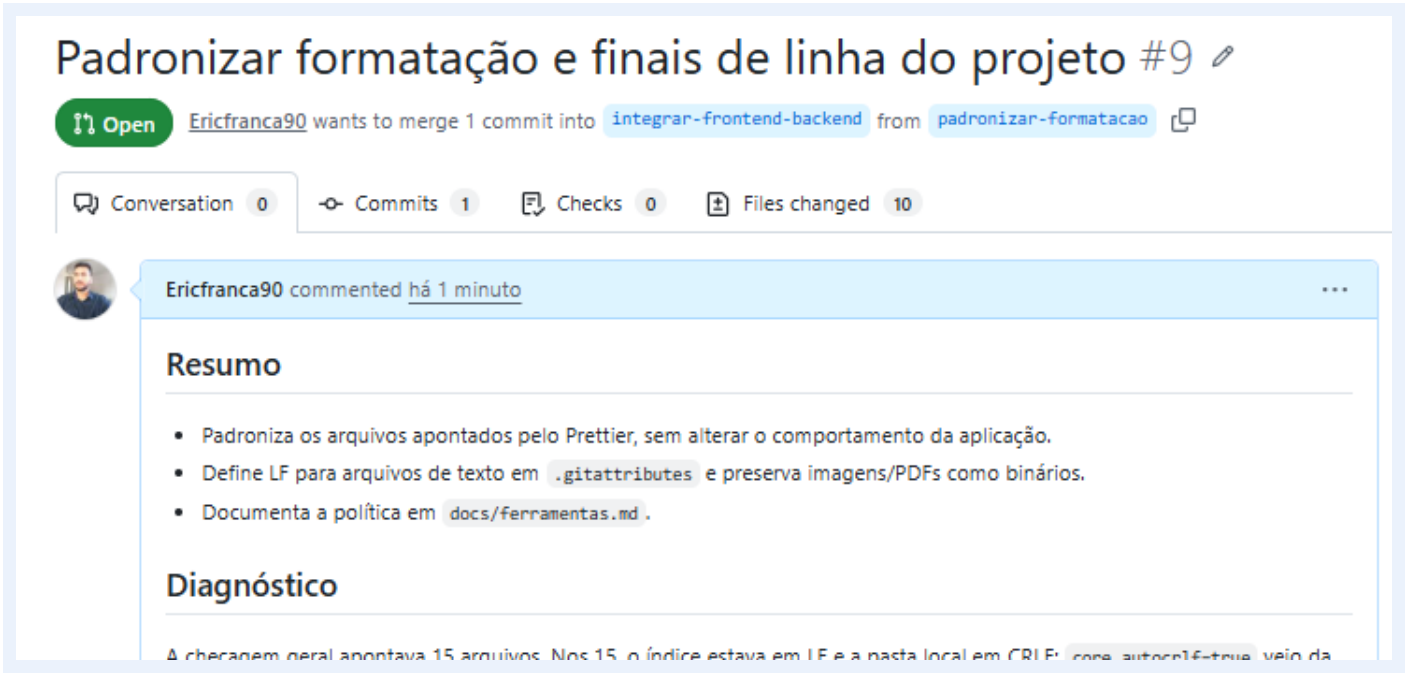
A branch padronizar-formatacao contém um commit específico; o resumo mostra somente os dez arquivos da manutenção.


PS C:\Users\Eric\Desktop\wagner> git branch --show-current
padronizar-formatacao
PS C:\Users\Eric\Desktop\wagner> git log -1 --oneline
66da991 (HEAD -> padronizar-formatacao, origin/padronizar-formatacao) Padronizar formatação e finais de linha do projeto
PS C:\Users\Eric\Desktop\wagner> git diff --stat origin/integrar-frontend-backend...HEAD
.gitattributes | 7 ++
docs/ferramentas.md | 4 +
frontend/README.md | 12 +--
frontend/css/style.css | 176 +++++++++++++++----------------
frontend/index.html | 123 +++++++++++++---------
frontend/js/api.test.js | 124 +++++++++++++---------
frontend/js/app.js | 224 +++++++++++++++

ETAPA 8

Pull request aberto

O PR #9 permanece aberto, de padronizar-formatacao para integrar-frontend-backend. Ele depende do PR #8 e não foi mesclado.



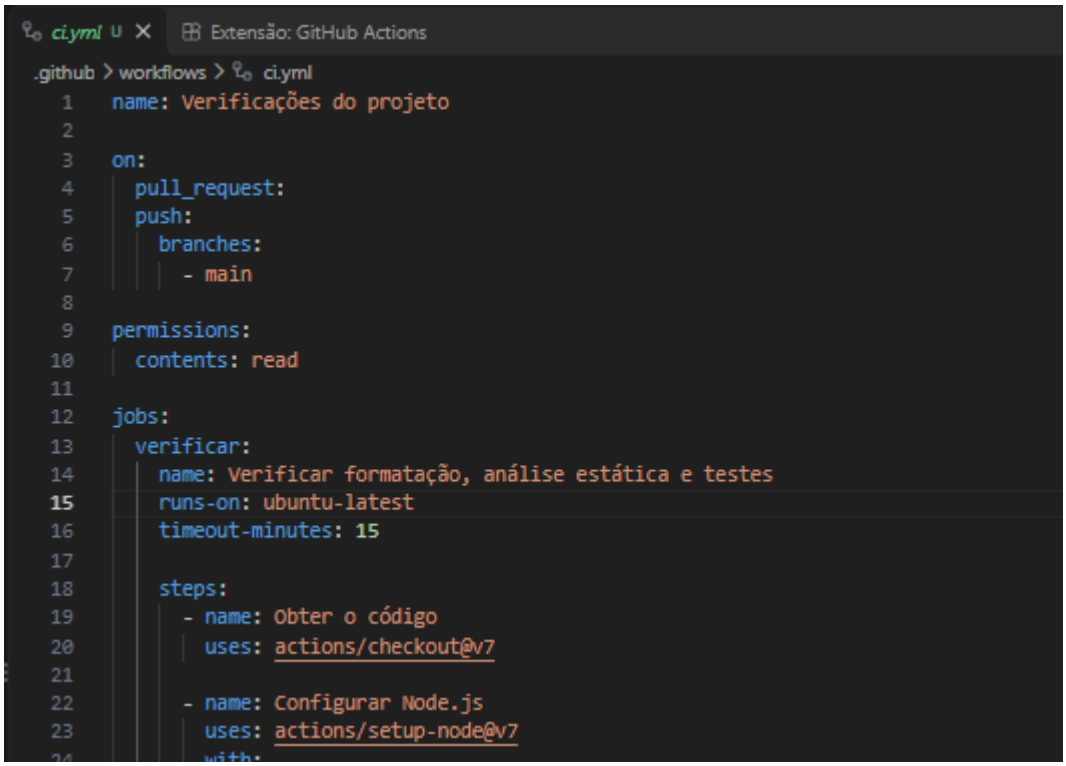
ETAPA 9

Integração contínua

O PR #10 acrescenta um workflow que instala dependências pelos lockfiles e verifica formatação, análise estática e testes em Node.js 24. A primeira execução revelou entradas indiretas ausentes no lockfile para Linux; após o ajuste, a segunda execução passou. Nenhum PR foi mesclado.

Gatilhos e configuração

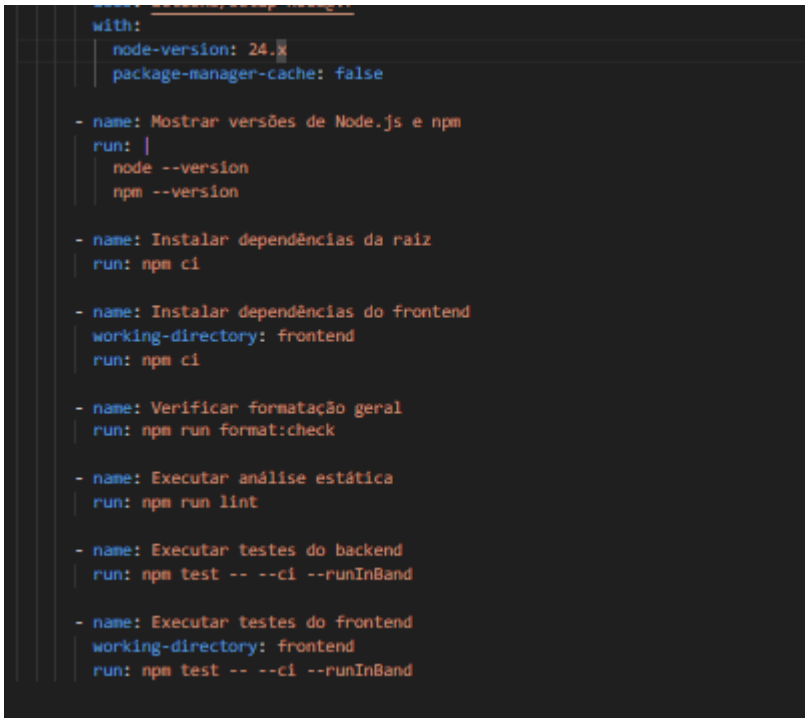
O workflow roda em pull requests sem limitar a base e está preparado para rodar em pushes para `main` após a integração futura. O job usa Ubuntu e permissão de leitura.



ETAPA 9

Passos definidos antes da publicação

Os passos separam instalação, Prettier, ESLint e testes do backend e do frontend. A branch foi criada a partir do commit publicado da etapa anterior.



```
      with:
        node-version: 24.x
        package-manager-cache: false

    - name: Mostrar versões de Node.js e npm
      run: |
        node --version
        npm --version

    - name: Instalar dependências da raiz
      run: npm ci

    - name: Instalar dependências do frontend
      working-directory: frontend
      run: npm ci

    - name: Verificar formatação geral
      run: npm run format:check

    - name: Executar análise estática
      run: npm run lint

    - name: Executar testes do backend
      run: npm test -- --ci --runInBand

    - name: Executar testes do frontend
      working-directory: frontend
      run: npm test -- --ci --runInBand
```

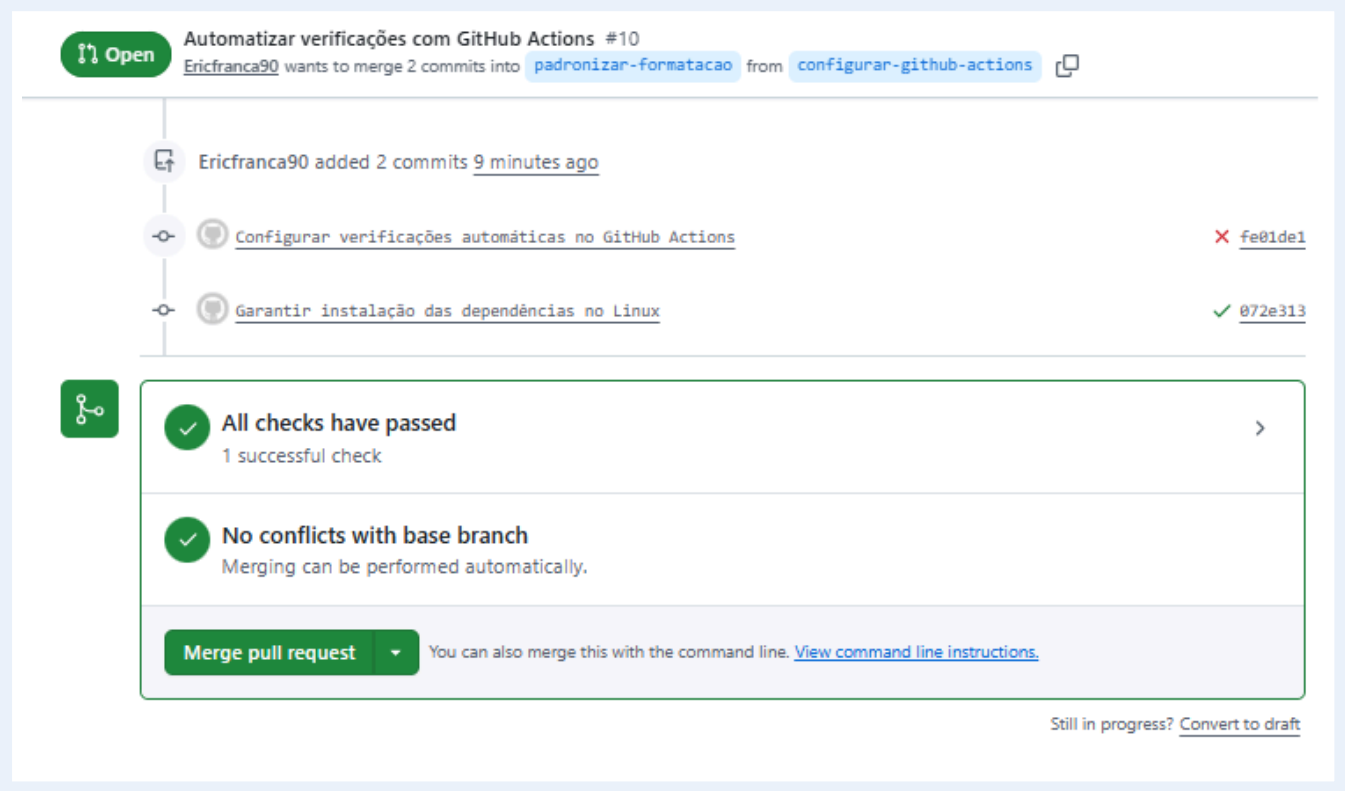


```
PS C:\Users\Eric\Desktop\wagner> git branch --show-current
configurar-github-actions
PS C:\Users\Eric\Desktop\wagner> git log -1 --oneline
74f398b (HEAD -> configurar-github-actions, origin/padronizar-formatacao, padronizar-formatacao) Registrar evidências da padronização de formatação
PS C:\Users\Eric\Desktop\wagner> git status --short --untracked-files=all -- .github/workflows/ci.yml package-lock.json frontend/package-lock.json
?? .github/workflows/ci.yml
PS C:\Users\Eric\Desktop\wagner>
```

ETAPA 9

Pull request e revisão

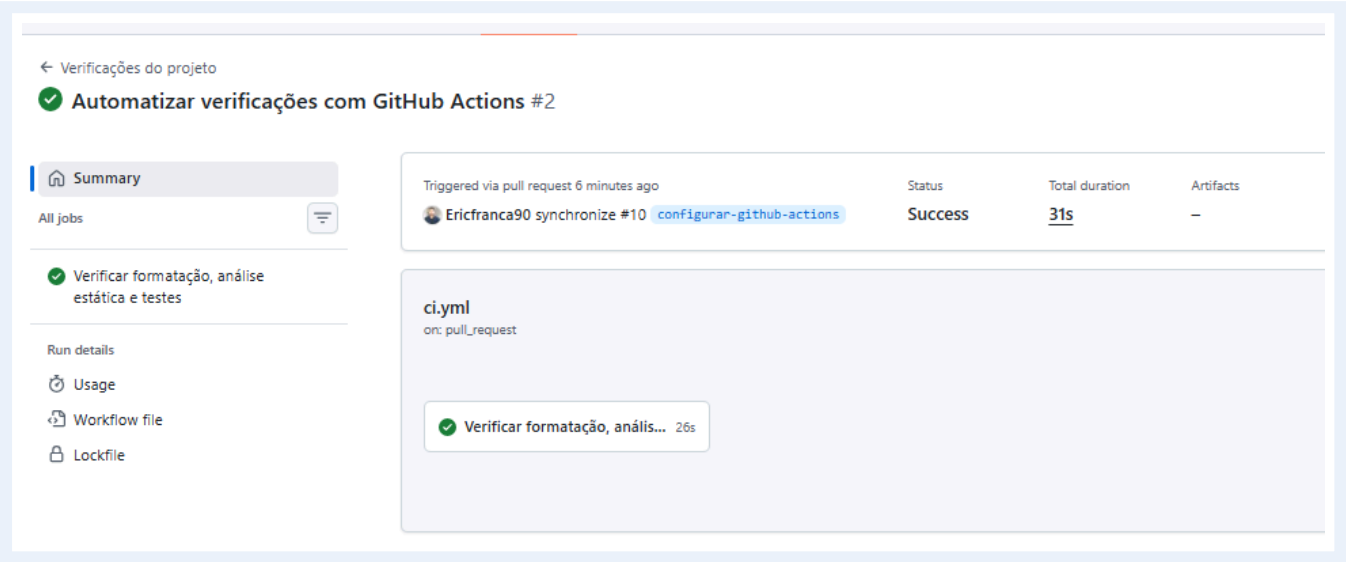
O PR #10 propõe a automação sobre `padronizar-formatacao`, da qual depende. O check verde corresponde à execução após o ajuste do lockfile. O botão de merge está visível, mas não foi acionado.



ETAPA 9

Execução remota aprovada

A execução #2 foi disparada pelo evento `pull_request` e concluiu com sucesso. O checkout pode testar um merge simulado pelo GitHub; isso não mescla o PR de verdade.



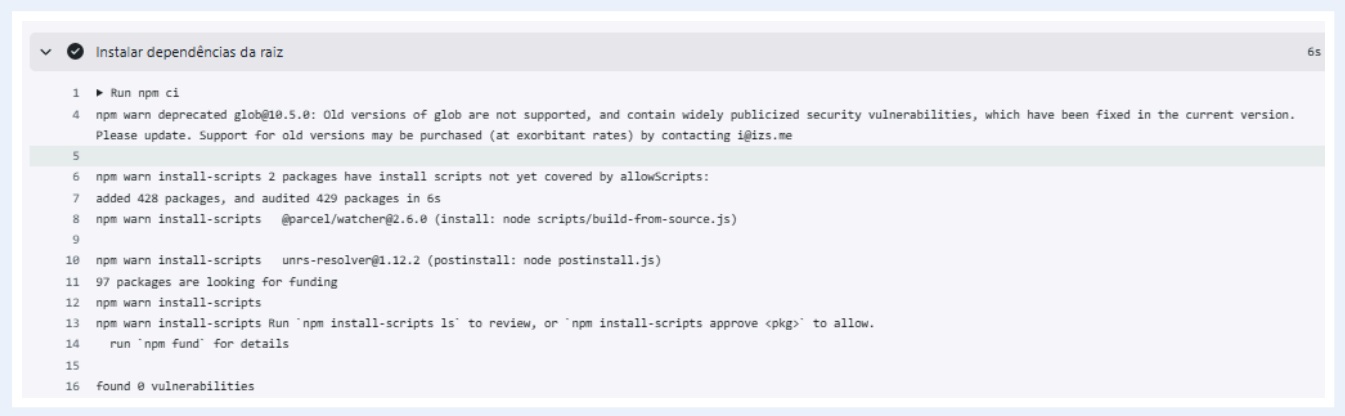
> Ambiente registrado no log: Node.js 24.21.0 e npm 11.19.0. Execução: github.com/Ericfranca90/cadastro-pessoas-ci-cd/actions/runs/39877456631.

ETAPA 9

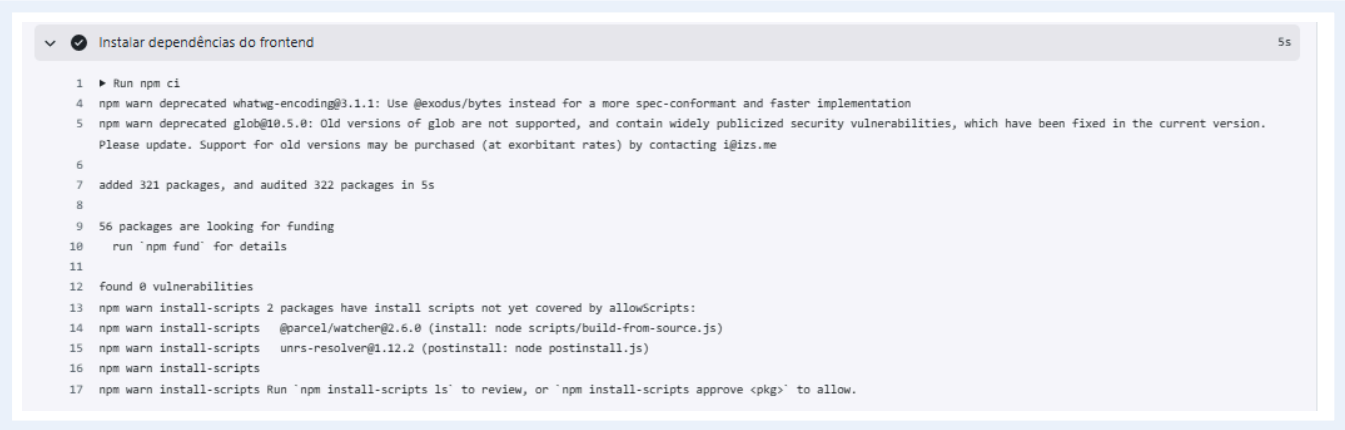
Instalações reproduzíveis

O GitHub executou `npm ci` na raiz e no frontend com êxito. A primeira execução havia parado na raiz porque faltavam duas entradas indiretas no lockfile; o commit seguinte completou essas entradas sem alterar as versões declaradas.

Dependências da raiz



Dependências do frontend



ETAPA 9

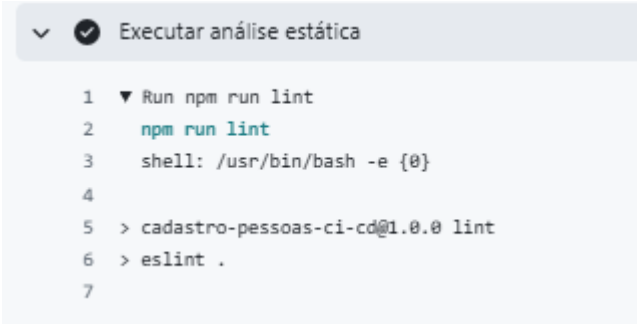
Formatação e análise estática

O Prettier conferiu a formatação geral sem apontar arquivos. O ESLint terminou sem erros. Esses passos são verificações, não comandos que reformatam ou modificam o código.

Prettier



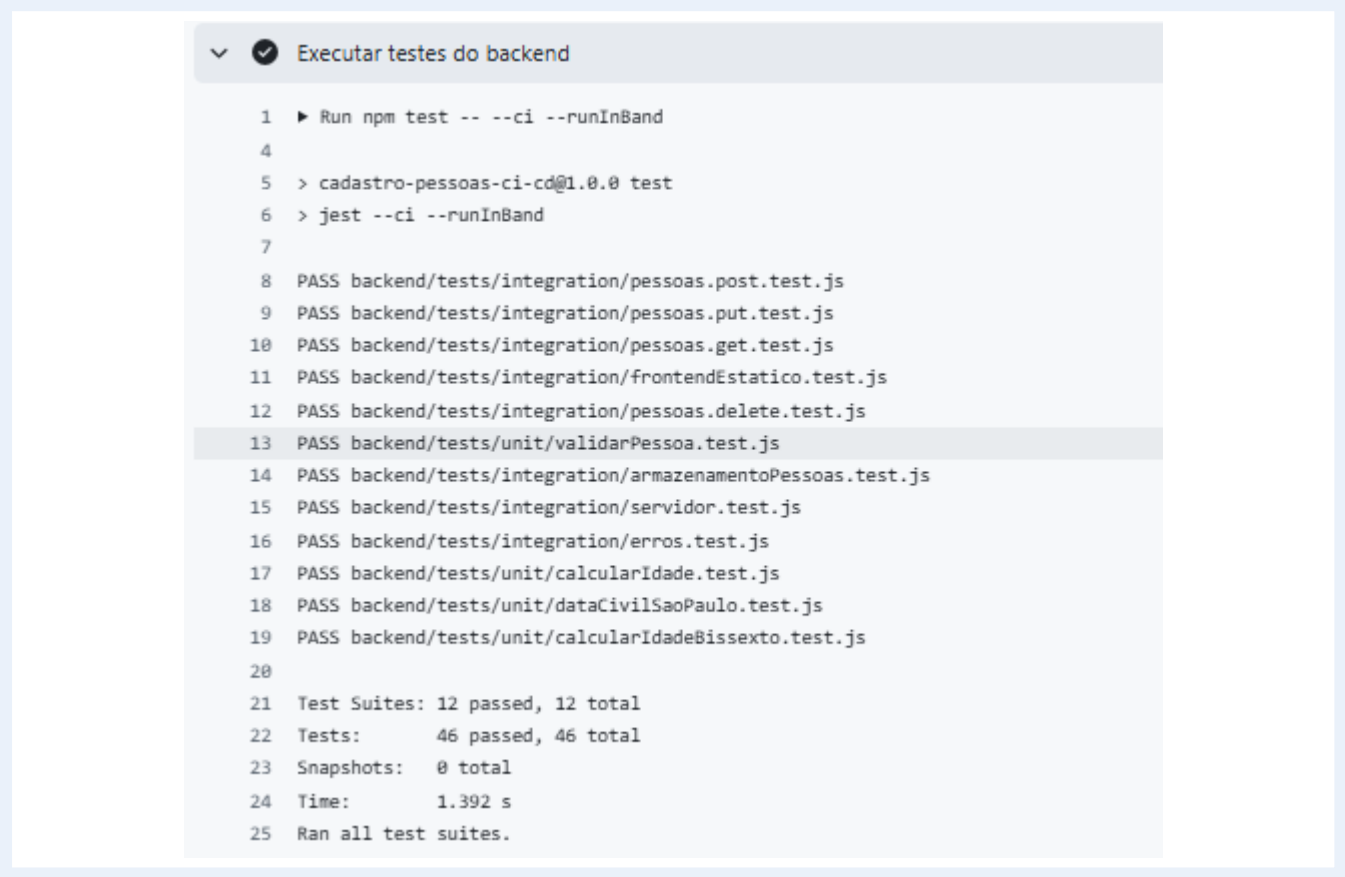
ESLint



ETAPA 9

Testes do backend

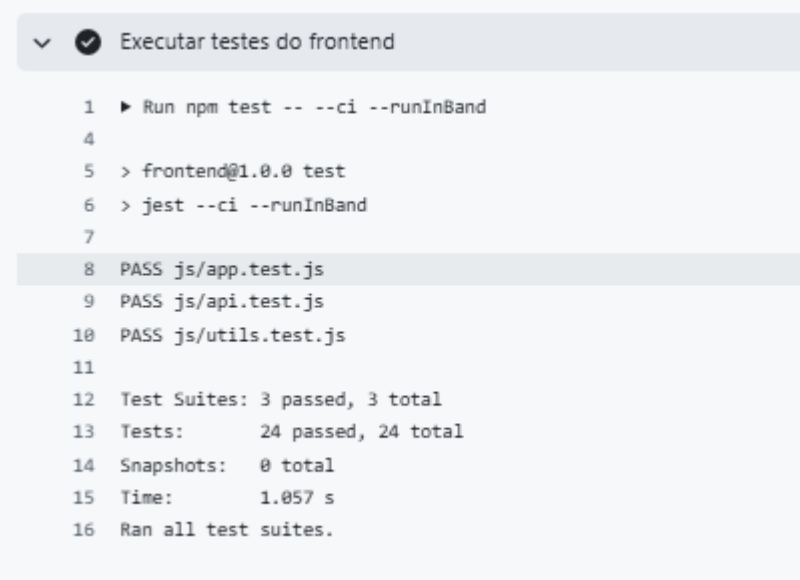
O Jest da raiz executou 12 suítes e 46 testes do backend, todos aprovados no GitHub Actions.



ETAPA 9

Testes do frontend

Na pasta `frontend`, o Jest executou 3 suítes e 24 testes da interface, todos aprovados na mesma execução remota.

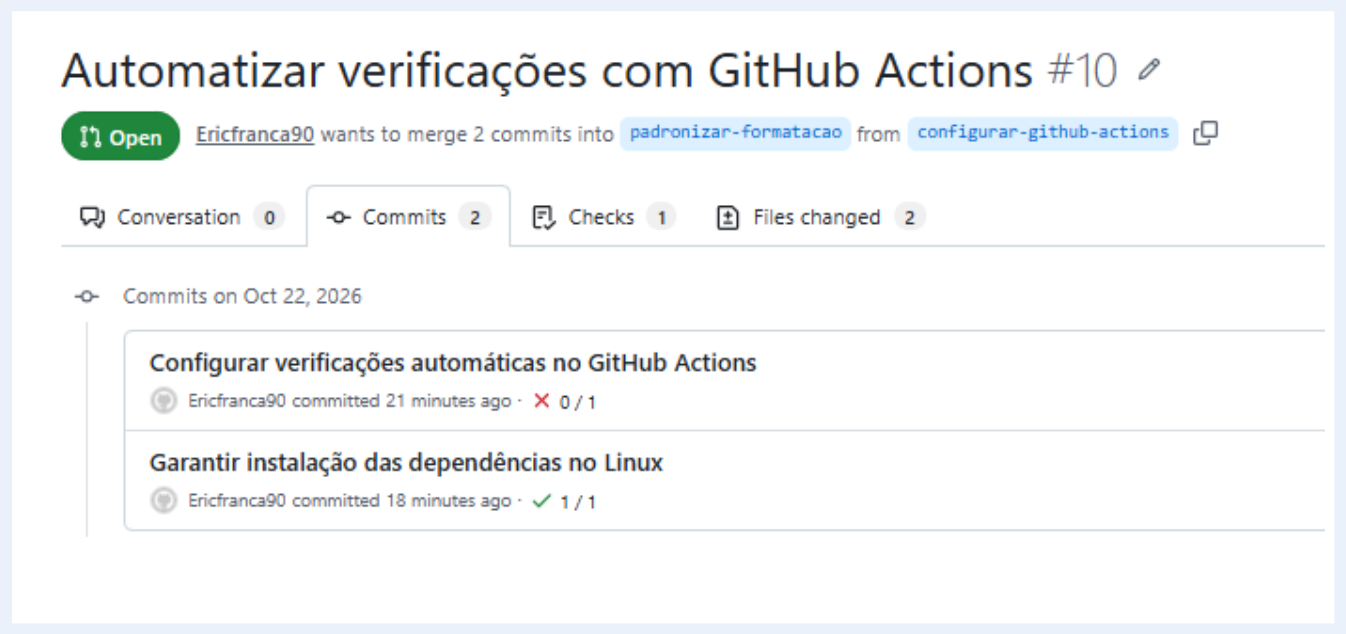


ETAPA 9

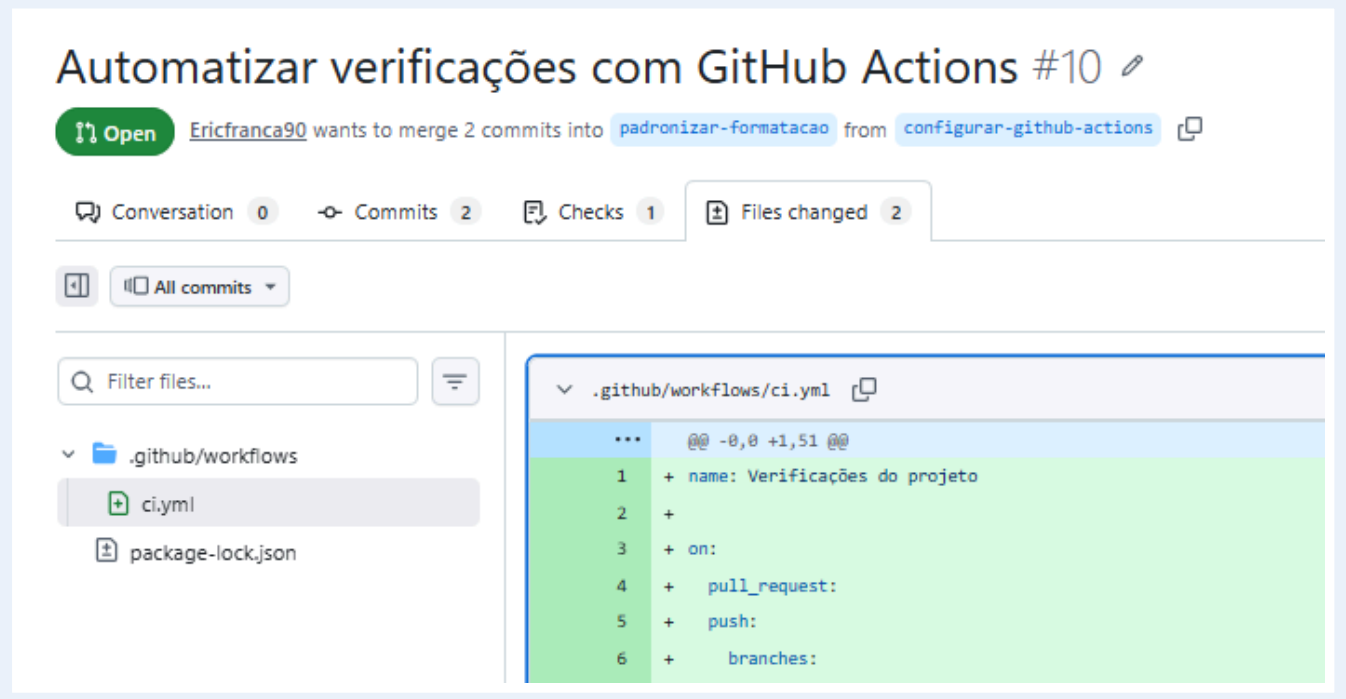
Histórico e escopo do PR

O primeiro commit criou o workflow; o segundo completou o lockfile após a falha real no Linux. Antes do commit documental, a comparação continha somente esses dois arquivos.

Commits



Arquivos alterados

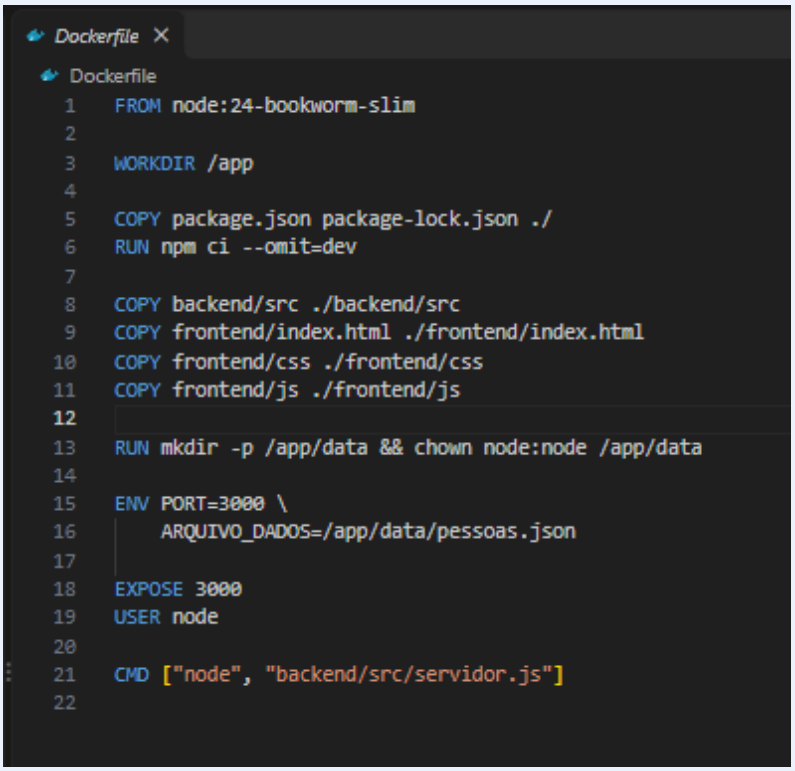


ETAPA 10

Arquivos da imagem

O Dockerfile instala somente dependências de execução e inicia o servidor como usuário sem privilégios. O .dockerignore exclui histórico, testes, dados locais, segredos e evidências do contexto.

86 - Dockerfile



```
Dockerfile
FROM node:24-bookworm-slim

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY backend/src ./backend/src
COPY frontend/index.html ./frontend/index.html
COPY frontend/css ./frontend/css
COPY frontend/js ./frontend/js

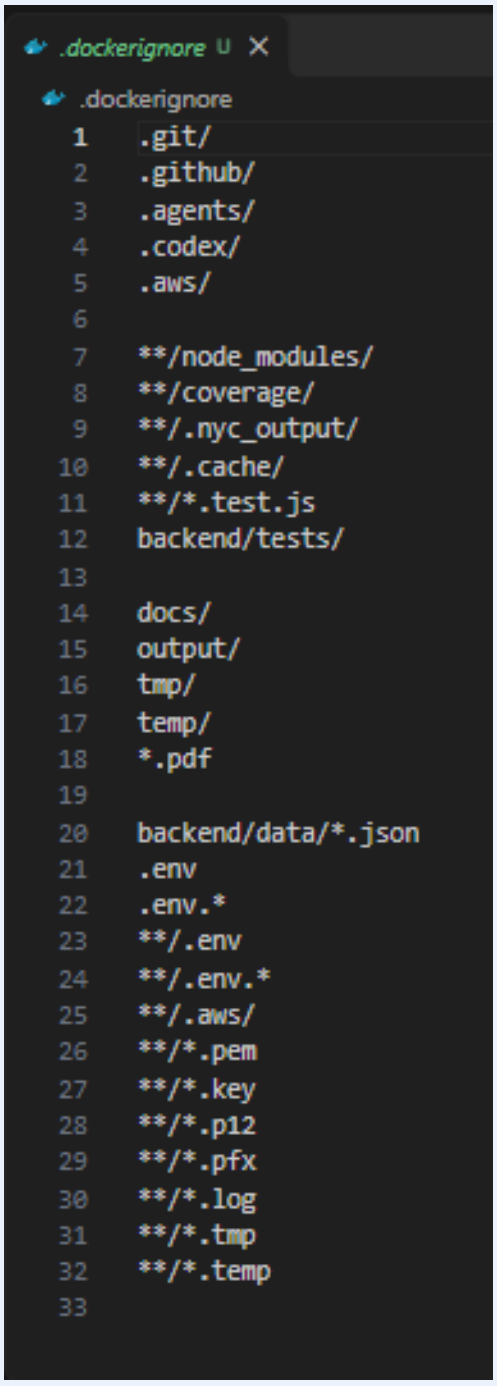
RUN mkdir -p /app/data && chown node:node /app/data

ENV PORT=3000 \
    ARQUIVO_DADOS=/app/data/pessoas.json

EXPOSE 3000
USER node

CMD ["node", "backend/src/servidor.js"]
```

87 - .dockerignore



```
.dockerignore
.git/
.github/
.agents/
.codex/
.aws/

**/node_modules/
**/coverage/
**/.nyc_output/
**/.cache/
**/*.test.js
backend/tests/

docs/
output/
tmp/
temp/
*.pdf

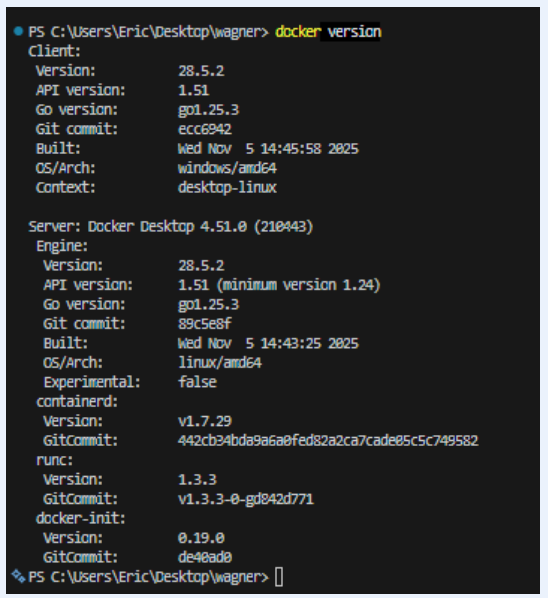
backend/data/*.json
.env
.env.*
**/.env
**/.env.*
**/.aws/
**/*.pem
**/*.key
**/*.p12
**/*.pfx
**/*.log
**/*.tmp
**/*.temp
```

ETAPA 10

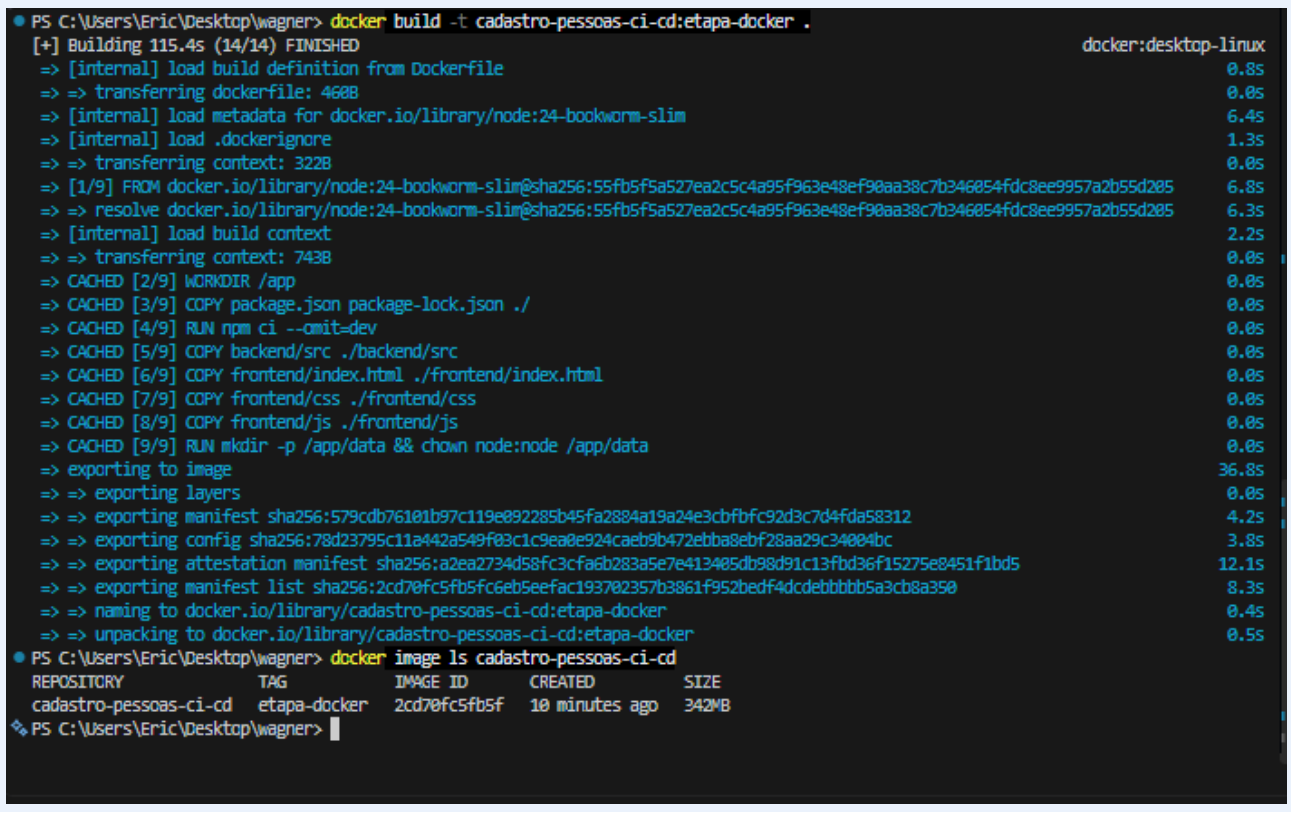
Docker disponível e imagem construída

O cliente e o mecanismo de contêineres Linux responderam. O build terminou e a imagem cadastro-pessoas-ci-cd:etapa-docker apareceu com 342 MB.

88 - Disponibilidade do Docker



89 - Build e identificação da imagem

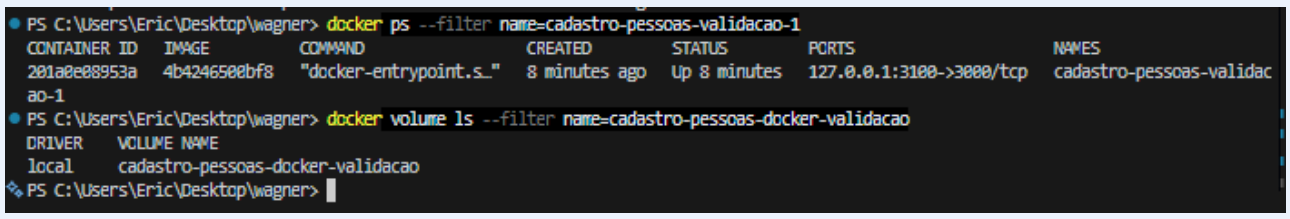


ETAPA 10

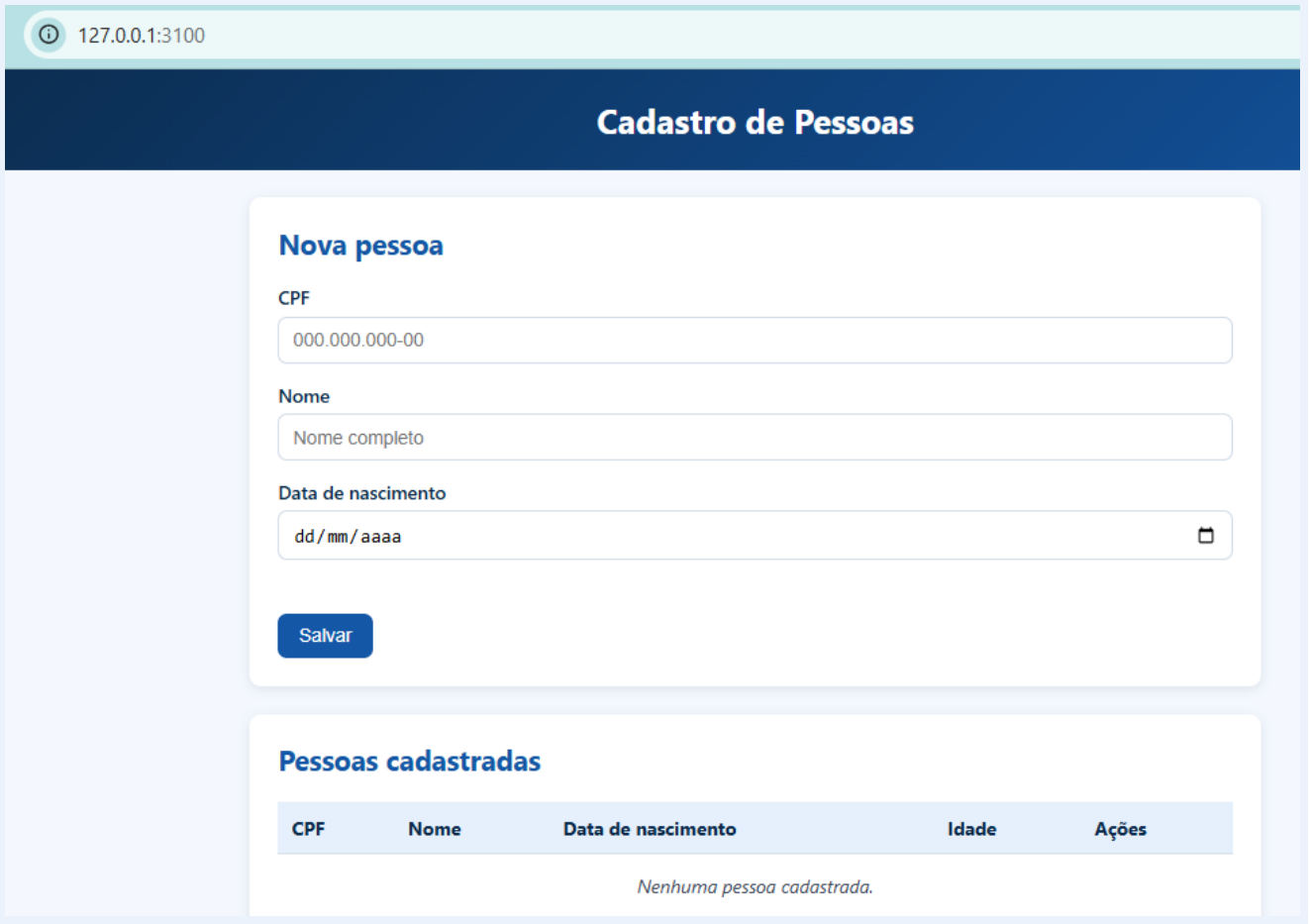
Contêiner, volume e tela inicial

O primeiro contêiner publicou a porta apenas em 127.0.0.1 e usou um volume exclusivo para o JSON. A interface abriu com a lista vazia.

90 - Contêiner e volume nomeado



91 - Aplicação inicial no navegador

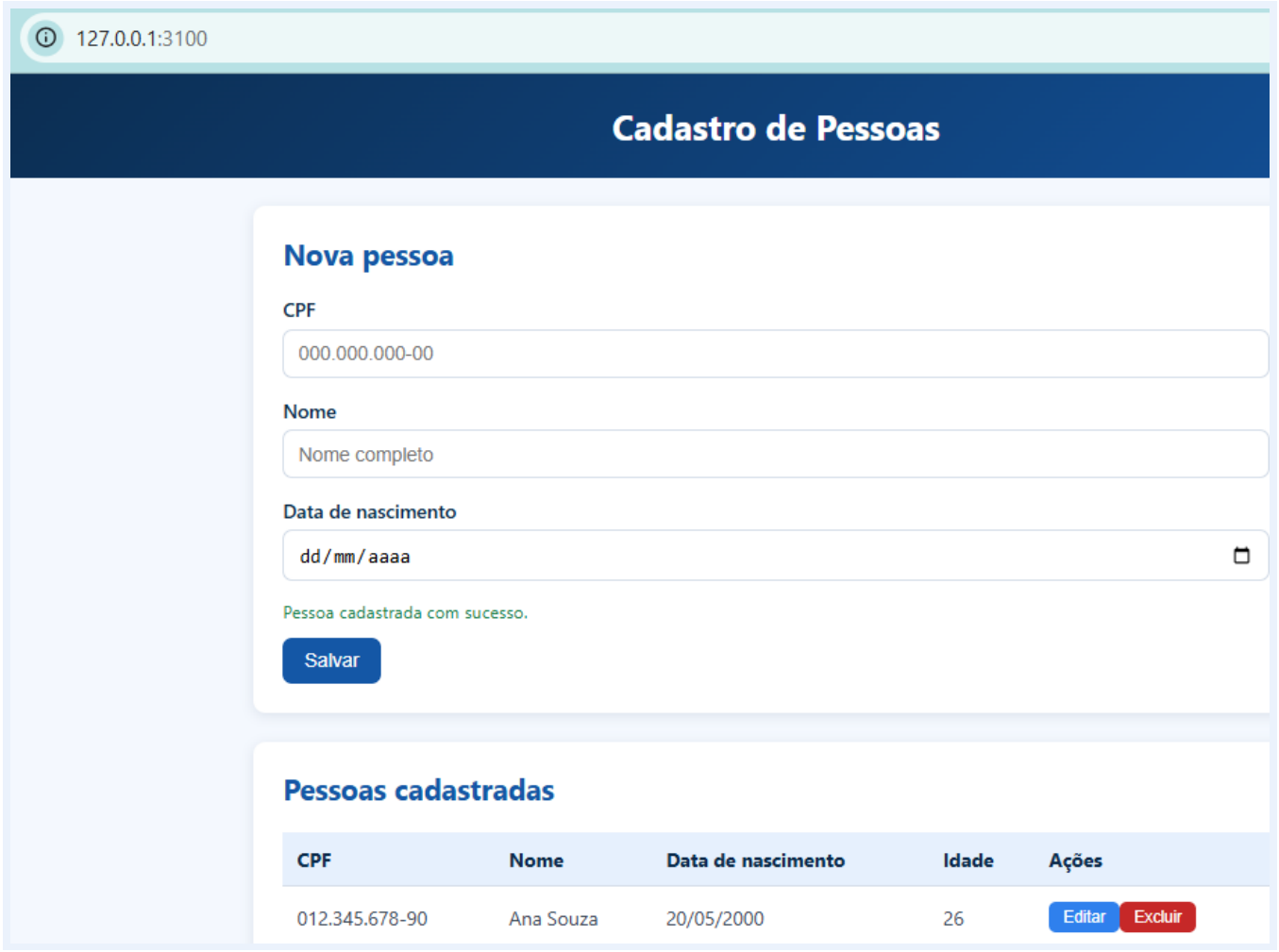


ETAPA 10

Cadastro e idade calculada

O cadastro fictício foi criado pela interface. O CPF, nome e nascimento aparecem na lista, com idade de 26 anos calculada pela aplicação.

92 - Cadastro realizado

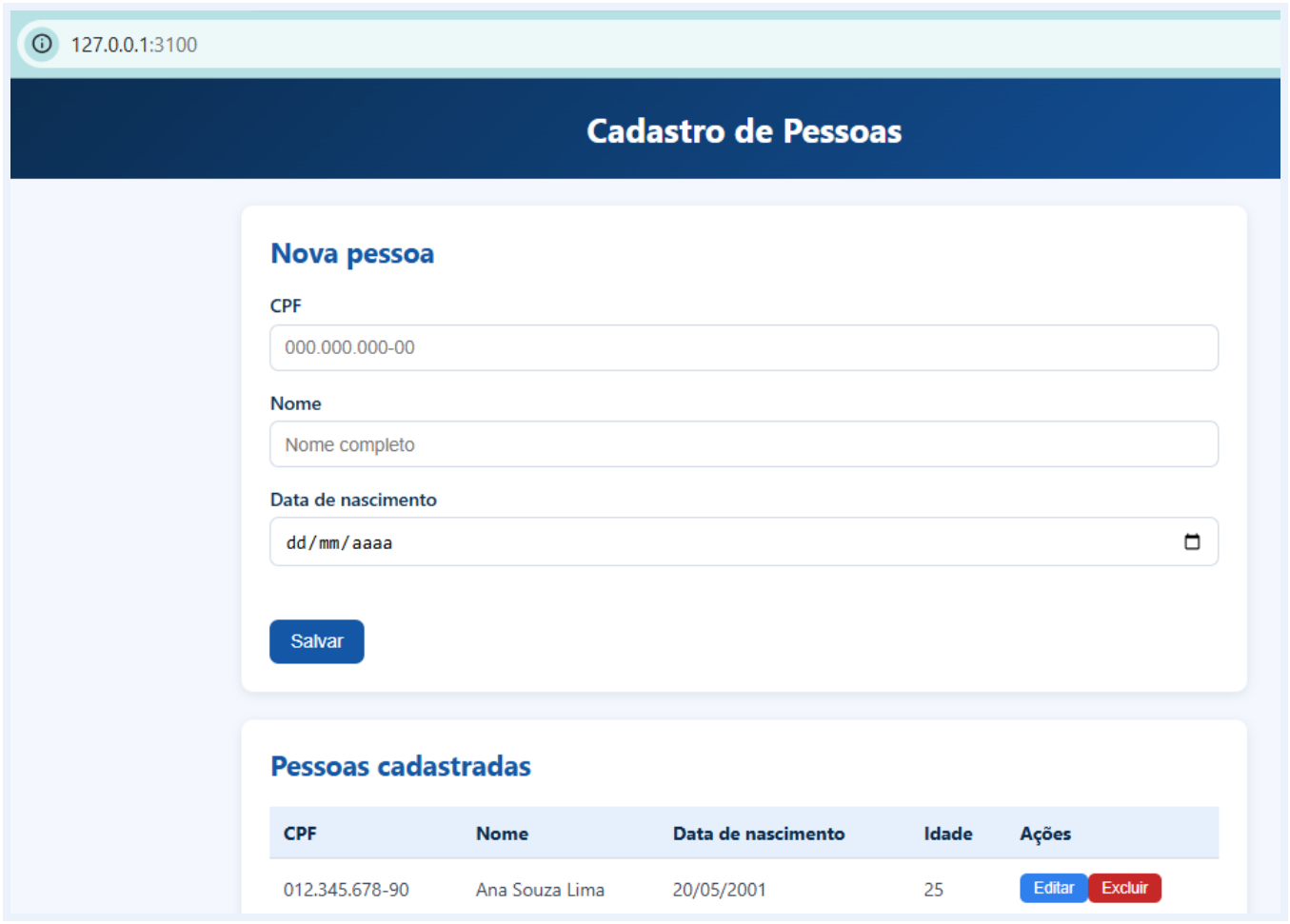


ETAPA 10

Edição do cadastro

Nome e nascimento foram alterados pela interface; o CPF permaneceu o mesmo. A idade exibida passou a 25 anos.

93 - Cadastro editado

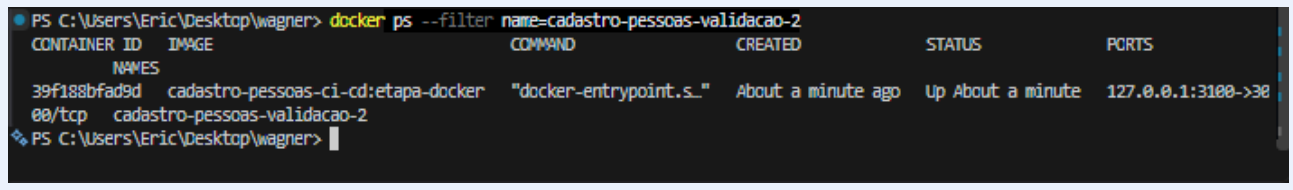


ETAPA 10

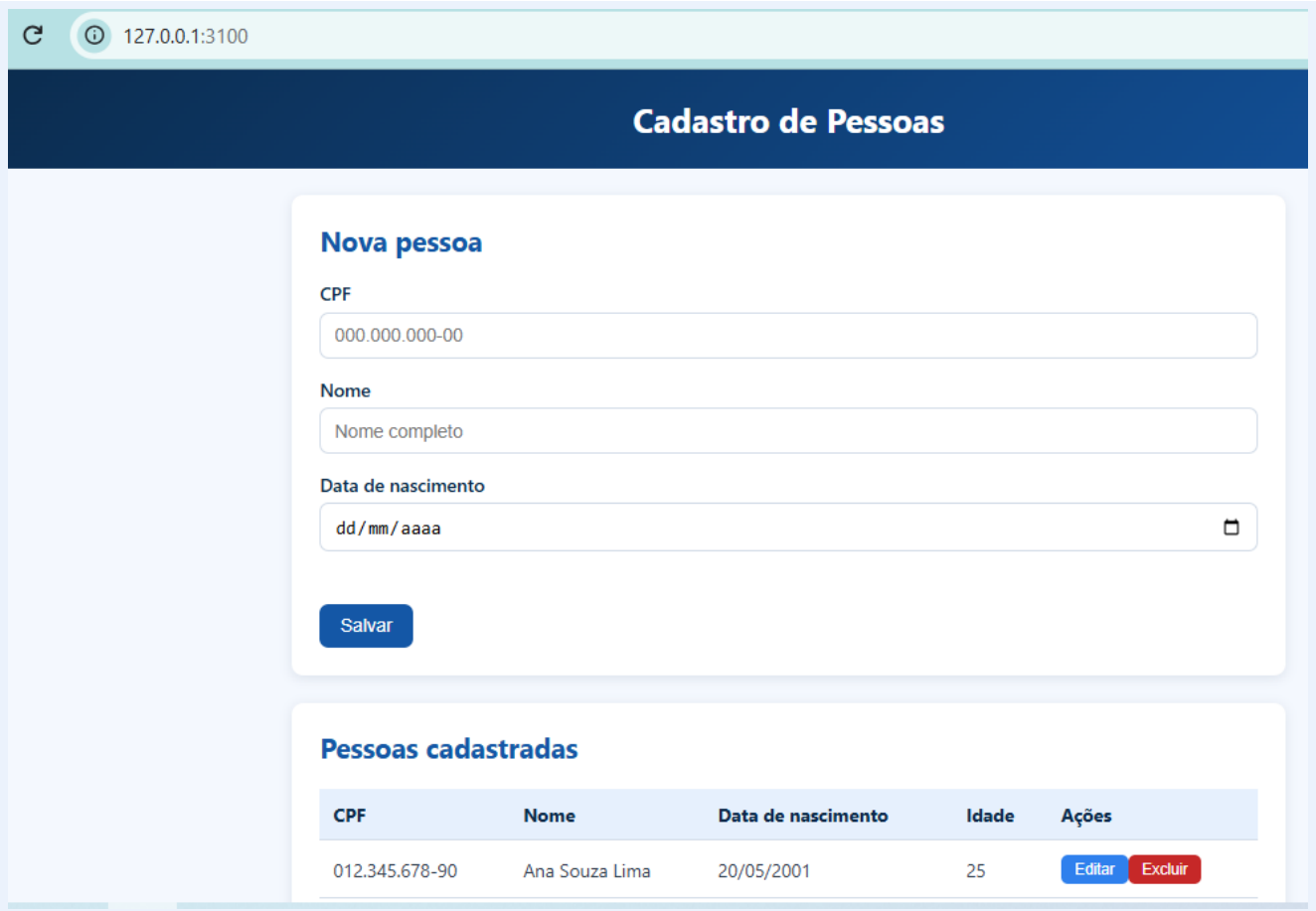
Persistência após recriar o contêiner

O primeiro contêiner foi removido. Outro contêiner da mesma imagem reutilizou o volume e, após atualizar a página, o cadastro editado continuou disponível.

94 - Segundo contêiner em execução



95 - Cadastro preservado

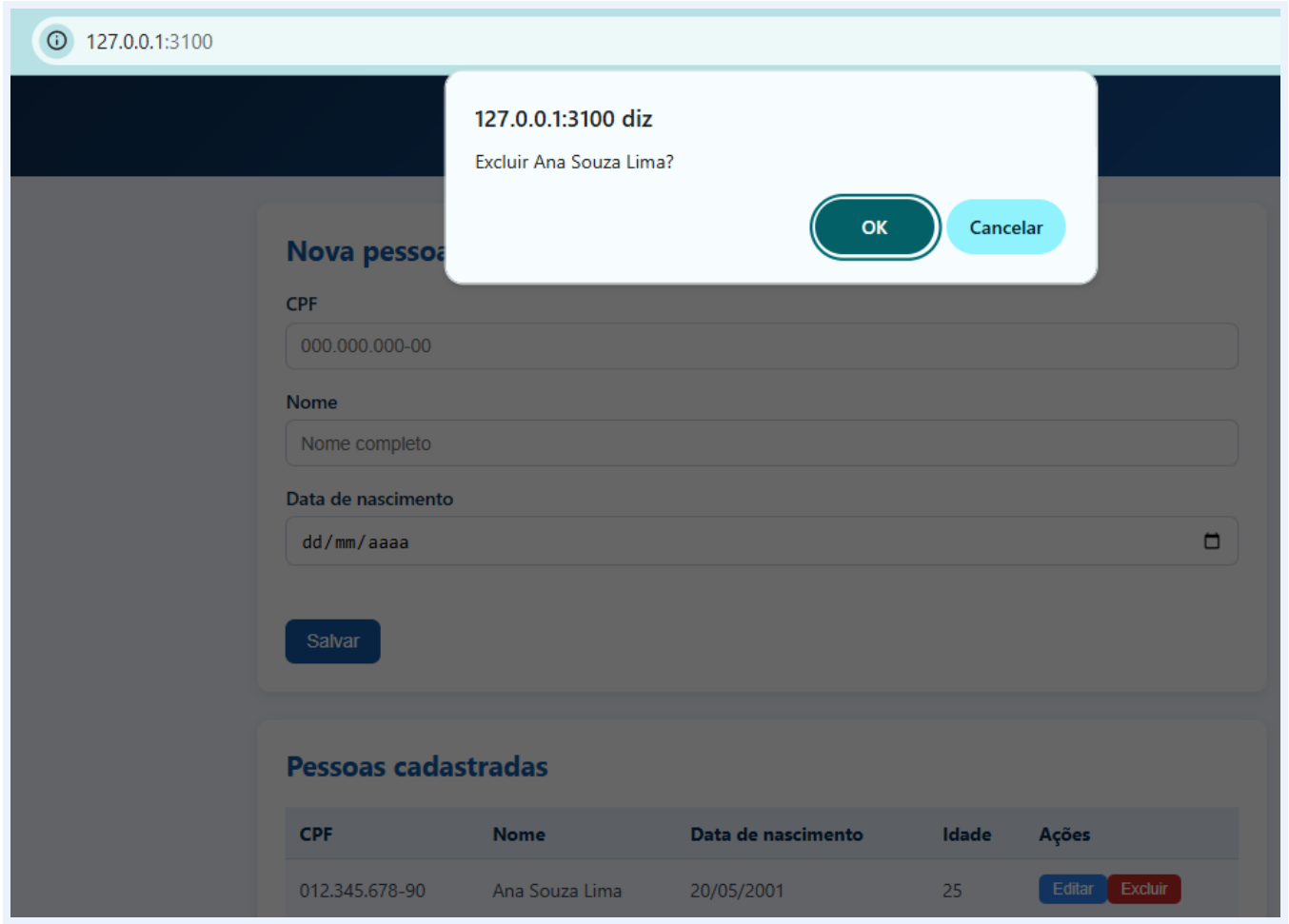


ETAPA 10

Confirmação da exclusão

A interface pediu confirmação antes de excluir o cadastro fictício. Esta captura registra o pedido, ainda não a exclusão concluída.

96 - Diálogo de confirmação

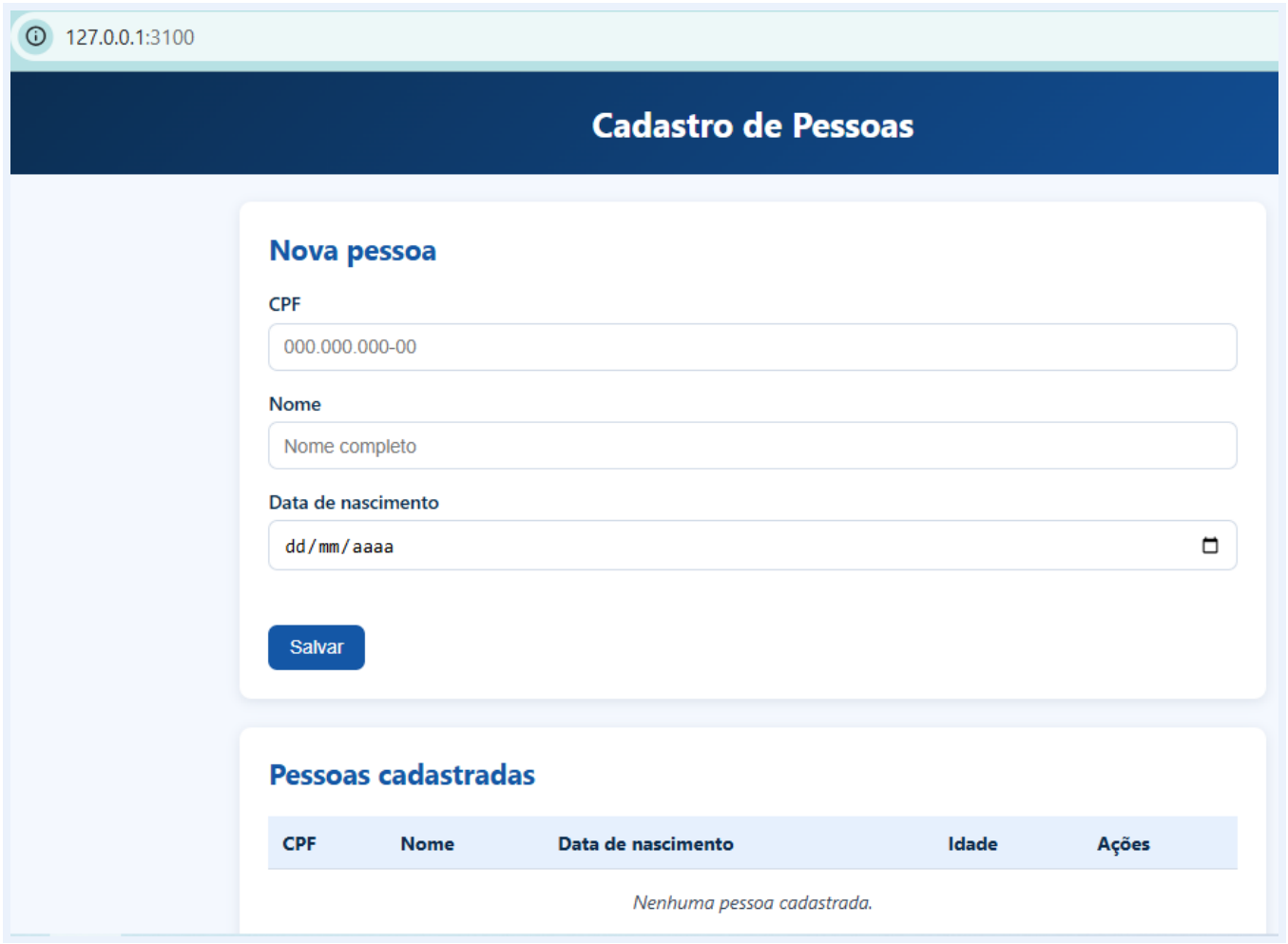


ETAPA 10

Exclusão concluída

Depois da confirmação e de atualizar a página, a lista ficou vazia. A consulta à API e o JSON do volume também retornaram uma lista vazia.

97 - Lista vazia após exclusão

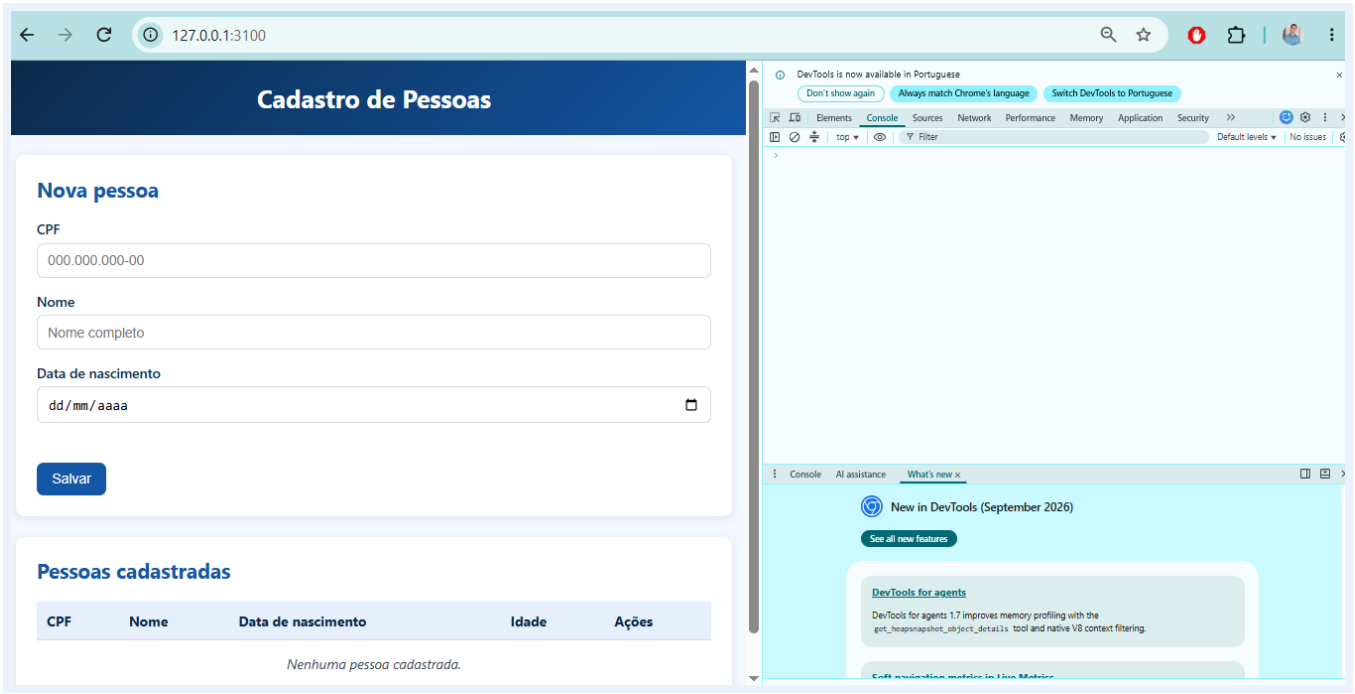


ETAPA 10

Console e logs sem erros observados

O console do navegador ficou sem erros visíveis após os fluxos. Os logs do contêiner mostraram apenas a mensagem de inicialização da API.

98 - Console do navegador

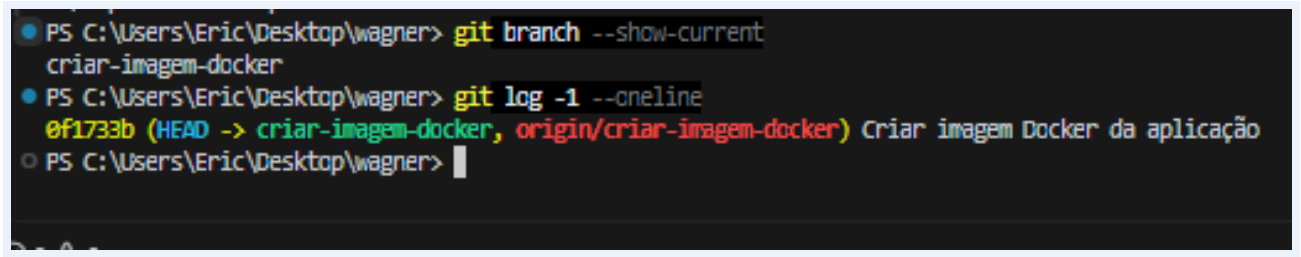


ETAPA 10

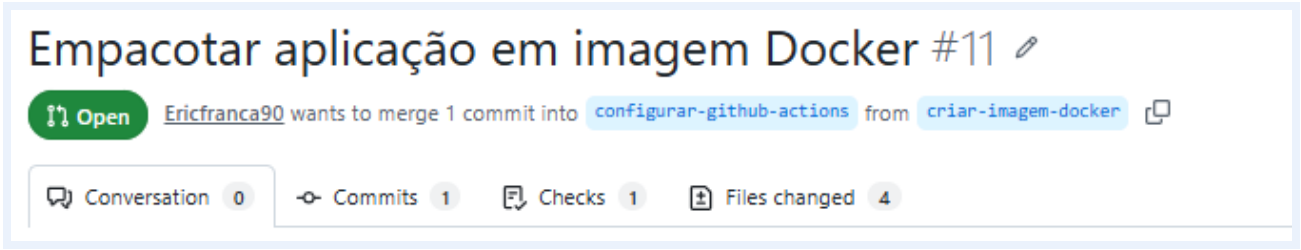
Commit e pull request

O commit do código foi publicado na branch criar-imagem-docker. O PR #11 permanece aberto contra configurar-github-actions e depende do PR #10.

99 - Branch e commit do código



100 - PR #11 aberto e base

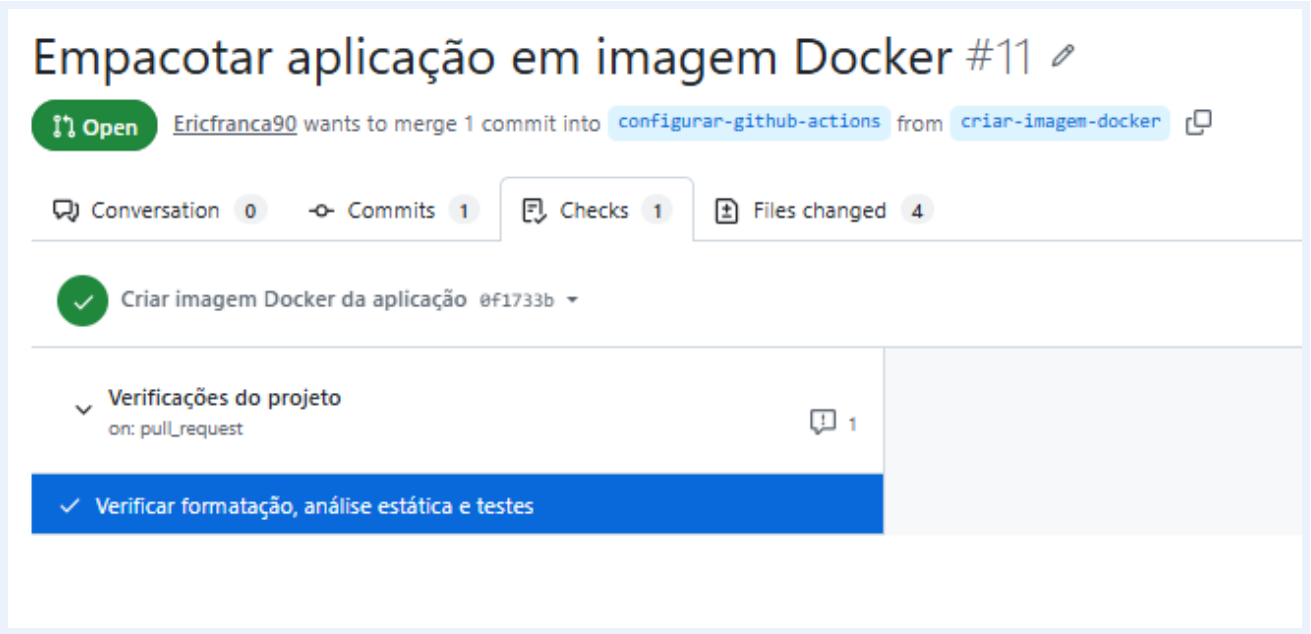


ETAPA 10

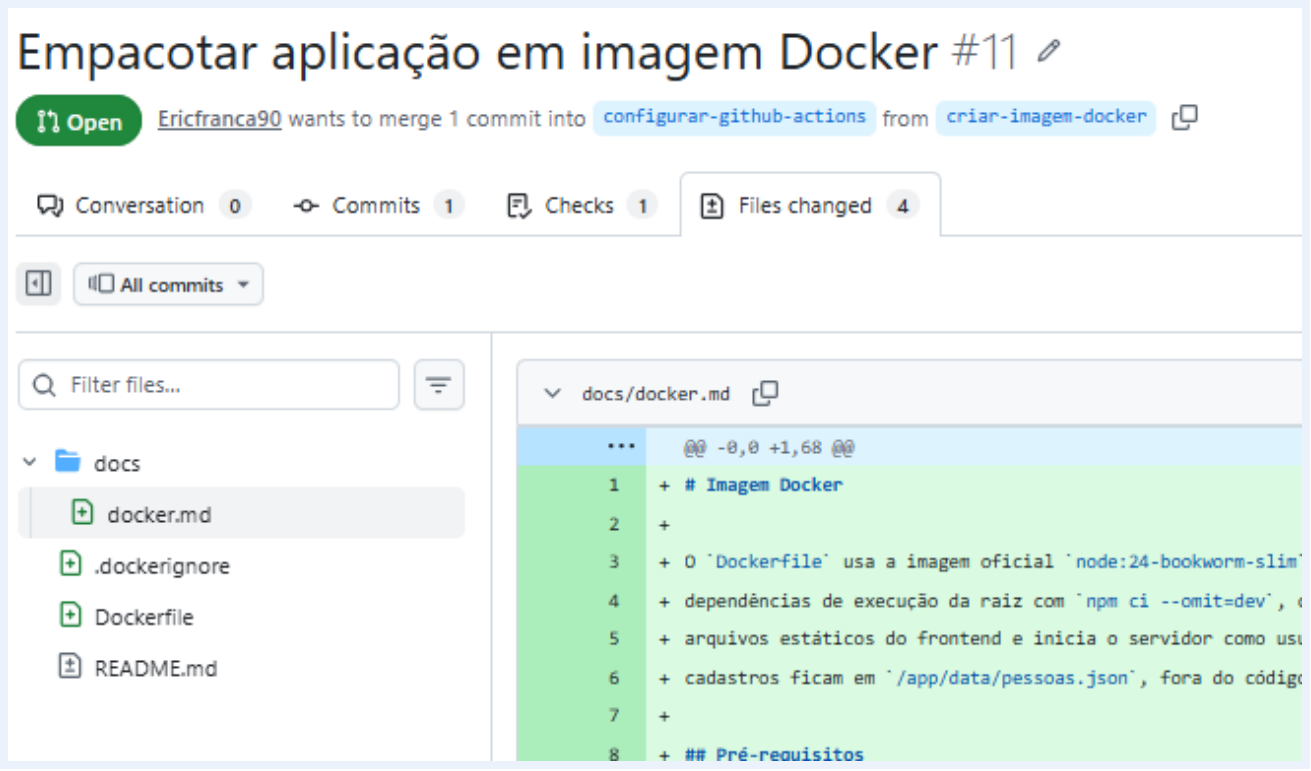
CI aprovada e escopo do PR

A verificação remota do PR #11 foi concluída com sucesso. Antes do commit documental, a comparação mostrava apenas Dockerfile, .dockerignore, README e documentação Docker.

101 - Check do GitHub Actions

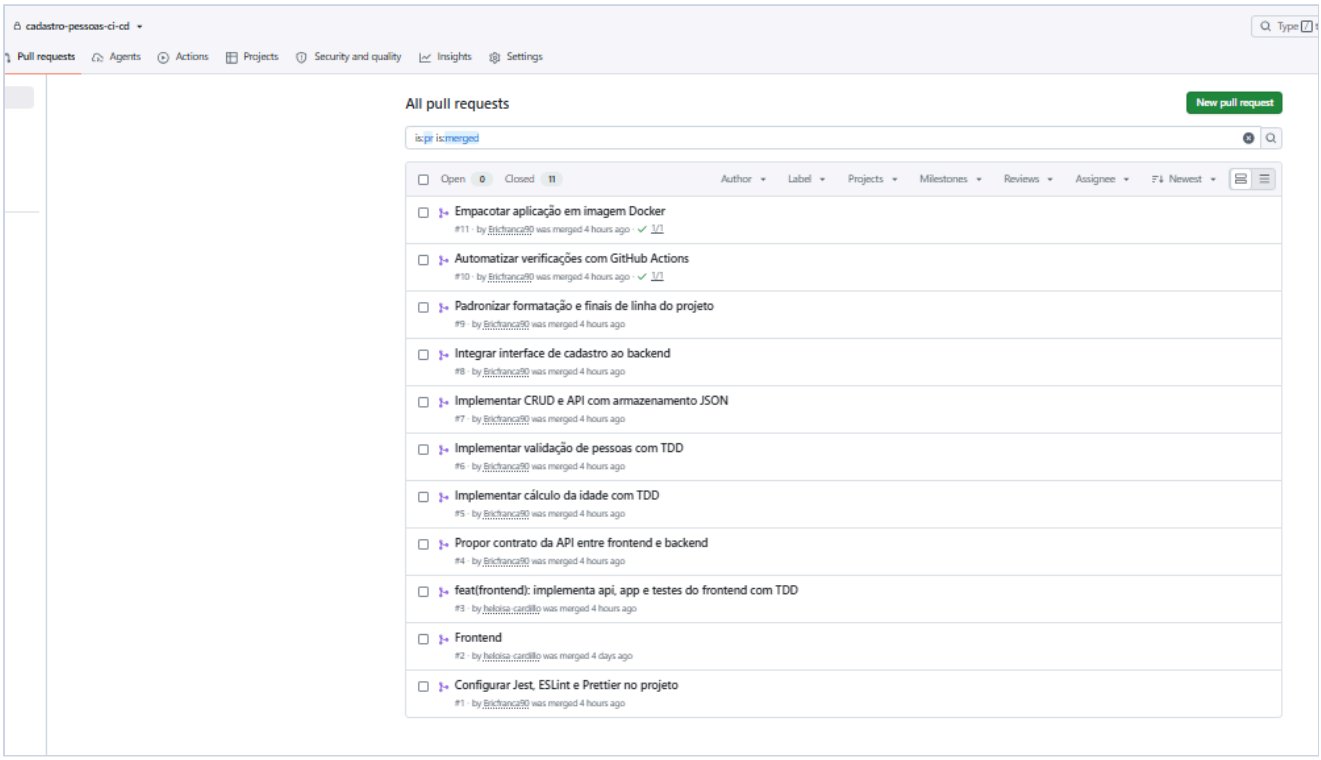


102 - Quatro arquivos alterados



Pull requests integrados

Após a simulação local documentada nas páginas anteriores, os 11 pull requests planejados foram mesclados na **main**. A captura mostra a lista real do GitHub, incluindo os trabalhos de frontend da Heloisa e as etapas de backend, testes, integração, formatação, CI e Docker.



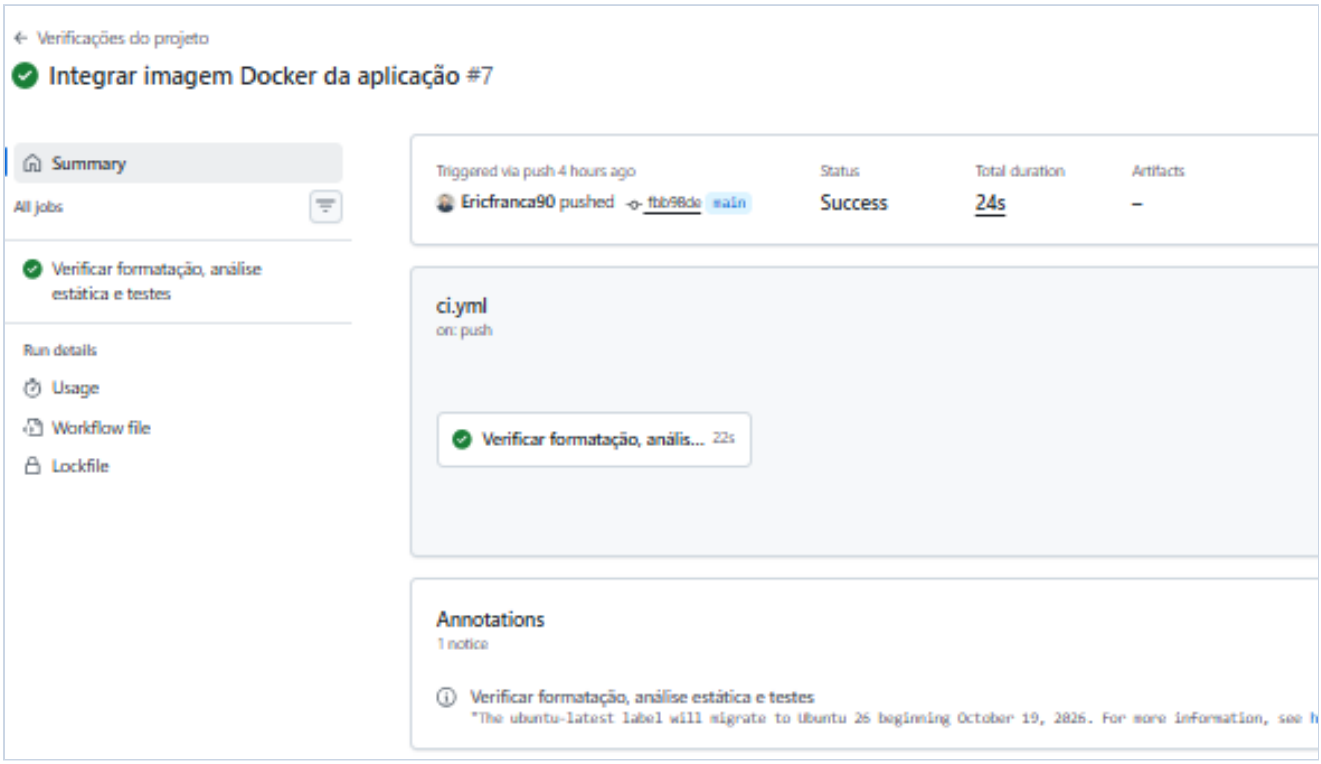
Print 119. Visão dos PRs mesclados no repositório. Os registros anteriores de PRs abertos representam a situação histórica de cada etapa, não o estado final.

> A integração preservou os commits originais dos participantes. Este print registra os merges reais; não deve ser confundido com a simulação local apresentada anteriormente.

FECHAMENTO DA ENTREGA · EVIDÊNCIA 120

CI aprovada na main

O GitHub Actions executou o workflow após a integração e concluiu com sucesso no commit funcional **fbb98de** da **main**. Esta é uma execução remota real, distinta dos testes locais e dos checks realizados nos pull requests.



Print 120. Resumo da execução final com resultado verde, branch main e commit fbb98de. Execução: github.com/Ericfranca90/cadastro-pessoas-ci-cd/actions/runs/39942796274.

> Na combinação integrada **ef02ed3**, Prettier, ESLint, 46 testes do backend e 24 do frontend passaram localmente. A CI remota passou em **fbb98de**. A imagem Docker desse commit foi construída e validada com persistência; não foi publicada em registro.